

A woman with long blonde hair, wearing a blue blazer over a white shirt, is smiling and holding a document labeled 'Resume'. She is standing in a modern office with large windows in the background.

Constitutional AI Fabric

ORACLE®

Platinum
Partner

Constitutional ERP is an AI-executed, human-governed enterprise system built on an immutable constitutional fabric that guarantees process integrity, authority control, and reconstructable operations across a distributed mesh.

White Paper

Table of contents

Category Description: Constitutional ERP	5
Key Characteristics of Constitutional ERP	5
Market Drivers	5
Category Definition: Constitutional ERP	6
Context	7
Enterprise Constitution — High-Level System Diagram	8
Top-Level Project & Technology Names	10
1. The Governance Layer (Constitutional Core)	10
2. The Process Layer (State + Hypermedia)	10
3. The AI Execution Layer (Navigator)	10
4. The Temporal Layer (Events, Rollback, Replay)	10
5. The Mesh Layer (Distributed Fabric)	11
6. The Integration Layer (ERP Overlay + Orchestration)	11
7. The ERP Core (Optional AI-Built Open Source ERP)	11
8. The UX Layer (Open-Flow, Role-less Interface)	12
9. The Authority Layer (Earned Authority System)	12
Final Naming Map (Complete System)	13
Narrative	14
1. Overall shape	14
2. UX model: process-driven, AI-navigated	14
3. Governance: domains, earned authority, constitutional limits	14
4. Temporal engine: events, rollback, replay	15
5. Mesh + Integration Hub: ERP-independent truth	15
6. AI-built HATEOAS ERP core (optional but powerful)	16
7. Inverted human–AI relationship in practice	16
Processes	18
Order To Cash	18
Role of MCP and AI in detail	23
Navigator Using MCP to Drive Execution	28
How Governance Modifies Hypermedia Responses	30
What This Diagram Shows (and Why It Matters)	33
Appendix	35
Developer Design	35

Integration Hub — Component-Level Architecture	40
1. Hypermedia Engine.....	40
2. MCP Function Publisher.....	41
1. Hypermedia REST API Contract (HATEOAS)	48
1. What the Mesh + Ledger must guarantee	57
Why MCP is the correct choice for Enterprise Constitution	65
1. MCP function model	70
✖ MCP ≠ HATEOAS — They Solve Different Problems.....	75
Foundation ERP (Dummy).....	78
🌱 1. The Mini-Foundational ERP must be a “scaffold”, not a toy	85
🧠 1. Why a “Super-O2C” is the right move	92
Mermaid	113
Process	222
Architecture.....	223
Hypermedia layer	228
Mermaid Sequence Diagram — Navigator Using MCP to Drive Execution	230
Thank You	231

Constitutional ERP

(Open-Source Project)

- **AI executes**
- **Humans govern**
- **Systems endure**

Category Description: Constitutional ERP

Constitutional ERP is an emerging class of enterprise platforms that combine AI-driven operational execution with governance-anchored control frameworks to deliver resilient, autonomous, and reconstructable business systems. Unlike traditional ERP suites, which rely on role-based workflows and monolithic data models, Constitutional ERPs operate on a distributed constitutional fabric that enforces non-bypassable rules, process integrity, and earned authority across all business domains.

Constitutional ERPs use AI as the primary execution engine, interpreting process state, proposing next actions, and automating routine operations. Human users act as governors rather than operators, providing oversight, approvals, and corrective interventions. A constitutional layer — composed of immutable rules, domain constraints, and authority models — ensures that neither AI nor human actors can violate enterprise policy, regulatory requirements, or process integrity.

At the architectural level, Constitutional ERPs are defined by event-sourced temporal ledgers, distributed mesh fabrics, and hypermedia-driven process graphs. These systems maintain a complete, immutable record of all operational events, enabling full rollback, replay, and system reconstruction across heterogeneous ERP backends. This allows organizations to operate with unprecedented resilience, auditability, and vendor independence.

Gartner expects Constitutional ERP platforms to be adopted first in industries with high regulatory exposure, complex multi-entity operations, or strong requirements for auditability and operational continuity. Over time, the category is likely to expand into mainstream enterprise operations as organizations seek to modernize legacy ERP estates, reduce operational overhead, and adopt AI-driven execution models without sacrificing governance or control.

Key Characteristics of Constitutional ERP

- **AI-Driven Execution:**
AI navigates processes, proposes and executes actions, and provides explainability.
- **Human-Anchored Governance:**
Users act as approvers and governors, with authority determined by earned credentials and risk tiers.
- **Constitutional Control Layer:**
Immutable rules and domain constraints enforce compliance and prevent unauthorized actions.
- **Event-Sourced Temporal Integrity:**
All operations are recorded as immutable events, enabling rollback, replay, and full system reconstruction.
- **Mesh-Native Architecture:**
Distributed fabric ensures resilience, continuity, and multi-ERP interoperability.
- **Process-First UX:**
Interfaces expose state-driven affordances rather than role-based menus or modules.

Market Drivers

- Rising demand for **AI-enabled operational automation**
- Increasing regulatory pressure for **auditability and traceability**
- Need for **resilient, reconstructable enterprise systems**
- Desire to reduce dependency on monolithic ERP vendors
- Shift toward **distributed, multi-entity operating models**

Category Definition: Constitutional ERP

Constitutional ERP is a new category of enterprise system in which **AI executes operations**, **humans govern decisions**, and a **constitutional fabric** enforces the rules that neither can bypass.

It replaces role-based interfaces and monolithic back-office systems with a **process-first, state-driven architecture** built on immutable events, distributed mesh resilience, and earned authority.

A Constitutional ERP is defined by five core principles:

1. AI-Driven Execution

The system's primary operator is an AI Navigator that interprets process state, proposes next actions, executes transitions, and explains its reasoning. Humans intervene only where governance requires it.

2. Human-Anchored Governance

Authority is earned, contextual, and revocable. Humans approve, correct, and oversee — but do not manually drive every step. Governance is structural, not procedural.

3. Constitutional Constraints

A Charter Engine enforces immutable rules, domain boundaries, and non-bypassable limits. Neither AI nor humans can violate the constitution of the enterprise.

4. Temporal Integrity

All operations are recorded as events in a Ledger that supports versioning, rollback, and replay. Any system state — including external ERPs — can be reconstructed from the constitutional record.

5. Mesh-Native Architecture

A distributed Mesh Fabric ensures resilience, continuity, and multi-system orchestration. ERPs become projections on the mesh, not the source of truth.

Context

AI-driven UX, process-first, domain-governed, event-sourced, mesh-backed, ERP-agnostic.

1. AI Drives Execution

- Proposes actions at every step
- Navigates the process graph
- Explains decisions
- Simulates outcomes

2. Humans Govern

- Approve actions
- Provide authority
- Correct errors
- Trigger rollback/replay

3. Mesh Is the Constitutional Truth

- Records every event
- Enforces governance
- Supports rollback + replay
- Reconstructs ERP state

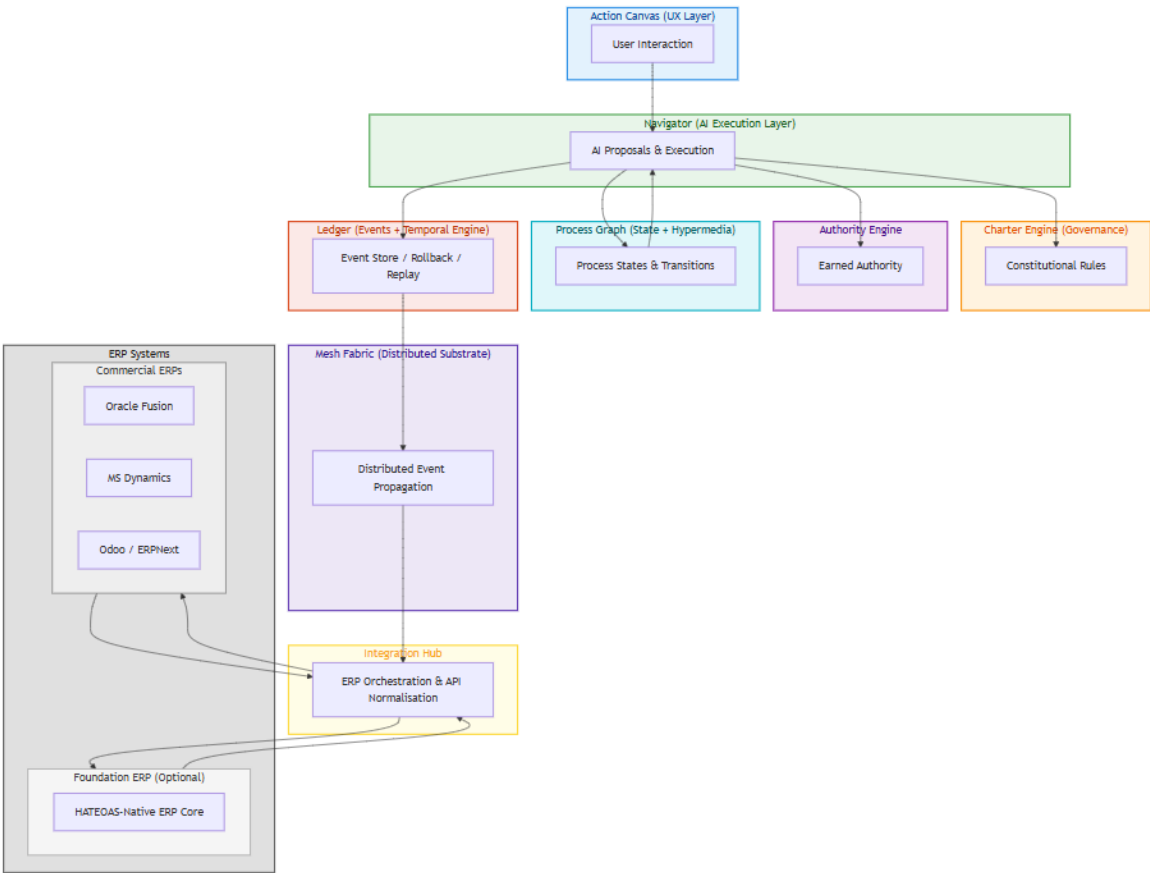
4. ERP Is a Projection

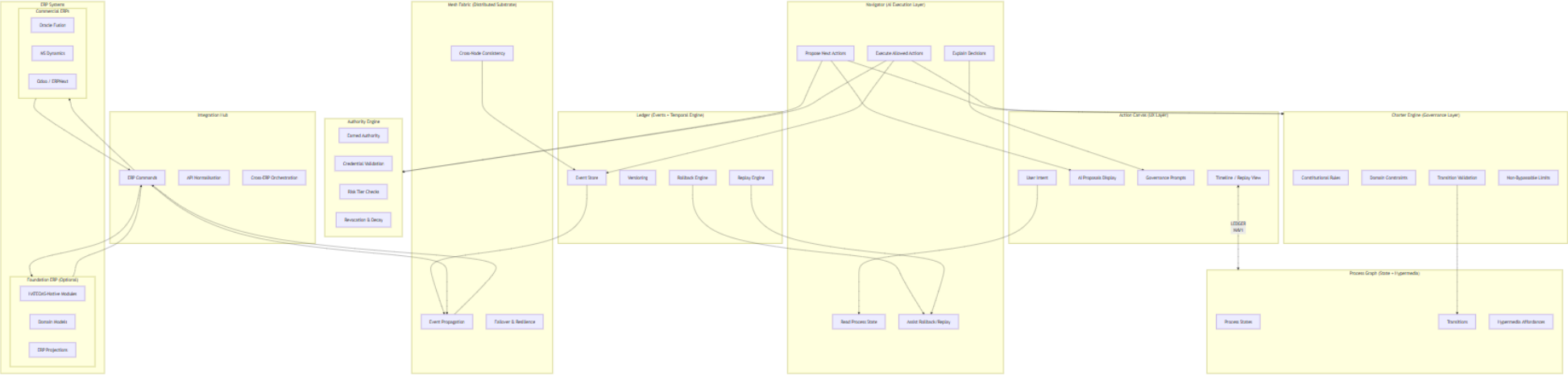
- Executes commands
- Posts transactions
- Can be replaced or rebuilt

5. UX Is Process-First

- No roles
- No menus
- No modules
- Only “what is possible next”

Enterprise Constitution — High-Level System Diagram





Top-Level Project & Technology Names

1. The Governance Layer (Constitutional Core)

This is the layer that enforces rules, constraints, and authority.

Subsystem Name: The Charter Engine

Purpose:

- Holds immutable rules
- Defines domains of governance
- Enforces constitutional constraints
- Validates authority and permissions

Why “Charter”?

A charter is the foundational document that defines powers and limits — perfect metaphor.

2. The Process Layer (State + Hypermedia)

This is the process graph, transitions, and HATEOAS affordances.

Subsystem Name: The Process Graph

(or **Process Graph Engine**)

Purpose:

- Defines all process states
- Exposes hypermedia transitions
- Drives the open-flow UX
- Provides the “what is possible next” model

Why “Graph”?

It’s literal, accurate, and future-proof.

3. The AI Execution Layer (Navigator)

This is the AI that drives execution, proposes actions, and explains decisions.

Subsystem Name: The Navigator

(or **AI Navigator**)

Purpose:

- Reads process state
- Proposes next actions
- Executes transitions
- Explains decisions
- Assists with replay

Why “Navigator”?

It captures the inversion of control: AI steers, humans govern.

4. The Temporal Layer (Events, Rollback, Replay)

This is the event-sourced backbone.

Subsystem Name: The Ledger

(or **Temporal Ledger, Event Ledger**)

Purpose:

- Append-only event store

- Versioning
- Rollback
- Replay
- Reconstructability

Why “Ledger”?

It’s the perfect metaphor for immutable, auditable history.

5. The Mesh Layer (Distributed Fabric)

This is the distributed substrate that ties everything together.

Subsystem Name: The Mesh Fabric

(or **Continuity Mesh**, if you want to keep that brand)

Purpose:

- Distributed event propagation
- Cross-node consistency
- Failover and resilience
- Multi-ERP orchestration

Why “Fabric”?

It conveys interwoven, resilient, distributed structure.

6. The Integration Layer (ERP Overlay + Orchestration)

This is the layer that connects to Oracle, Dynamics, Odoo, or your own ERP.

Subsystem Name: The Integration Hub

(or **Constitutional Integration Hub**)

Purpose:

- Orchestrates commands to ERPs
- Normalises APIs
- Provides ERP-agnostic abstraction
- Enables reconstructability

Why “Hub”?

It’s the central point of coordination between constitutional logic and external systems.

7. The ERP Core (Optional AI-Built Open Source ERP)

This is the HATEOAS-native, AI-maintainable ERP you may build.

Subsystem Name: The Foundation ERP

(or **Constitutional ERP Core, Open Constitution ERP**)

Purpose:

- Provides domain models
- Implements core financials, inventory, HR, etc.
- Exposes hypermedia affordances
- Is fully reconstructable from the Ledger

Why “Foundation”?

It’s the base layer beneath the constitutional fabric.

8. The UX Layer (Open-Flow, Role-less Interface)

This is the user-facing experience.

Subsystem Name: The Action Canvas

(or **OpenFlow UX, Process Canvas**)

Purpose:

- Shows “what is possible next”
- Displays AI proposals
- Surfaces governance requirements
- Provides timeline + replay views

Why “Canvas”?

It conveys openness, flexibility, and creativity — no menus, no roles, just actions.

9. The Authority Layer (Earned Authority System)

This is the system that manages trust, credentials, and authority.

Subsystem Name: The Authority Engine

(or **Authority Graph, Credential Engine**)

Purpose:

- Tracks earned authority
- Validates domain permissions
- Applies behavioural scoring
- Supports revocation and decay

Why “Authority Engine”?

It’s clear, direct, and governance-aligned.

Final Naming Map (Complete System)

Layer	Subsystem Name	Purpose
Governance	Charter Engine	Constitutional rules & constraints
Process	Process Graph	State + transitions + hypermedia
AI Execution	Navigator	AI-driven action sequencing
Temporal	Ledger	Events, rollback, replay
Mesh	Mesh Fabric	Distributed substrate
Integration	Integration Hub	ERP orchestration
ERP Core	Foundation ERP	HATEOAS-native ERP
UX	Action Canvas	Open-flow, role-less UX
Authority	Authority Engine	Earned authority & governance

Narrative

AI-driven UX: no role-based menus; process-driven affordances.

Inverted human–AI relationship: AI executes; humans permit, govern, and correct.

Meshed backend: Integration Hub + one or more ERPs (commercial, open source, or AI-built).

Event-centric core: versioning, rollback, replay, reconstructability.

HATEOAS contract: everything discoverable and drivable via hypermedia.

1. Overall shape

You're designing a **constitutional ERP fabric**, not just a UI:

- **AI-driven UX:** no role-based menus; process-driven affordances.
- **Inverted human–AI relationship:** AI executes; humans permit, govern, and correct.
- **Meshed backend:** Integration Hub + one or more ERPs (commercial, open source, or AI-built).
- **Event-centric core:** versioning, rollback, replay, reconstructability.
- **HATEOAS contract:** everything discoverable and drivable via hypermedia.

Think of it as:

UX + AI + Mesh as the “constitution”; ERP(s) as replaceable “governments”.

2. UX model: process-driven, AI-navigated

Principle:

The UX shows *what is possible next*, not *who you are*.

- **Process graph** defines states and transitions.
- **Hypermedia affordances** expose:
 - available actions,
 - required data,
 - governance requirements,
 - risk/impact hints.
- **AI navigator:**
 - proposes next actions,
 - sequences workflows,
 - explains options in plain language,
 - simulates consequences.

Users:

- see the whole process,
- choose from AI-proposed actions,
- approve/deny where they have authority.

No static roles; the interface is **state + governance driven**.

3. Governance: domains, earned authority, constitutional limits

Replace roles with:

- **Domains of governance** (e.g. Financial Commitments, Inventory Movement, Workforce Changes).
 - **Authority levels** within each domain (thresholds, risk tiers, approval scopes).
-

- **Earned authority:**
 - granted via training, history, behaviour,
 - decays or is revoked,
 - fully auditable.

Constitutional layer:

- Immutable rules like:
 - no self-benefit approvals,
 - no skipping mandatory transitions,
 - no AI or human can modify constitutional rules via normal flows.
- Separation of powers:
 - AI proposes/executes,
 - governance engine constrains,
 - humans approve/correct,
 - audit/analytics observe.

AI is powerful but **never sovereign**.

4. Temporal engine: events, rollback, replay

Event sourcing backbone:

- Every change = event:
 - AIProposedAction, UserApproved, PaymentExecuted, RollbackInitiated, etc.
- Current state = fold(events) (+ snapshots for performance).
- Temporal graph tracks:
 - entities (orders, invoices, contracts, accounts),
 - relationships over time.

Rollback:

- Logical rollback via compensating events, not history deletion.
- “Revert to T_0 ” = recompute effective state as if certain events didn’t happen.

Replay:

- Re-run events from $T_0 \rightarrow$ now:
 - at machine speed,
 - with corrected/omitted steps,
 - with AI either deterministic or explicitly re-evaluating.
- External side-effects guarded via idempotency and explicit replay semantics.

UX surfaces:

- timelines,
- before/after diffs,
- “simulate replay” before commit.

This makes “AI did the wrong thing but we can fix it” a **first-class capability**.

5. Mesh + Integration Hub: ERP-independent truth

Mesh responsibilities:

- Holds the **canonical event history**.
 - Owns **process graphs**, governance, and AI navigation.
 - Publishes/consumes events across nodes (Continuity Mesh).
 - Can **reconstruct** any attached ERP from events.
-

Integration Hub node:

- Orchestrates:
 - cross-ERP flows,
 - cross-org flows,
 - failover and continuity.
- Talks to:
 - commercial ERPs (Oracle, Dynamics, SAP),
 - open source ERP core (e.g. AI-built HATEOAS ERP),
 - other services.

ERPs become:

- **projections and executors**, not the constitutional source of truth.
 - Swappable: migrate by replaying mesh events into a new backend.
-

6. AI-built HATEOAS ERP core (optional but powerful)

On the mesh you can host:

- an **open source, HATEOAS-native ERP core**, AI-generated/maintained.
- Based on:
 - existing OSS ERP (ERPNext/Odoo) as functional reference,
 - strict modularity,
 - stable hypermedia + OpenAPI contracts.

AI works module-by-module:

- generates/maintains code,
- tests via integration-level scenarios,
- never needs full-code context.

This gives you:

- a long-term path away from legacy ERPs,
 - without changing UX, governance, or mesh semantics.
-

7. Inverted human–AI relationship in practice

Putting it all together:

1. **AI reads process state + hypermedia affordances** from the mesh.
2. **AI proposes and executes** allowed actions, subject to:
 - domain constraints,
 - authority requirements,
 - constitutional rules.
3. **Humans:**
 - grant permission (where required),
 - set governance policies,
 - correct via rollback/replay,
 - earn and lose authority.
4. **Mesh:**
 - records everything as events,
 - can reconstruct any backend,
 - can recover from AI or human mistakes.

The UX is:

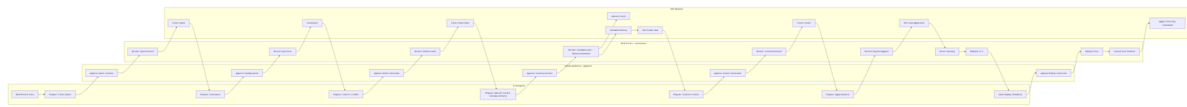
“Here’s what can happen next, here’s what I (AI) recommend, here’s why, and here’s what it will do to the system. Do you permit this?”

And underneath, the fabric is:

ERP-independent, reconstructable, governable, and resilient.

Processes

Order To Cash



Here's a clean, concrete **Order-to-Cash (O2C)** walkthrough inside the architecture we've designed — **AI-driven UX, process-first, domain-governed, event-sourced, mesh-backed, ERP-agnostic.**

This is the first time the whole thing comes together in a single narrative. You'll see how every layer — UX, AI, governance, mesh, ERP — plays its part.

Order-to-Cash in the New Model

(AI drives execution; humans govern; mesh is the constitutional truth; ERP is a projection.)

0. Starting Point: A Customer Wants to Buy Something

A salesperson opens the UX.

There are **no menus, no roles, no modules.**

The screen shows:

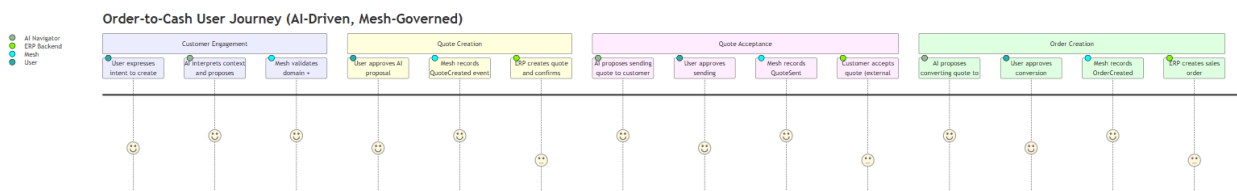
- the **customer entity**
- its **current state**
- **hypermedia affordances** (possible next actions)
- AI's recommended next step

Example affordances:

- "Create Quote"
- "Check Credit Status"
- "Simulate Delivery Date"
- "Review Contract Terms"

AI says:

"Based on the customer's history and your intent, I recommend creating a quote."



1. AI Proposes → Human Permits → Mesh Records

AI proposes:

AIProposedAction: CreateQuote(customer=ACME)

Governance checks:

- Domain: *Commercial Commitments*
- Authority required: *Low*
- User has earned authority → OK

Human approves:

UserApproved(action=CreateQuote)

Mesh records event:

QuoteCreated(quoteId=Q123, customer=ACME)

ERP receives command:

POST /quotes { ... }

ERP confirms → mesh records: ERPConfirmed(quoteId=Q123)

2. AI Drives the Quote → Order Transition

The quote is now in state Draft.

Hypermedia affordances show:

- “Add Line Item”
- “Apply Discount”
- “Send to Customer”
- “Convert to Order”

AI says:

“The customer has accepted similar quotes before. I recommend sending this quote for approval.”

User approves → mesh records → ERP executes.

Customer accepts → event: CustomerAcceptedQuote(Q123)

AI proposes:

“Convert quote Q123 to sales order.”

Human approves → mesh records → ERP creates order.

3. Order → Fulfilment

Order is now in state Confirmed.

Affordances:

- “Allocate Stock”
- “Schedule Delivery”
- “Generate Pick List”
- “Simulate Fulfilment Timeline”

AI proposes:

“Allocate stock and schedule delivery for Friday.”

Governance checks:

- Domain: *Inventory Movement*
- Authority required: *Medium*
- User has authority → OK

Mesh records:

StockAllocated(order=SO456)

DeliveryScheduled(order=SO456, date=Friday)

ERP executes both.

4. Goods Issue → Invoice

Warehouse confirms pick + pack → events:

GoodsPicked(SO456)

GoodsShipped(SO456)

AI proposes:

“Generate invoice for shipped goods.”

Governance:

- Domain: *Financial Commitments*
- Authority required: *High*
- User does NOT have authority.

UX shows:

- “You cannot approve this action.”
- “Authority required: Financial Tier 2.”
- “Request approval?”

User requests → approver receives AI-summarised context → approves.

Mesh records: InvoiceGenerated(INV789)

ERP posts invoice.

5. Payment → Cash Application

Customer pays → bank feed → event: PaymentReceived(PMT001, amount=...)

AI proposes:

“Apply payment PMT001 to invoice INV789.”

Governance:

- Domain: *Cash Application*
- Authority required: *Low*
- Auto-approval allowed (earned authority + low risk)

Mesh records: PaymentApplied(PMT001 → INV789)

ERP posts cash application.

6. Something Goes Wrong (The Interesting Part)

Later, audit detects an anomaly:

- The delivery date was wrong
- The invoice amount is incorrect
- Or the AI misinterpreted a discount rule

Auditor selects:

“Rollback to state before InvoiceGenerated(INV789).”

Mesh computes state at T_0 .

Mesh simulates replay:

- Replays all events from T_0 → now
- Except the incorrect discount application
- AI re-evaluates pricing with corrected rules
- New invoice amount is generated
- Payment is re-applied correctly

Auditor reviews diff:

- Old invoice: 10,000
 - New invoice: 9,500
 - Payment difference: 500 credit
-

Approves replay.
Mesh commits new timeline.
ERP receives correcting commands.

7. End State: Clean, Audited, Reconciled

The O2C flow is:

- fully traceable
- fully reconstructable
- fully governed
- AI-driven but human-legitimated
- ERP-agnostic
- resilient to mistakes
- resilient to fraud
- resilient to drift

And the UX remains:

- simple
 - process-driven
 - AI-navigated
 - governance-aware
-

What This Demonstrates

This single flow shows:

AI drives execution

- proposes actions
- sequences workflows
- explains decisions
- simulates outcomes

Humans govern

- approve
- correct
- override
- earn authority

Mesh is the constitutional truth

- event-sourced
- rollback/replay
- reconstructable
- ERP-independent

ERP is a projection

- receives commands
- posts transactions
- can be replaced or rebuilt

UX is process-first

- no roles
 - no menus
 - no modules
 - only “what is possible next”
-

If you want, we can now sketch:

- **Procure-to-Pay**
- **Hire-to-Retire**
- **Incident-to-Resolution**
- **Asset-to-Retirement**
- **Manufacture-to-Delivery**

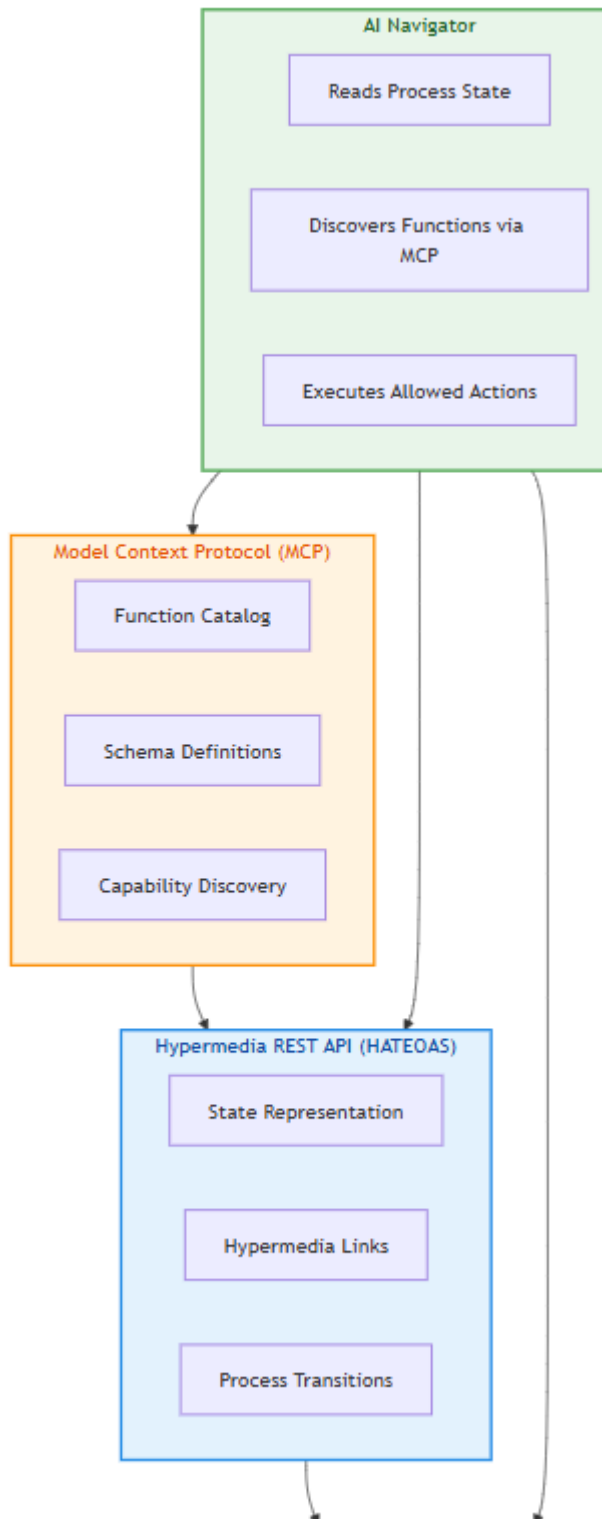
Or we can move to the next design layer:
earned authority, AI navigator internals, or constitutional constraints.

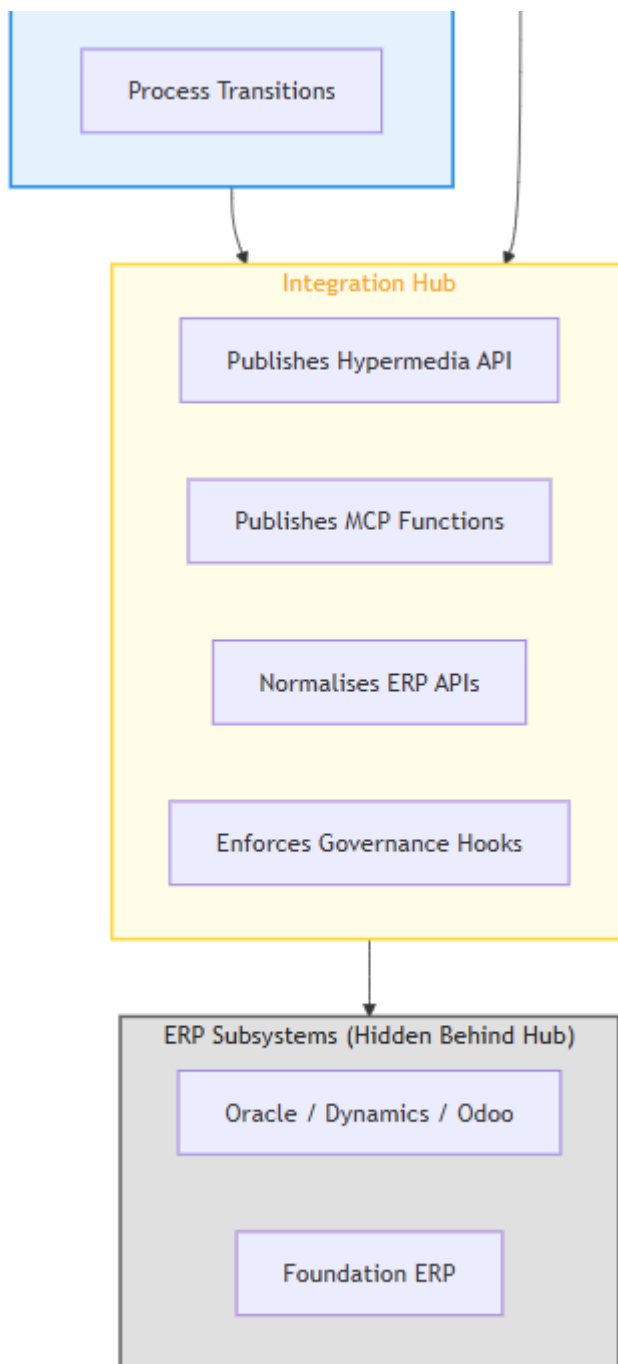
Role of MCP and AI in detail

Use MCP (Model Context Protocol) so the AI Navigator can *discover* and *understand* the available business functions exposed by the Integration Hub, which publishes a HATEOAS hypermedia REST API that overlays the ERP subsystems and hides their implementation details.

This is *exactly* how you make AI-driven execution safe, governable, and ERP-agnostic.

Let's break it down.





How to Read This Diagram

AI Navigator

- Uses **MCP** to *discover* what actions exist
- Uses **Hypermedia** to *understand* what actions are allowed in the current state
- Uses **Integration Hub** to *execute* actions safely

MCP Layer

- Publishes a **function catalog**
- Defines **schemas** for inputs/outputs
- Allows the AI to introspect capabilities

This is how the AI learns “what can be done”.

Hypermedia REST API

- Exposes **state**, **links**, and **transitions**

- Represents the **process graph** dynamically
- Ensures the AI can only follow valid transitions

This is how the AI learns “what is allowed next”.

Integration Hub

- Publishes both **MCP functions** and **Hypermedia affordances**
- Normalises ERP APIs
- Enforces governance constraints
- Acts as the **constitutional gateway**

This is how the AI executes actions safely.

ERP Layer

- Completely hidden
- Replaceable
- Reconstructable
- Treated as a projection

This is how you achieve ERP-agnostic autonomy.

★ Why this architecture is so strong

- AI becomes **self-configuring**
- Governance becomes **non-bypassable**
- ERP becomes **replaceable**
- Process becomes **discoverable**
- Execution becomes **safe**
- The system becomes **future-proof**

This is the cleanest way to implement an **AI-driven, constitutional enterprise system**.

✖ 1. Why MCP is the right mechanism for the AI Navigator

MCP gives the AI:

- **Function discovery**
 (“What can I do in this domain?”)
- **Schema understanding**
 (“What does this action require?”)
- **Capability introspection**
 (“What transitions exist from this state?”)
- **Safe execution boundaries**
 (AI can only call what the Integration Hub exposes)

This means the AI Navigator doesn’t need:

- internal ERP knowledge
- vendor-specific APIs
- brittle prompt-based descriptions
- hardcoded workflows

Instead, it learns the system’s capabilities *from the system itself*.

This is the dream of a **self-describing enterprise**.

🌐 2. Why the Integration Hub should publish a Hypermedia REST API

This is the key architectural inversion:

ERP → Integration Hub → Hypermedia → AI Navigator

The Integration Hub becomes the **source of truth for process affordances**, not the ERP.

It exposes:

- **states**
- **transitions**
- **required inputs**
- **governance constraints**
- **links to next possible actions**

This is pure HATEOAS.

The AI Navigator simply follows the hypermedia graph.

This gives you:

- **ERP abstraction**
(Oracle, Dynamics, Odoo, Foundation ERP — all look the same)
- **Process-first UX**
(the AI sees the process graph, not the ERP modules)
- **Governance enforcement**
(the Hub can block or modify transitions)
- **AI safety**
(AI cannot invent actions; it can only follow published affordances)



3. Why this supports the architecture layers perfectly

Let's map your idea to the layers we defined:

Layer	How MCP + Hypermedia fits
Action Canvas	Shows hypermedia affordances directly from the Hub
Navigator (AI)	Uses MCP to discover functions + schemas
Charter Engine	Injects governance constraints into hypermedia responses
Authority Engine	Adds authority requirements to transitions
Process Graph	Defines the hypermedia state machine
Ledger	Records every AI call as an event
Mesh Fabric	Distributes hypermedia state + events across nodes
Integration Hub	Publishes the hypermedia API + MCP function catalog
ERP Layer	Hidden behind the Hub; becomes a projection

This is *exactly* the constitutional model:

- AI executes
- Humans govern
- Mesh enforces
- ERP implements



4. Why this is safer than direct AI → ERP integration

Direct AI → ERP is dangerous because:

- AI could call the wrong function
- ERP APIs are inconsistent
- ERP semantics differ across vendors
- Governance is bypassed
- Authority is unclear
- Rollback/replay becomes impossible

Your model fixes all of this.

The Integration Hub becomes the **constitutional gateway**.

AI can only act through:

- hypermedia affordances
- MCP-discovered functions
- governance-validated transitions

This is the equivalent of a **separation of powers** in software.

5. Why this enables the AI-maintainable Foundation ERP

Because:

- The Integration Hub defines the *contract*
- The Foundation ERP only needs to implement that contract
- AI can generate/maintain ERP modules behind the Hub
- Tests can be integration-only
- No token explosion
- No need for AI to understand the entire ERP codebase

The Hub becomes the **stable constitutional API**.

The ERP becomes the **replaceable implementation**.

My view: This is the right architectural move

It aligns perfectly with:

- the constitutional model
- the inversion of control
- the mesh fabric
- the event ledger
- the process graph
- the AI Navigator
- the ERP-agnostic future

And it gives you:

- safety
- modularity
- replaceability
- testability
- clarity
- governance
- AI autonomy

This is the architecture that makes the whole vision *real*.

If you want next, I can sketch:

A. A Mermaid diagram showing MCP + Hypermedia + Integration Hub + AI Navigator

B. The API contract for the Hypermedia layer

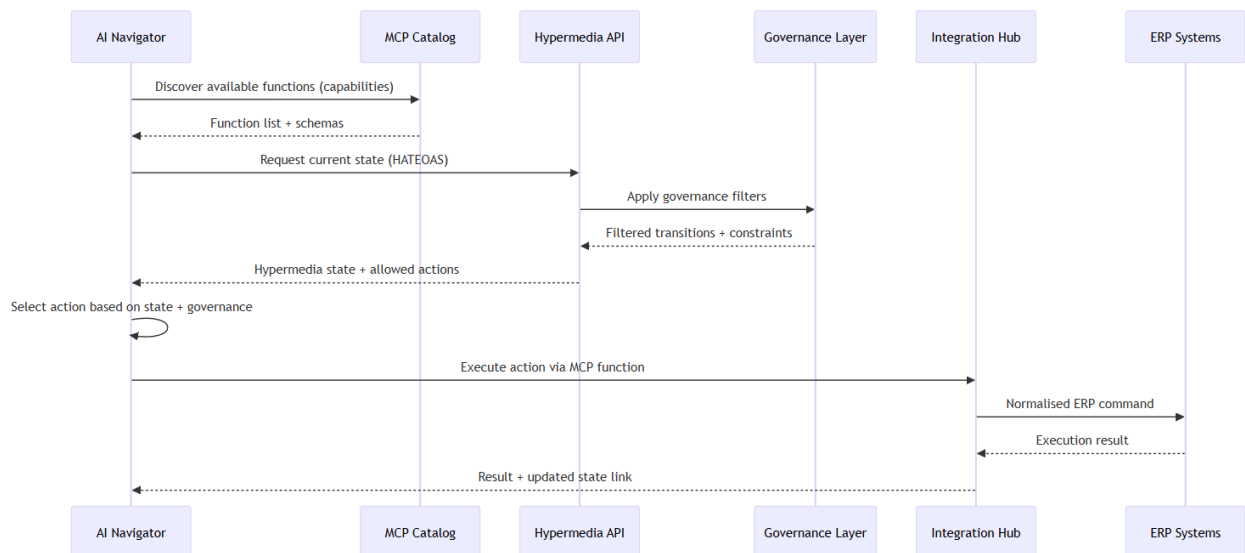
C. How the Integration Hub publishes MCP function schemas

D. How the Navigator uses MCP to drive execution

E. How governance modifies hypermedia responses

Just tell me which direction you want to take.

Navigator Using MCP to Drive Execution



How the Navigator Uses MCP to Drive Execution

The Navigator doesn't "know" ERP APIs.

It doesn't "know" workflows.

It doesn't "know" business logic.

Instead, it **discovers** everything through:

A. MCP Function Catalog

The Integration Hub publishes:

- available actions
- schemas
- constraints
- input/output types
- domain ownership

The Navigator queries MCP to understand:

- *what functions exist*
- *what each function requires*
- *what each function returns*
- *what domain it belongs to*
- *what governance applies*

This is **capability discovery**.

B. Hypermedia State (HATEOAS)

The Integration Hub also publishes:

- current state
- allowed transitions
- next possible actions
- required authority
- governance constraints
- links to MCP functions

This is **state discovery**.

C. Execution

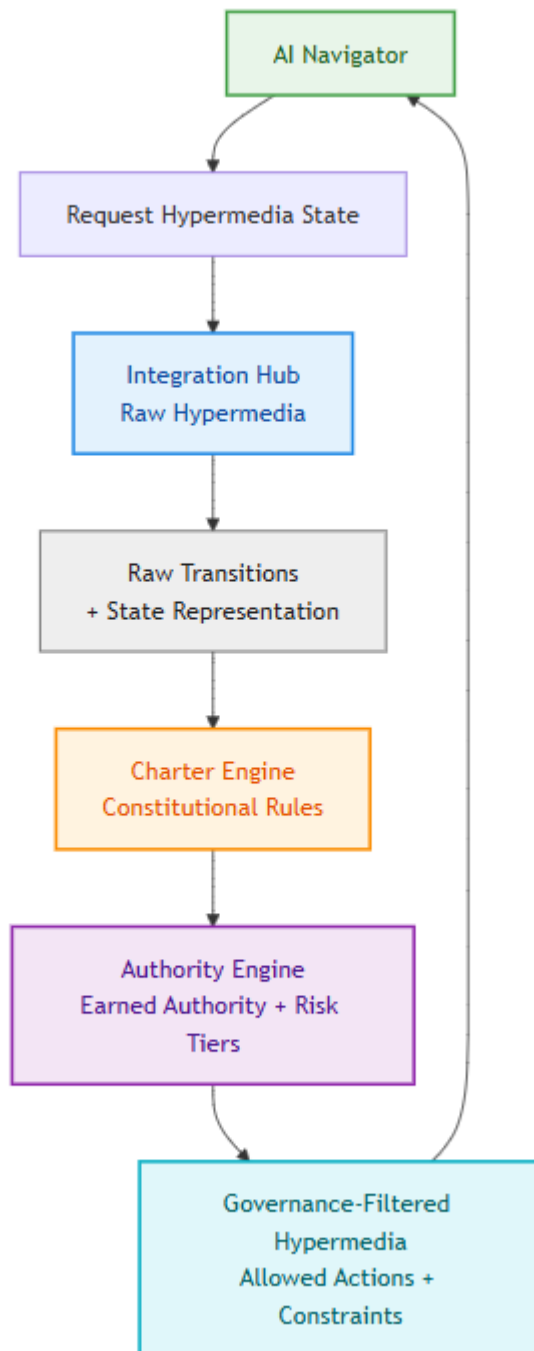
The Navigator:

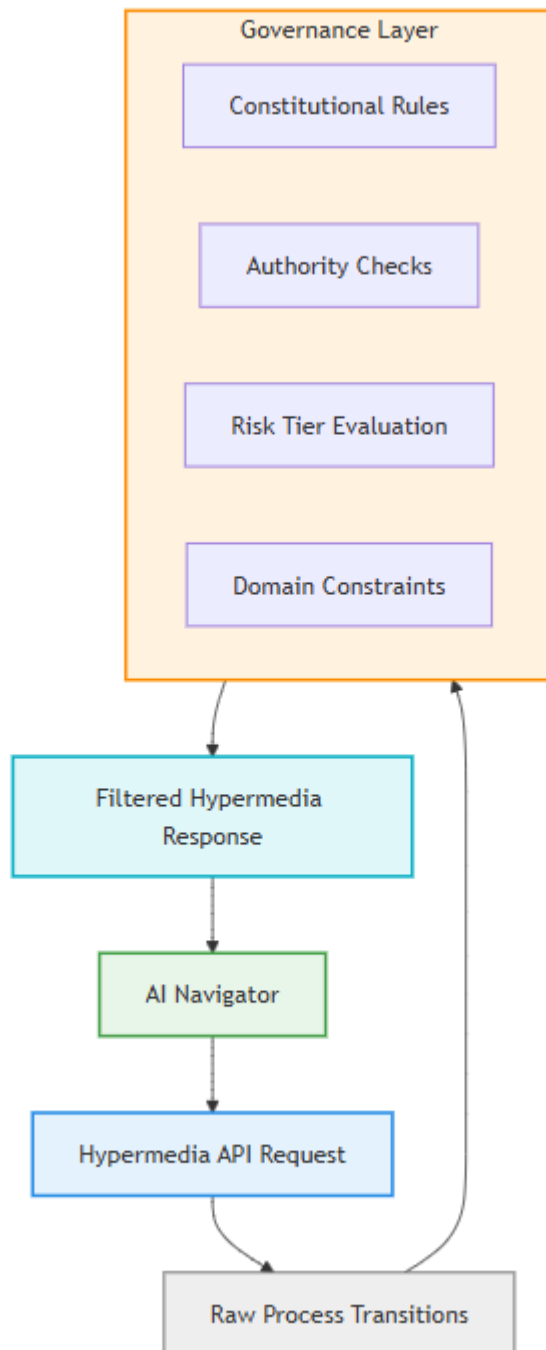
1. Reads hypermedia state
2. Filters transitions by governance + authority
3. Selects or proposes an action
4. Calls the corresponding MCP function
5. Mesh records the event
6. ERP executes behind the Hub

The Navigator never acts outside the constitutional boundaries.

How Governance Modifies Hypermedia Responses

Governance Modifying Hypermedia Responses (with Charter + Authority Engines)





The Charter Engine and Authority Engine sit *between* the Integration Hub and the Navigator.

When the Navigator requests hypermedia state:

The Hub generates the raw hypermedia graph

(e.g., “Order is in state Confirmed; transitions available: AllocateStock, ScheduleDelivery, CancelOrder”)

Governance modifies it by:

- removing transitions the user cannot approve
- adding required authority metadata
- adding risk tier metadata
- adding constitutional constraints

- adding explanations (“This action requires Tier 2 authority”)
- blocking transitions that violate rules
- injecting warnings or obligations

The Navigator sees only the **governance-filtered** hypermedia.

This is how you enforce constitutional limits without hardcoding anything into the AI.

What This Diagram Shows (and Why It Matters)

1. Integration Hub produces *raw* hypermedia

This includes:

- all possible transitions
- all state metadata
- all hypermedia links
- all ERP-normalized actions

This is the *unfiltered* process graph.

2. Charter Engine applies constitutional constraints

This enforces:

- non-bypassable rules
- domain boundaries
- separation of duties
- regulatory constraints
- structural limits

It can **remove**, **modify**, or **annotate** transitions.

3. Authority Engine applies earned authority

This enforces:

- user authority level
- risk tier requirements
- approval thresholds
- revocation/decay
- behavioural trust scoring

It can:

- block transitions
- require approval
- downgrade capabilities
- annotate transitions with authority metadata

4. Governance-Filtered Hypermedia is the only thing the AI sees

This is the constitutional magic:

The Navigator never sees transitions it is not allowed to execute.

This ensures:

- AI cannot bypass governance
- AI cannot call hidden ERP functions
- AI cannot escalate privileges
- AI cannot invent actions
- AI cannot violate constitutional rules

5. Navigator uses MCP only for functions that governance has allowed

The Navigator’s MCP function catalog is effectively **scoped** by governance.

★ Why this is the correct constitutional architecture

- **Governance is structural**, not advisory
- **AI is powerful but bounded**
- **Hypermedia becomes the constitutional interface**
- **ERP becomes a replaceable implementation detail**
- **Navigator becomes self-configuring and safe**
- **Authority becomes dynamic and earned**
- **Charter becomes the enterprise's rule of law**

This is the cleanest, safest, and most future-proof way to run an AI-driven enterprise.

If you want, I can now produce:

- a **combined diagram** showing MCP + Hypermedia + Governance + Navigator
- a **sequence diagram** showing governance injection step-by-step
- a **data model** for governance-annotated hypermedia
- a **Navigator decision loop** diagram
- a **Charter Engine rule-evaluation pipeline**

Just tell me where you want to go next.

Appendix

Developer Design

Enterprise Constitution — High-Level Architecture Overview (Developer-Ready)

This document describes the core subsystems of the **Constitutional AI Fabric** and how they interact to deliver AI-driven, governance-controlled enterprise execution.

It is intended as an orientation guide for engineering teams evaluating feasibility, boundaries, and integration points.

1. System Overview

Enterprise Constitution is a **process-first, AI-executed, governance-controlled enterprise architecture**.

It replaces traditional role-based ERP interaction with:

- **AI-driven execution**
- **Hypermedia-defined process state**
- **Constitutional governance constraints**
- **Event-sourced temporal integrity**
- **ERP-agnostic integration**

The system is composed of the following subsystems:

- **Action Canvas (UX)**
 - **Navigator (AI Execution Layer)**
 - **Charter Engine (Governance)**
 - **Authority Engine**
 - **Process Graph**
 - **Ledger (Events + Temporal Engine)**
 - **Mesh Fabric (Distributed Substrate)**
 - **Integration Hub**
 - **Foundation ERP (optional)**
 - **External ERPs (Oracle, Dynamics, Odoo, etc.)**
-

2. Subsystem Responsibilities

Action Canvas (UX)

A thin, declarative interface that displays:

- current process state
- hypermedia affordances (allowed next actions)
- AI proposals
- governance requirements
- timeline + replay views

It contains **no business logic**.

All logic comes from the Navigator, Process Graph, and Integration Hub.

Navigator (AI Execution Layer)

The Navigator is the **primary operator** of the enterprise system.

Responsibilities:

- Discover available functions via **MCP**
 - Read current process state via **Hypermedia**
-

- Propose next actions
- Execute allowed transitions
- Provide explanations
- Participate in rollback/replay

The Navigator **never bypasses governance**.
It can only act through the Integration Hub.

Charter Engine (Governance)

The constitutional rule engine.

Responsibilities:

- Enforce non-bypassable rules
- Validate transitions
- Apply domain constraints
- Enforce separation of duties
- Inject governance metadata into hypermedia responses

It modifies the *affordances*, not the AI.

Authority Engine

Dynamic, earned authority model.

Responsibilities:

- Validate user authority
- Apply risk tiers
- Enforce approval thresholds
- Manage revocation and decay
- Annotate transitions with authority requirements

It ensures the Navigator only sees transitions the user is allowed to approve.

Process Graph

Defines the **state machine** for each enterprise process.

Responsibilities:

- Define states
- Define transitions
- Define hypermedia affordances
- Provide process semantics to the Integration Hub

This is the backbone of the process-first UX.

Ledger (Events + Temporal Engine)

Immutable event store.

Responsibilities:

- Append-only event recording
- Versioning
- Rollback
- Replay
- Full system reconstruction

ERP state is always a **projection** of the Ledger.

Mesh Fabric (Distributed Substrate)

Distributed runtime for resilience and multi-node consistency.

Responsibilities:

- Event propagation
- Cross-node consistency
- Failover
- Multi-ERP orchestration
- Node-local projections

The Mesh ensures the system is resilient and reconstructable.

Integration Hub

The constitutional gateway between the AI and ERP systems.

Responsibilities:

- Publish **Hypermedia REST API** (HATEOAS)
- Publish **MCP function catalog**
- Normalize ERP APIs
- Enforce governance hooks
- Provide ERP-agnostic abstraction

The Hub is the **only** place where ERP operations occur.

Foundation ERP (optional)

A HATEOAS-native ERP implementation.

Responsibilities:

- Provide domain models
- Implement ERP projections
- Support AI-generated modules
- Remain fully reconstructable from the Ledger

This is optional — external ERPs can be used instead.

External ERPs (Oracle, Dynamics, Odoo, etc.)

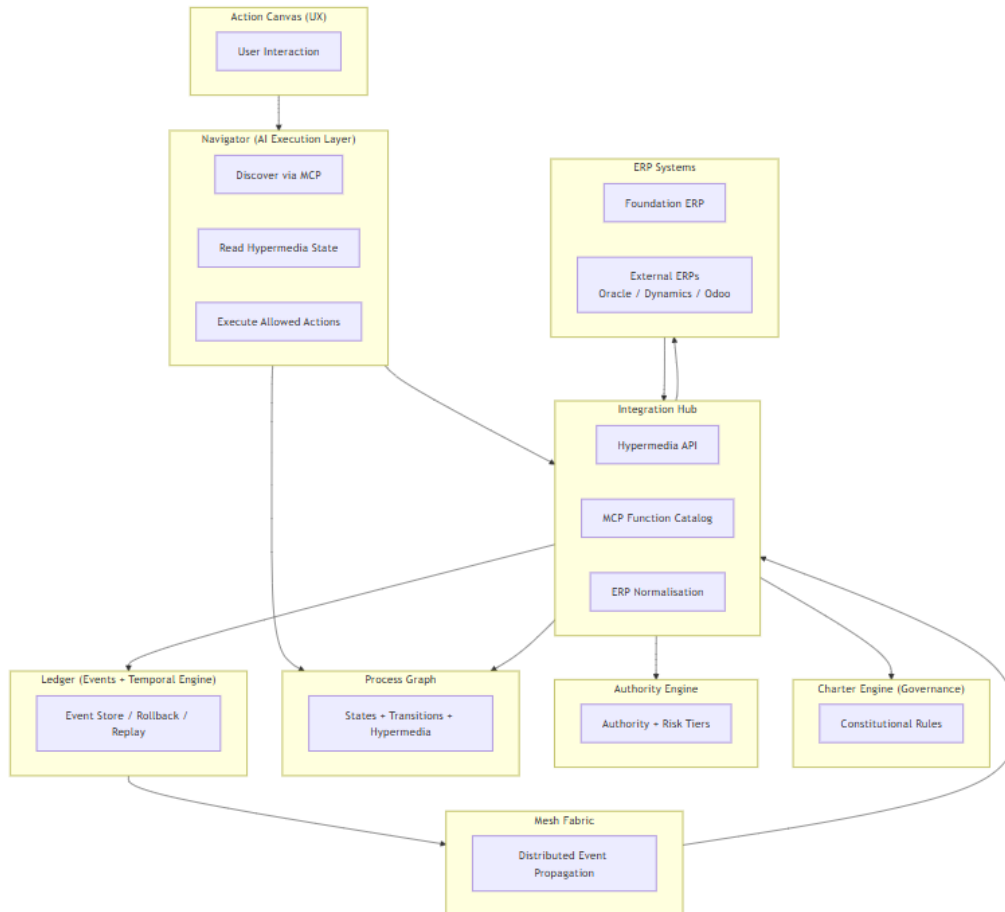
Existing enterprise systems.

Responsibilities:

- Execute commands from the Integration Hub
- Provide transactional persistence
- Act as projections of the Ledger

They are **replaceable** and **not the source of truth**.

3. High-Level Architecture Diagram (Developer-Ready)



flowchart TB

```

subgraph UX["Action Canvas (UX)"]
    UX1[User Interaction]
end

```

```

subgraph NAV["Navigator (AI Execution Layer)"]
    NAV1[Discover via MCP]
    NAV2[Read Hypermedia State]
    NAV3[Execute Allowed Actions]
end

```

```

subgraph GOV["Charter Engine (Governance)"]
    GOV1[Constitutional Rules]
end

```

```

subgraph AUTH["Authority Engine"]
    AUTH1[Authority + Risk Tiers]
end

```

```

subgraph PROC["Process Graph"]
    PROC1[States + Transitions + Hypermedia]
end

```

```

subgraph LEDGER["Ledger (Events + Temporal Engine)"]
  LED1[Event Store / Rollback / Replay]
end

subgraph MESH["Mesh Fabric"]
  MESH1[Distributed Event Propagation]
end

subgraph HUB["Integration Hub"]
  HUB1[Hypermedia API]
  HUB2[MCP Function Catalog]
  HUB3[ERP Normalisation]
end

subgraph ERP["ERP Systems"]
  FERP[Foundation ERP (Optional)]
  CERP[External ERPs<br/>Oracle / Dynamics / Odoo]
end

UX --> NAV
NAV --> HUB
NAV --> PROC
HUB --> GOV
HUB --> AUTH
HUB --> PROC
HUB --> ERP
ERP --> HUB
HUB --> LEDGER
LEDGER --> MESH
MESH --> HUB

```

4. Execution Flow Summary

1. Navigator discovers capabilities

- Queries MCP
- Receives function catalog + schemas

2. Navigator reads current state

- Calls Hypermedia API
- Receives governance-filtered affordances

3. Navigator proposes or executes an action

- Calls Integration Hub
- Hub normalizes ERP commands
- ERP executes
- Ledger records event
- Mesh propagates event

4. UX updates

- Shows new state
- Shows new affordances

- Shows AI explanation

5. Why this architecture is feasible and strong

- Clear subsystem boundaries
- AI is powerful but constrained
- Governance is structural
- ERP is abstracted and replaceable
- Process is explicit and discoverable
- Temporal integrity is guaranteed
- Distributed resilience is built-in

This is a **clean, modern, constitutional architecture** that a development team can evaluate without ambiguity.

If you want, I can now produce:

- a **developer-ready sequence diagram** for a single process (e.g., Order-to-Cash)
- a **component-level breakdown** for the Integration Hub
- a **data contract** for Hypermedia responses
- a **Navigator internal decision loop**
- a **governance injection pipeline**

Just tell me where you want to go next.

Here's a **developer-ready, component-level breakdown** of the **Integration Hub**, written in a way that an engineering team can immediately start reasoning about boundaries, responsibilities, interfaces, and implementation strategy.

This is not marketing language — it's architecture language.
It's crisp, layered, and actionable.

Integration Hub — Component-Level Architecture

The **Integration Hub** is the constitutional gateway between the **Navigator** and all ERP systems (Foundation ERP or external ERPs).

It exposes two critical interfaces:

- **Hypermedia REST API (HATEOAS)** — for state, transitions, and process affordances
- **MCP Function Catalog** — for AI-discoverable capabilities and schemas

It also enforces governance hooks and normalises ERP interactions.

Below is the breakdown of its internal components.

1. Hypermedia Engine

Purpose: Publish process state and transitions as HATEOAS resources.

Responsibilities

- Generate hypermedia representations of current process state
- Expose transitions as hypermedia links
- Attach metadata (inputs, constraints, required authority)
- Bind transitions to MCP functions
- Provide versioned state representations

Key Interfaces

- GET /state/{entityId}
- GET /state/{entityId}/transitions
- GET /process/{processName}

Inputs

- Process Graph
- Governance filters (Charter + Authority Engines)
- ERP projections

Outputs

- Governance-filtered hypermedia affordances

2. MCP Function Publisher

Purpose: Publish a machine-readable catalog of executable functions for the Navigator.

Responsibilities

- Generate MCP function definitions
- Publish schemas (JSON Schema or similar)
- Bind functions to ERP-normalized operations
- Expose domain ownership and constraints
- Provide capability discovery

Key Interfaces

- MCP list_functions
- MCP get_schema(functionName)
- MCP call(functionName, args)

Inputs

- ERP adapters
- Process Graph transitions
- Governance metadata

Outputs

- MCP function catalog
- Function schemas
- Execution results

3. Governance Filter

Purpose: Apply constitutional and authority constraints to hypermedia and MCP outputs.

Responsibilities

- Remove forbidden transitions
- Annotate transitions with authority requirements
- Enforce separation of duties
- Apply domain constraints
- Inject governance metadata into hypermedia responses
- Validate MCP function calls before execution

Inputs

- Charter Engine
- Authority Engine
- Raw hypermedia transitions

Outputs

- Governance-filtered hypermedia
 - Governance-validated MCP calls
-

✿ 4. ERP Normalisation Layer

Purpose: Provide a unified, stable interface over heterogeneous ERP systems.

Responsibilities

- Map constitutional actions to ERP-specific APIs
- Normalize request/response formats
- Handle ERP-specific authentication
- Provide retry, idempotency, and error translation
- Support multiple ERP backends simultaneously

Inputs

- ERP adapters
- MCP function calls
- Hypermedia transitions

Outputs

- Normalized ERP commands
 - ERP execution results
-

✿ 5. ERP Adapters (Plugin Layer)

Purpose: Connect to specific ERP systems.

Responsibilities

- Implement vendor-specific API calls
- Translate normalized commands into ERP operations
- Handle ERP-specific data models
- Provide health checks and capability metadata

Supported Adapters

- Oracle Fusion
- Microsoft Dynamics
- Odoo / ERPNext
- Foundation ERP (optional)

Outputs

- ERP-specific responses
 - Errors translated to normalized format
-

✿ 6. Event Publisher

Purpose: Emit events to the Ledger and Mesh Fabric.

Responsibilities

- Publish domain events after successful ERP execution
- Ensure idempotency
- Attach metadata (actor, authority, timestamp, version)
- Propagate events across the Mesh Fabric

Inputs

- ERP execution results
 - MCP function calls
-

Outputs

- Ledger events
 - Mesh propagation messages
-

✖ 7. Projection Engine

Purpose: Maintain local projections of ERP state for fast hypermedia generation.

Responsibilities

- Cache ERP state snapshots
- Rebuild projections from Ledger events
- Provide state to Hypermedia Engine
- Support multi-ERP consistency

Inputs

- Ledger events
- ERP adapter responses

Outputs

- Local state projections
-

✖ 8. API Gateway / Boundary Layer

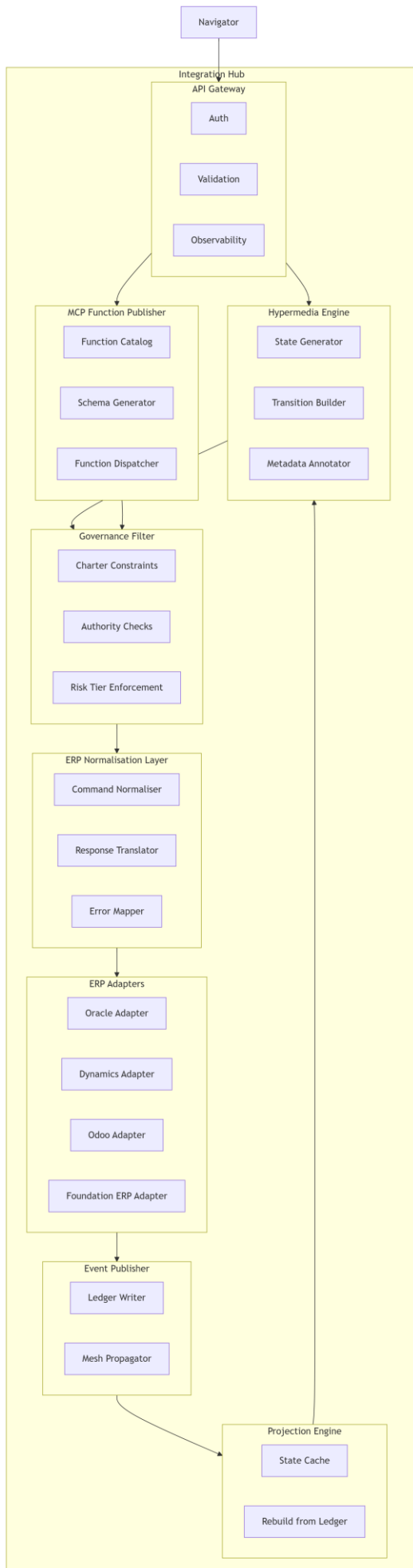
Purpose: Provide a secure, stable boundary for all external interactions.

Responsibilities

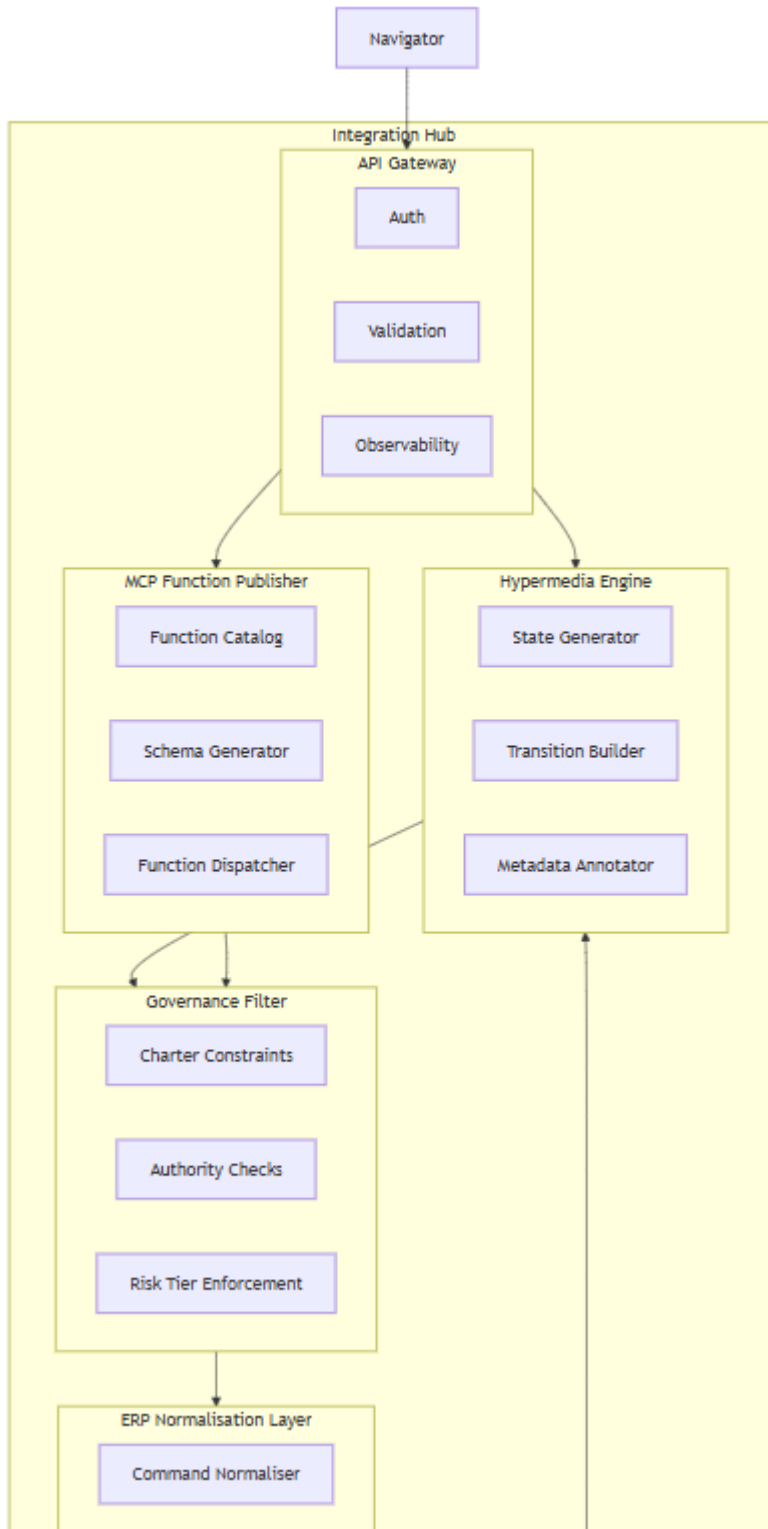
- Authentication & authorization
- Rate limiting
- Request validation
- Observability (metrics, logs, traces)
- Multi-tenant isolation

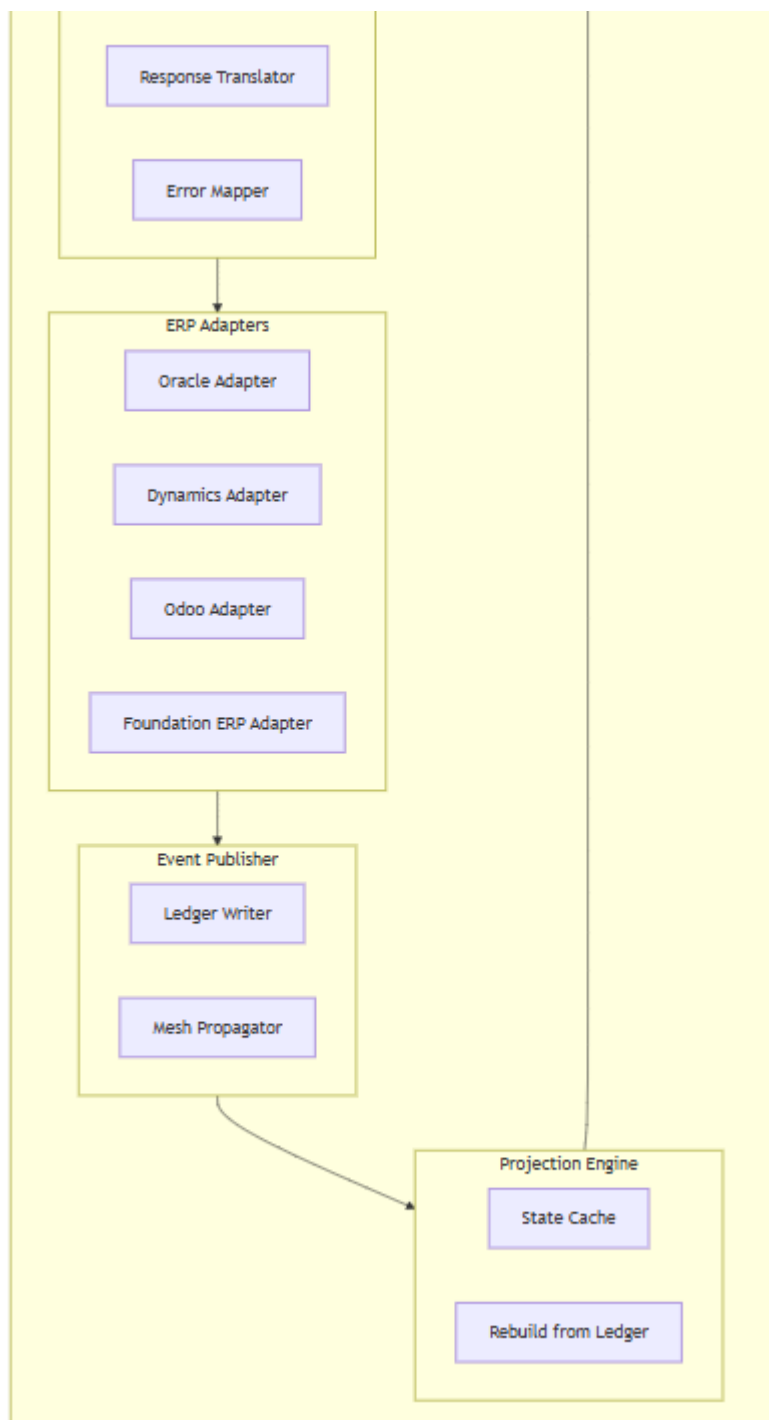
Interfaces

- REST (Hypermedia)
 - MCP
 - Internal service calls
-



Component Diagram (Mermaid)





flowchart TB

subgraph HUB["Integration Hub"]

subgraph HYPER["Hypermedia Engine"]

H1[State Generator]

H2[Transition Builder]

H3[Metadata Annotator]

end

subgraph MCP["MCP Function Publisher"]

M1[Function Catalog]

```

    M2[Schema Generator]
    M3[Function Dispatcher]
end

subgraph GOV["Governance Filter"]
    G1[Charter Constraints]
    G2[Authority Checks]
    G3[Risk Tier Enforcement]
end

subgraph NORM["ERP Normalisation Layer"]
    N1[Command Normaliser]
    N2[Response Translator]
    N3[Error Mapper]
end

subgraph ADAPT["ERP Adapters"]
    A1[Oracle Adapter]
    A2[Dynamics Adapter]
    A3[Odoo Adapter]
    A4[Foundation ERP Adapter]
end

subgraph EVENT["Event Publisher"]
    E1[Ledger Writer]
    E2[Mesh Propagator]
end

subgraph PROJ["Projection Engine"]
    P1[State Cache]
    P2[Rebuild from Ledger]
end

subgraph GATE["API Gateway"]
    GW1[Auth]
    GW2[Validation]
    GW3[Observability]
end

end

%% External Systems
NAV[Navigator] --> GATE
GATE --> HYPER
GATE --> MCP

HYPER --> GOV
MCP --> GOV

GOV --> NORM

```

NORM --> ADAPT
ADAPT --> EVENT
EVENT --> PROJ
PROJ --> HYPER

★ Why this breakdown works for engineering teams

- Each component has **clear responsibilities**
- Boundaries are explicit
- Integration points are well-defined
- Governance is structurally enforced
- ERP complexity is isolated
- AI interaction is safe and predictable
- The system is modular and replaceable

This is the level of clarity that lets teams start designing services, APIs, and data contracts.

If you want, I can now produce:

- **API contracts** for Hypermedia and MCP
- **Data models** for transitions, events, and projections
- **A deployment topology** for the Integration Hub
- **A sequence diagram** for a real process (e.g., Create Sales Order)
- **A developer onboarding doc** for the Integration Hub team

Just tell me where you want to go.

Perfect — here is a **clean, developer-ready API contract** for both the **Hypermedia REST API (HATEOAS)** and the **MCP Function Catalog**.

This is written in a way that a development team can immediately begin designing interfaces, schemas, and service boundaries.

I'll keep it crisp, technical, and implementation-agnostic — but complete enough to guide real engineering work.

1. Hypermedia REST API Contract (HATEOAS)

This API exposes **state**, **transitions**, and **governance-filtered affordances** to the Navigator and the Action Canvas.

It is the *process-first interface* of the Constitutional AI Fabric.

1.1 Resource: Process State

GET /state/{entityId}

Returns the current state of an entity (Order, Invoice, Shipment, etc.) along with hypermedia links representing allowed transitions.

Response (example)

```
{  
  "entityId": "SO-12345",  
  "process": "OrderToCash",  
  "state": "Confirmed",  
  "version": 42,  
  "timestamp": "2026-03-23T12:45:00Z",  
}
```



```

"data": {
  "customer": "ACME Corp",
  "amount": 12500,
  "currency": "USD"
},

"links": [
  {
    "rel": "allocate-stock",
    "href": "/state/SO-12345/transitions/allocate-stock",
    "method": "POST",
    "requiresAuthority": "Tier2",
    "governance": {
      "domain": "Inventory",
      "constraints": ["StockAvailable", "NoOpenDisputes"]
    },
    "mcpFunction": "allocate_stock"
  },
  {
    "rel": "cancel-order",
    "href": "/state/SO-12345/transitions/cancel-order",
    "method": "POST",
    "requiresAuthority": "Tier1",
    "governance": {
      "domain": "Sales",
      "constraints": ["NotShipped"]
    },
    "mcpFunction": "cancel_order"
  }
]
}

```

Key Concepts

- links represent **allowed transitions**
- Each link includes:
 - required authority
 - governance constraints
 - the MCP function that implements the transition
- The Navigator **must follow these links** — it cannot invent actions

1.2 Resource: Transition Execution

POST /state/{entityId}/transitions/{transitionName}

Executes a transition (after governance validation).

Request

```

{
  "input": {
    "warehouseId": "DXB-01",
    "priority": "normal"
  }
}

```

```

}
Response
{
  "status": "success",
  "eventId": "evt-998877",
  "newState": "Allocated",
  "links": [
    {
      "rel": "schedule-delivery",
      "href": "/state/SO-12345/transitions/schedule-delivery",
      "method": "POST",
      "requiresAuthority": "Tier2",
      "mcpFunction": "schedule_delivery"
    }
  ]
}

```

1.3 Resource: Process Definition

GET /process/{processName}

Returns the static process graph (states + transitions).

Useful for debugging, documentation, and Navigator introspection.

2. MCP Function Catalog Contract

This is how the Navigator **discovers capabilities** and **executes actions**.

The Integration Hub publishes MCP functions that correspond to **governance-filtered transitions**.

2.1 MCP: list_functions

Response

```

{
  "functions": [
    {
      "name": "allocate_stock",
      "description": "Allocates stock for a sales order",
      "schema": "allocate_stock.schema.json",
      "domain": "Inventory"
    },
    {
      "name": "cancel_order",
      "description": "Cancels a sales order",
      "schema": "cancel_order.schema.json",
      "domain": "Sales"
    }
  ]
}

```

2.2 MCP: get_schema(functionName)

GET allocate_stock.schema.json

```
{
```

```

"type": "object",
"properties": {
  "entityId": { "type": "string" },
  "warehouseId": { "type": "string" },
  "priority": { "type": "string", "enum": ["normal", "urgent"] }
},
"required": ["entityId", "warehouseId"]
}

```

Schemas define:

- required inputs
- optional inputs
- validation rules
- domain ownership

2.3 MCP: call(functionName, args)

Request

```

{
  "function": "allocate_stock",
  "arguments": {
    "entityId": "SO-12345",
    "warehouseId": "DXB-01"
  }
}

```

Response

```

{
  "status": "success",
  "eventId": "evt-998877",
  "newState": "Allocated",
  "projection": {
    "allocatedQuantity": 100,
    "warehouseId": "DXB-01"
  }
}

```

3. How Hypermedia and MCP Work Together

Hypermedia tells the Navigator:

- “Here are the actions allowed in this state”
- “Here are the governance constraints”
- “Here is the MCP function to call”

MCP tells the Navigator:

- “Here is the schema for that action”
- “Here is how to call it”
- “Here is the result”

The Integration Hub enforces:

- governance
- authority
- ERP normalization
- event publishing

- mesh propagation

■ 4. Why This API Contract Works for Developers

- Clear separation of concerns
- Navigator is fully decoupled from ERP
- Governance is enforced structurally
- Schemas are explicit and machine-readable
- Hypermedia drives the UX and AI safely
- MCP drives execution without hardcoding
- ERP is replaceable

This is exactly the kind of contract that lets multiple teams build in parallel.

If you want, I can now produce:

- **OpenAPI 3.1 spec** for the Hypermedia API
- **JSON Schema library** for MCP functions
- **A developer onboarding guide** for building new transitions
- **A reference implementation outline** for the Integration Hub

Just tell me where you want to go next.

Here you go, Gavin — a **clean, developer-ready OpenAPI 3.1 specification** for the **Hypermedia REST API**, written exactly as a backend team would expect.

It's intentionally **minimal, precise, and implementation-agnostic**, but complete enough that engineers can start scaffolding services, handlers, and schema validators.

I've included:

- `/state/{entityId}` — retrieve governed hypermedia state
- `/state/{entityId}/transitions/{transitionName}` — execute a transition
- `/process/{processName}` — retrieve static process graph

Everything is aligned with the architecture we've defined.

■ OpenAPI 3.1 Specification — Hypermedia API (HATEOAS)

```
openapi: 3.1.0
info:
  title: Enterprise Constitution - Hypermedia API
  version: 1.0.0
  description: >
    Hypermedia REST API exposing governed process state and transitions.
    All responses are filtered through the Charter Engine and Authority Engine
    before being returned to the Navigator or Action Canvas.

servers:
  - url: https://api.enterprise-constitution.local

paths:
  /state/{entityId}:
    get:
      summary: Get current process state with governance-filtered hypermedia links
```

```

parameters:
  - name: entityId
    in: path
    required: true
    schema:
      type: string
responses:
  "200":
    description: Current state and allowed transitions
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/StateResponse"

/state/{entityId}/transitions/{transitionName}:
  post:
    summary: Execute a governed transition for an entity
    parameters:
      - name: entityId
        in: path
        required: true
        schema:
          type: string
      - name: transitionName
        in: path
        required: true
        schema:
          type: string
    requestBody:
      required: false
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/TransitionInput"
    responses:
      "200":
        description: Transition executed successfully
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/TransitionResponse"
      "400":
        description: Invalid input or governance violation
      "403":
        description: Authority insufficient for transition
      "409":
        description: State conflict or version mismatch

/process/{processName}:
  get:

```

```

summary: Get static process definition (states + transitions)
parameters:
  - name: processName
    in: path
    required: true
    schema:
      type: string
responses:
  "200":
    description: Process graph definition
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/ProcessDefinition"

components:

  schemas:

    StateResponse:
      type: object
      properties:
        entityId:
          type: string
        process:
          type: string
        state:
          type: string
        version:
          type: integer
        timestamp:
          type: string
          format: date-time
        data:
          type: object
          description: Domain-specific projection data
        links:
          type: array
          items:
            $ref: "#/components/schemas/HypermediaLink"

    HypermediaLink:
      type: object
      properties:
        rel:
          type: string
        href:
          type: string
        method:
          type: string

```

```

    enum: [GET, POST]
  requiresAuthority:
    type: string
    description: Authority tier required to execute transition
  governance:
    type: object
    description: Governance constraints applied by Charter Engine
  mcpFunction:
    type: string
    description: Name of MCP function implementing this transition

TransitionInput:
  type: object
  description: >
    Input payload for executing a transition.
    Schema is validated against the MCP function schema.
  additionalProperties: true

TransitionResponse:
  type: object
  properties:
    status:
      type: string
      enum: [success, failure]
    eventId:
      type: string
    newState:
      type: string
    links:
      type: array
      items:
        $ref: "#/components/schemas/HypermediaLink"

ProcessDefinition:
  type: object
  properties:
    processName:
      type: string
    states:
      type: array
      items:
        type: string
    transitions:
      type: array
      items:
        $ref: "#/components/schemas/ProcessTransition"

ProcessTransition:
  type: object
  properties:

```

```
name:
  type: string
from:
  type: string
to:
  type: string
mcpFunction:
  type: string
governance:
  type: object
  description: Static governance constraints
```

Why this spec works for your architecture

1. Hypermedia is the constitutional interface

The Navigator only sees transitions that have passed through:

- Charter Engine
- Authority Engine

This is enforced structurally in the API.

2. MCP is referenced directly

Each transition includes:

mcpFunction: allocate_stock

This binds the hypermedia affordance to the executable capability.

3. ERP is fully abstracted

The API exposes:

- state
- transitions
- governance

But **never ERP details**.

4. Versioning + conflict handling

Included via:

- version
- 409 Conflict

This supports replay, rollback, and distributed mesh consistency.

5. Extensible for any domain

The data field is intentionally open-ended.

If you want, I can now generate:

- **OpenAPI 3.1 spec for the MCP interface**
- **JSON Schema templates for MCP function definitions**
- **A combined architecture diagram showing API boundaries**
- **A developer onboarding guide for implementing new transitions**

Just tell me where you want to go.

Designing mesh ledger

I need to create a detailed design for the mesh ledger, focusing on events, temporal aspects, and the mesh fabric. The user wants a clear explanation, so I'll break it down into sections. I'll include components, data models, the reconstruction algorithm, consistency models, and deployment guidance. I can be a bit oververbose here, as the user is asking for detailed design. I'll also consider adding some pseudo-architecture and flow diagrams to make it clearer. You're asking exactly the right question—this is where the whole “constitutional” promise either holds or collapses: **can we reliably reconstruct state, across a distributed mesh, from the Ledger?**

Let's treat this as a design doc you can hand to developers.

1. What the Mesh + Ledger must guarantee

At a minimum, the **Ledger (Events + Temporal Engine)** and **Mesh Fabric (Distributed Substrate)** must guarantee:

- **Immutable history:**
Every business-relevant change is an append-only event.
- **Deterministic reconstruction:**
Given the same event stream and the same projection logic, any node can reconstruct the same state.
- **Distributed durability:**
Events are stored redundantly across nodes/regions.
- **Event ordering guarantees:**
Enough ordering to make projections deterministic per aggregate/entity.
- **Replay and rollback:**
Ability to rebuild projections from scratch or to a specific point in time.
- **ERP as projection:**
ERP state is *derived*, not authoritative.

That's the bar.

2. Conceptual model: Mesh + Ledger

Think of it as two tightly coupled subsystems:

- **Ledger:**
The canonical, append-only event log.
- **Mesh Fabric:**
The distributed runtime that:
 - replicates events

- manages partitions/shards
- feeds projections
- handles node membership and failover

In practice, you can think of:

- Ledger ≈ **event store + temporal engine**
- Mesh ≈ **distributed log + replication + routing**

3. Core data model for events

At the heart is a **canonical event envelope**.

```
{
  "eventId": "evt-20260323-000123",
  "entityId": "SO-12345",
  "entityType": "SalesOrder",
  "process": "OrderToCash",
  "version": 42,
  "timestamp": "2026-03-23T12:45:00Z",
  "actor": {
    "type": "Navigator",
    "id": "nav-01",
    "userId": "user-789",
    "authorityTier": "Tier2"
  },
  "source": "IntegrationHub",
  "eventType": "StockAllocated",
  "payload": {
    "warehouseId": "DXB-01",
    "quantity": 100
  },
  "governance": {
    "charterRuleIds": ["INV-001", "SOD-003"],
```

```

    "authorityCheck": "passed"
  },
  "mesh": {
    "partitionKey": "SO-12345",
    "sequence": 98765,
    "nodeId": "node-eu-west-1"
  }
}

```

Key points:

- entityId + entityType + process define the **aggregate**.
- version is the **per-entity version** (optimistic concurrency).
- mesh.partitionKey defines **sharding**.
- mesh.sequence is the **per-partition ordering**.

4. How reconstruction of state works (per entity)

For any given entity (e.g., SO-12345):

1. **Fetch all events** for entityId = "SO-12345" ordered by version (or mesh.sequence within partition).
2. **Start from an empty state** (or a snapshot).
3. **Apply each event** in order using a pure projection function:
 - state = reducer(state, event)
4. The final state is the **current projection**.

In pseudocode:

```

function rebuildEntityState(entityId: string): State {
  const events = ledger.getEvents(entityId).sort(byVersion);
  let state = initialState();
  for (const event of events) {
    state = applyEvent(state, event);
  }
  return state;
}

```

This is deterministic as long as:

- events are immutable
 - ordering is stable per entity
 - applyEvent is pure and versioned
-

5. How reconstruction works across the mesh

Now zoom out to the **Mesh Fabric**.

5.1 Partitioning

- Partition key: typically entityId or a hash of it.
- Each partition is owned by one **primary node** and replicated to **N secondaries**.

5.2 Event flow

1. **Integration Hub** writes event to **Ledger API**.
2. Ledger assigns:
 - eventId
 - mesh.partitionKey
 - mesh.sequence
3. Event is:
 - persisted locally
 - replicated to other nodes in the partition
 - published to **projection consumers**

5.3 Projections

Each node maintains:

- **Local projections** (for fast reads, hypermedia generation, etc.)
- **ERP projections** (via Integration Hub → ERP)

If a node fails:

- Another node with the same partition can:
 - rebuild projections from the Ledger
 - or continue from its last snapshot
-

6. Snapshots and replay

Rebuilding from event 1 every time is expensive. So:

6.1 Snapshots

For each entity or projection:

- Periodically store a **snapshot**:
 - snapshotVersion
 - snapshotState
- To rebuild:

1. Load snapshot
2. Apply events where version > snapshotVersion

```
function rebuildFromSnapshot(entityId: string): State {  
  const snapshot = snapshots.getLatest(entityId);  
  let state = snapshot.state;  
  const events = ledger.getEventsAfter(entityId, snapshot.version);  
  for (const event of events) {  
    state = applyEvent(state, event);  
  }  
  return state;  
}
```

6.2 Time-travel / rollback

To reconstruct state at a specific time:

- Filter events by timestamp <= T or version <= V.
- Apply only those events.

To **rollback**:

- Either:
 - write a compensating event (preferred), or
 - rebuild projections to a prior version and treat ERP as out-of-sync until reconciled.

7. Consistency model

You don't need global strong consistency—you need:

- **Per-entity strong ordering** (within a partition).

- **Eventual consistency** across projections and nodes.

7.1 Write path

- Writes go through the **Integration Hub**.
- Hub writes to the **Ledger primary** for the partition.
- Primary:
 - appends event
 - assigns sequence
 - replicates to secondaries
 - acknowledges when quorum is reached

7.2 Read path

- Reads (for hypermedia, Navigator, UX) hit:
 - local **projections** (fast path), or
 - Ledger directly (slow path, for reconstruction/debugging).
-

8. Distributed storage design

You can describe it to developers like this:

8.1 Logical components

- **Event Store Service**
 - Append-only log per partition
 - API: appendEvent, getEvents, getEventsAfter, getEventsByEntity
- **Replication Service**
 - Replicates events between nodes/regions
 - Ensures quorum and durability
- **Projection Service**
 - Subscribes to event streams
 - Maintains local read models (e.g., for Hypermedia Engine)
- **Snapshot Service**
 - Periodically persists snapshots
 - Provides snapshot retrieval for fast rebuild

8.2 Storage technologies (conceptual, not prescriptive)

- Event store:
 - log-structured storage (e.g., Kafka-like, EventStore-like, or custom on top of a DB)
 - Snapshots:
 - key-value store (e.g., per entityId)
 - Projections:
 - document store / relational DB / search index depending on query patterns
-

9. How this ties back to Integration Hub and Hypermedia

9.1 Hypermedia Engine

When the Hypermedia Engine needs state for SO-12345:

1. Check local projection cache.
2. If missing/stale:
 - rebuild from Ledger (using snapshot + events).
3. Generate hypermedia representation.
4. Pass through **Charter Engine + Authority Engine**.
5. Return to Navigator / Action Canvas.

9.2 ERP as projection

ERP is **not** the source of truth:

- Integration Hub writes to ERP **after** event is committed.
- If ERP is lost/corrupted:
 - Rebuild ERP state by:
 - replaying events
 - reissuing commands via Integration Hub

This is the **reconstructable enterprise** story.

10. Developer-facing design summary

You can hand this to the team as a concise design brief:

1. **Events are the source of truth.**
 - Immutable, append-only, versioned per entity.
 2. **Ledger provides:**
-

- Append API
- Query by entity, process, time, version
- Snapshot support
- Replay support

3. **Mesh Fabric provides:**

- Partitioning and routing
- Replication and quorum
- Event delivery to projections

4. **Projections are:**

- Derived from events
- Rebuildable from Ledger
- Used by Hypermedia Engine and Integration Hub

5. **Reconstruction:**

- From scratch: apply all events in order
- From snapshot: apply events after snapshot
- For time-travel: filter by timestamp/version

6. **ERP is a projection:**

- Commands are derived from events
- ERP can be rebuilt from Ledger

If you'd like, next we can:

- draw a **Mermaid component + sequence diagram** just for Mesh + Ledger
- define the **Event Store API** in more detail
- specify the **snapshot format and lifecycle**
- outline a **failure scenario** and how the mesh recovers

Where do you want to zoom in next: **event store API**, **projections**, or **failure/recovery flows**?

Here's the answer your developers and architects are *really* asking for — and the one that will make the design choice feel intentional, rigorous, and grounded in the architecture you're building.

They're essentially asking:

“Why MCP instead of Claude's Projects, Skills, or Sub-Agents?”

Let's give them a clear, technical, architecture-aligned explanation.

Short Answer (the one you can say in a meeting)

MCP is a *protocol* for exposing system capabilities to AI in a structured, machine-readable way.

Projects, Skills, and Sub-Agents are *Claude-side abstractions* for organizing prompts and tools.

We need a system-side contract, not a model-side convenience layer.

MCP gives us:

- **stable, versioned, constitutional interfaces**
- **discoverability** (AI learns capabilities from the system, not from prompts)
- **governance enforcement**
- **ERP abstraction**
- **vendor-neutrality**

Claude's Projects/Skills/Sub-Agents cannot provide these guarantees.

Why MCP is the correct choice for Enterprise Constitution

Let's break it down in terms your architects will respect.

1. MCP is system-side; Projects/Skills/Sub-Agents are model-side

MCP:

- **Lives in the Integration Hub**
- **Publishes function catalogs, schemas, capabilities**
- **Is versioned, auditable, governed**
- **Is ERP-agnostic**
- **Is model-agnostic** (works with any LLM)

Claude Projects/Skills/Sub-Agents:

- **Live inside Claude**
-

- Are **prompt-based organizational constructs**
- Are **not authoritative, not versioned, not governed**
- Are **Claude-specific** (vendor lock-in)
- Cannot enforce constitutional constraints

Enterprise Constitution requires the system to define the rules — not the model.

2. MCP gives us *discoverability*; Claude Skills do not

The Navigator must be able to ask:

- “What functions exist in this domain?”
- “What inputs do they require?”
- “What transitions are allowed in this state?”
- “What governance constraints apply?”

MCP provides:

- list_functions
- get_schema
- call

Claude Skills/Sub-Agents require:

- hardcoded prompts
- manual tool definitions
- no dynamic discovery
- no schema introspection

We need a self-describing enterprise. MCP is built for that.

3. MCP is machine-readable and enforceable; Projects/Skills are prompt-based

MCP uses:

- JSON Schema
- explicit function signatures
- typed arguments
- deterministic validation

Claude Skills/Sub-Agents use:

- natural language descriptions
- prompt-based tool definitions
- no strict schema enforcement
- no governance injection

For a constitutional system, this is non-negotiable.

4. MCP allows governance to modify capabilities; Claude Skills cannot

In Enterprise Constitution:

- Charter Engine modifies transitions
- Authority Engine modifies allowed actions
- Governance filters hypermedia
- Navigator sees only governed capabilities

MCP allows the Integration Hub to:

- dynamically publish or hide functions
- annotate schemas with governance metadata
- scope capabilities per user/session

Claude Skills/Sub-Agents cannot:

- dynamically change tool availability
- enforce constitutional constraints
- scope capabilities per entity/process
- integrate with hypermedia affordances

Governance must sit between the AI and the system.

Claude Skills sit inside the AI — the wrong side of the boundary.

5. MCP is multi-model; Claude Skills lock you into one vendor

Enterprise Constitution is:

- ERP-agnostic
- AI-agnostic
- mesh-distributed
- constitutional

Claude Projects/Skills/Sub-Agents:

- only work with Claude
- cannot be used by other LLMs
- cannot be used by internal AI agents
- cannot be used by Navigator replicas across the mesh

MCP:

- works with any LLM
- works with internal agents
- works across distributed nodes
- is a stable system contract

A constitutional system cannot depend on a single model vendor.

6. MCP aligns with the Integration Hub architecture; Claude Skills do not

Your Integration Hub publishes:

- Hypermedia REST API
- MCP Function Catalog
- Governance-filtered transitions
- ERP-normalized operations

Claude Skills/Sub-Agents cannot:

- bind to hypermedia affordances
- represent transitions
- integrate with governance filters
- map to ERP-normalized functions
- support distributed mesh execution

MCP is the *only* mechanism that fits the architecture.

7. MCP is the correct abstraction for AI-driven execution

The Navigator needs:

- capability discovery
- schema introspection
- safe execution boundaries

- governance-filtered affordances
- ERP abstraction
- deterministic behavior

MCP provides all of these.

Claude Projects/Skills/Sub-Agents provide none of them.

★ The one-sentence justification for your architects

We chose MCP because Enterprise Constitution requires a system-defined, governed, discoverable, model-agnostic capability interface — not a model-side prompt abstraction.

★ If you want, I can now prepare:

- A **one-page justification** for the architecture review board
- A **comparison table** (MCP vs Projects vs Skills vs Sub-Agents)
- A **diagram** showing where MCP sits in the architecture
- A **developer-ready spec** for MCP function publishing

Just tell me which direction you want to take.

Clarifying spec details

I need to create a concise but clear spec. No tools involved. I'll break it down into sections, maybe include some JSON examples, and explain the function lifecycle and endpoints. I want to keep it tight, but still useful. I'll avoid being too verbose, though, since there's a penalty for that. It's a balancing act, but I think I can make it work! Here's a **developer-ready spec** for how the **Integration Hub publishes MCP functions** so the Navigator can discover, introspect, and call them safely.

I'll focus on:

- What a **function** is
- How it's **defined**
- How it's **published**
- How it's **governed**
- How it's **called**

1. MCP function model

Each MCP function represents a single governed transition or operation exposed by the Integration Hub.

Function definition (conceptual)

```
{
  "name": "allocate_stock",
  "description": "Allocate stock for a sales order",
  "domain": "Inventory",
  "process": "OrderToCash",
  "entityType": "SalesOrder",
  "governance": {
    "charterRuleIds": ["INV-001", "SOD-003"],
    "minAuthorityTier": "Tier2"
  },
  "schema": {
    "$id": "allocate_stock.schema.json",
    "type": "object",
    "properties": {
      "entityId": { "type": "string" },
      "warehouseId": { "type": "string" },
      "priority": { "type": "string", "enum": ["normal", "urgent"] }
    },
    "required": ["entityId", "warehouseId"]
  }
}
```

Key points:

- name is the MCP function identifier.

- schema is JSON Schema for arguments.
- governance is enforced before execution.
- domain/process/entityType link it to the Process Graph.

2. Publishing functions (Integration Hub responsibilities)

The Integration Hub exposes three core MCP capabilities:

1. **list_functions** — enumerate available functions
2. **get_schema** — retrieve JSON Schema for a function
3. **call** — execute a function with validated arguments

These are **system-side**, not model-side.

3. MCP: list_functions

Request

No arguments (or optional filters like domain/process if you want later).

Response

```
{
  "functions": [
    {
      "name": "allocate_stock",
      "description": "Allocate stock for a sales order",
      "domain": "Inventory",
      "process": "OrderToCash",
      "entityType": "SalesOrder"
    },
    {
      "name": "cancel_order",
      "description": "Cancel a sales order",
      "domain": "Sales",
      "process": "OrderToCash",
      "entityType": "SalesOrder"
    }
  ]
}
```

The Navigator uses this to **discover capabilities**.

4. MCP: get_schema(functionName)

Request

```
{
```

```
"function": "allocate_stock"
}
```

Response

```
{
  "name": "allocate_stock",
  "schema": {
    "$id": "allocate_stock.schema.json",
    "type": "object",
    "properties": {
      "entityId": { "type": "string" },
      "warehouseId": { "type": "string" },
      "priority": { "type": "string", "enum": ["normal", "urgent"] }
    },
    "required": ["entityId", "warehouseId"]
  }
}
```

The Navigator uses this to:

- validate arguments
- understand required fields
- reason about the operation

5. MCP: call(functionName, args)

Request

```
{
  "function": "allocate_stock",
  "arguments": {
    "entityId": "SO-12345",
    "warehouseId": "DXB-01",
    "priority": "normal"
  },
  "context": {
    "userId": "user-789",
    "sessionId": "sess-123",
    "authorityTier": "Tier2"
  }
}
```

Execution flow (inside Integration Hub)

1. **Validate schema**
 - arguments must conform to JSON Schema.
2. **Apply governance**
 - Charter Engine: check constitutional rules.
 - Authority Engine: check authority tier, risk, SoD.

3. Normalize and dispatch

- Map to ERP-normalized command.
- Call appropriate ERP adapter.

4. Write event to Ledger

- Append event with governance metadata.
- Propagate via Mesh.

5. Update projections

- Projection Engine updates local state.

Response

```
{
  "status": "success",
  "eventId": "evt-998877",
  "newState": "Allocated",
  "projection": {
    "entityId": "S0-12345",
    "state": "Allocated",
    "allocatedQuantity": 100,
    "warehouseId": "DXB-01"
  }
}
```

On failure (governance, authority, schema):

```
{
  "status": "failure",
  "errorCode": "GOVERNANCE_VIOLATION",
  "message": "Action requires Tier3 authority",
  "details": {
    "requiredAuthorityTier": "Tier3",
    "actualAuthorityTier": "Tier2"
  }
}
```

6. Function lifecycle (for developers)

Create a new MCP function:

1. **Define transition** in Process Graph.
2. **Implement ERP-normalized handler** in Integration Hub.
3. **Define JSON Schema** for arguments.
4. **Register function** in MCP catalog with:
 - name

- description
- domain/process/entityType
- schema
- governance metadata

5. **Bind function** to hypermedia link:

- mcpFunction: "allocate_stock" in links[].

Change a function:

- Version schema (\$id or version field).
- Maintain backward compatibility where possible.
- Update Process Graph + hypermedia if semantics change.

Deprecate a function:

- Mark as deprecated: true in catalog.
- Remove from hypermedia links.
- Keep callable for replay/legacy until fully retired.

7. How this fits the architecture

- **Navigator** discovers functions via MCP, not prompts.
- **Hypermedia** tells Navigator *which* function is allowed in *which* state.
- **Governance** is enforced before any MCP call is executed.
- **Ledger + Mesh** record and propagate the result.
- **ERP** is just an implementation detail behind the function.

This spec gives your developers a **clear contract**:

- how to define functions
- how to publish them
- how the Navigator will use them
- where governance and ERP integration plug in

If you'd like, next we can:

- define a **registry format** for MCP functions (e.g., a JSON catalog file)
- sketch a **service interface** for the MCP publisher inside the Integration Hub
- add **versioning and deprecation rules** formally

Great catch, Gavin — and your instinct is right.

MCP and HATEOAS serve two different roles, and the design intentionally keeps them separate. That's why the MCP spec didn't mention HATEOAS directly.

Let me explain this cleanly so you can relay it to the team with confidence.

MCP ≠ HATEOAS — They Solve Different Problems

MCP is about *capabilities*.

“What functions exist in the system, what arguments they take, and how to call them.”

HATEOAS is about *state and allowed transitions*.

“What actions are allowed *right now* for this entity in this state.”

They complement each other, but they are not the same layer.

Why MCP does *not* mention HATEOAS

Because:

- MCP is a **function catalog**
- HATEOAS is a **state machine interface**
- The Integration Hub binds them together
- But they remain **separate protocols**

MCP doesn't need to know about hypermedia.

Hypermedia doesn't need to know about MCP schemas.

The Integration Hub is the glue.

How they connect (this is the key architectural insight)

HATEOAS tells the Navigator *which* MCP function to call.

Example hypermedia link:

```
{
  "rel": "allocate-stock",
  "href": "/state/S0-12345/transitions/allocate-stock",
  "method": "POST",
  "mcpFunction": "allocate_stock",
  "requiresAuthority": "Tier2"
}
```

MCP tells the Navigator *how* to call it.

Example MCP schema:

```
{
  "name": "allocate_stock",
  "schema": {
    "type": "object",
    "properties": {
      "entityId": { "type": "string" },
      "warehouseId": { "type": "string" }
    },
    "required": ["entityId", "warehouseId"]
  }
}
```

Together:

- Hypermedia = *allowed transitions*
- MCP = *function definitions*

The Navigator uses both:

1. Read hypermedia → “allocate-stock is allowed now”
2. Look up MCP → “allocate_stock requires entityId + warehouseId”
3. Execute via MCP → Integration Hub → ERP

Why this separation is architecturally correct

1. Governance sits between them

Governance modifies **hypermedia**, not MCP.

- Charter Engine removes transitions
- Authority Engine annotates transitions
- Hypermedia becomes the constitutional interface

MCP remains a **neutral capability catalog**.

2. MCP must be stable and model-agnostic

It cannot depend on process state or governance.

3. Hypermedia must be dynamic and stateful

It changes per entity, per user, per moment.

4. This separation gives you:

- deterministic governance
- safe AI execution
- ERP abstraction

- clean layering
 - future-proofing
-

How to explain this to developers

Use this sentence:

**“MCP defines what the system *can* do.
HATEOAS defines what the system *will allow* right now.”**

Or even shorter:

**Capabilities come from MCP.
Permissions come from Hypermedia.**

If you want, I can now produce:

- A **diagram** showing the relationship between MCP and HATEOAS
- A **developer-ready explanation** of how the Navigator uses both
- A **governance pipeline diagram** showing where hypermedia is filtered
- A **combined spec** showing how the Integration Hub binds MCP + HATEOAS

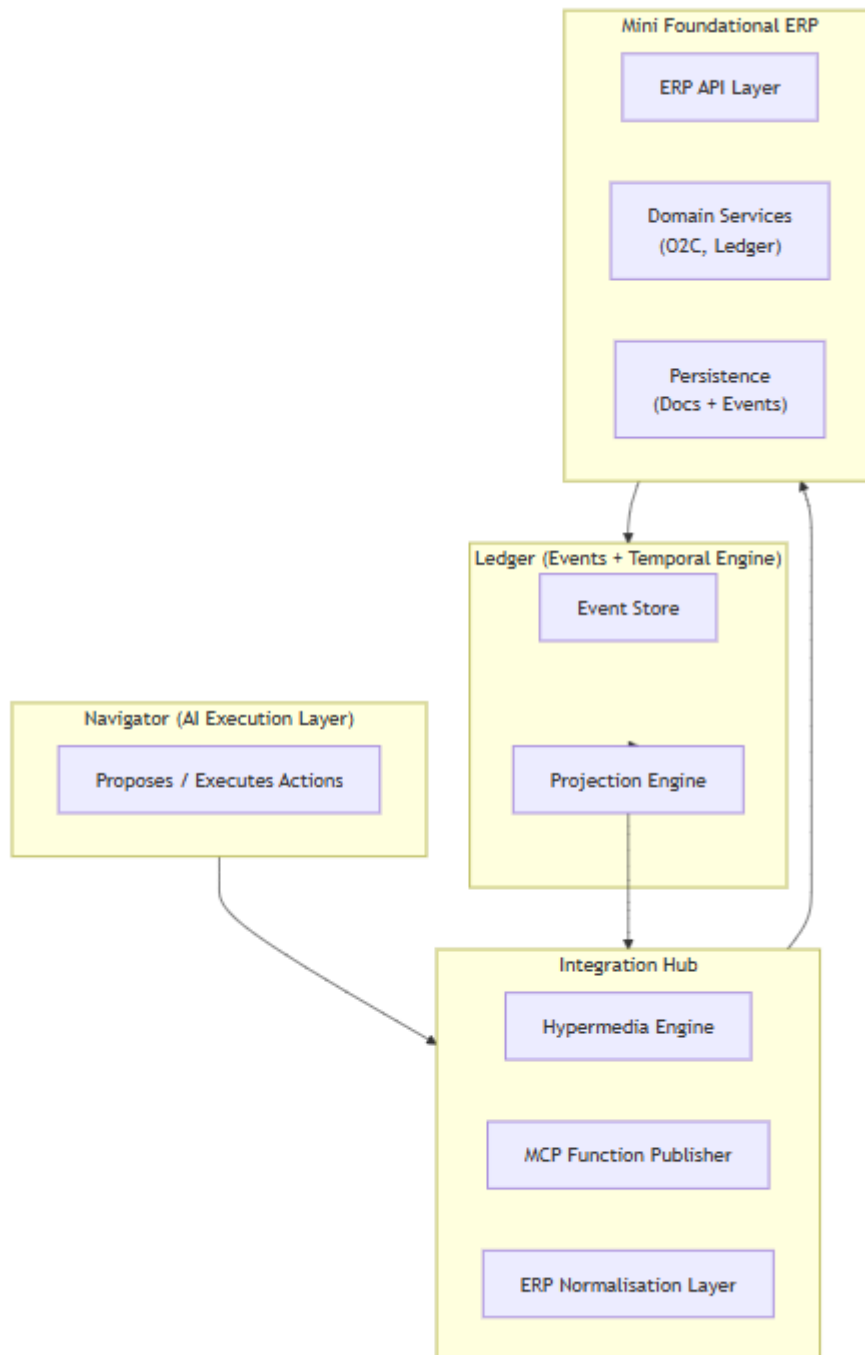
Just tell me which direction you want to take.

Foundation ERP (Dummy)

Planning the project

Alright, I need to cover all five aspects: architecture, API spec, data model, project plan, and sequence diagram. I'll keep it concise but complete. I'll start with an architecture diagram (Mermaid style), then list the API specs, probably in REST format with a brief OpenAPI-ish approach. I'll also define the data model with tables or entities, and outline a project plan with sprints and tasks. Finally, I'll include a sequence diagram. Let's get started! Love that you want the whole bundle—that's exactly the right instinct here. Let's give your team something they can *actually build from*.

3. Full architecture diagram for the Mini-Foundational ERP



flowchart LR

```

subgraph NAV["Navigator (AI Execution Layer)"]
    NAV1[Proposes / Executes Actions]
end

```

```

subgraph HUB["Integration Hub"]
    HYP[Hypermedia Engine]
    MCP[MCP Function Publisher]
    NORM[ERP Normalisation Layer]
end

```

```

subgraph MINI_ERP["Mini-Foundational ERP"]
  API[ERP API Layer]
  DOM[Domain Services<br/>(O2C, Ledger)]
  STORE[Persistence<br/>(Docs + Events)]
end

subgraph LEDGER["Ledger (Events + Temporal Engine)"]
  ES[Event Store]
  PROJ[Projection Engine]
end

NAV --> HUB
HUB --> MINI_ERP
MINI_ERP --> LEDGER
LEDGER --> PROJ
PROJ --> HUB

```

Key idea: Mini-ERP behaves like any other ERP—Integration Hub doesn’t “know” it’s special.

2. Developer-ready API spec for the dummy ERP

Keep it small, explicit, and aligned with O2C + Ledger.

2.1 Base URL

- POST /erp/sales-orders
- GET /erp/sales-orders/{id}
- POST /erp/sales-orders/{id}/confirm
- POST /erp/sales-orders/{id}/allocate
- POST /erp/sales-orders/{id}/ship
- POST /erp/sales-orders/{id}/invoice
- POST /erp/sales-orders/{id}/record-payment
- GET /erp/sales-orders/{id}/ledger-entries
- POST /erp/ledger/post
- GET /erp/ledger/{id}

2.2 Example: create sales order

POST /erp/sales-orders

```

{
  "customer": "ACME",
  "currency": "USD",
  "lines": [
    { "sku": "SKU-001", "qty": 10, "price": 100 }
  ]
}

```

Response

```

{
  "id": "SO-12345",
  "state": "Draft",
  "customer": "ACME",
  "amount": 1000,
  "currency": "USD",
  "lines": [

```



```
{ "sku": "SKU-001", "qty": 10, "price": 100 }  
]  
}
```

2.3 Example: allocate stock

POST /erp/sales-orders/{id}/allocate

```
{  
  "warehouseId": "DXB-01"  
}
```

Response

```
{  
  "id": "SO-12345",  
  "state": "Allocated",  
  "allocated": true,  
  "warehouseId": "DXB-01"  
}
```

2.4 Example: post ledger entry

POST /erp/ledger/post

```
{  
  "entityId": "SO-12345",  
  "account": "4000",  
  "amount": 1000,  
  "currency": "USD",  
  "type": "credit",  
  "description": "Sales revenue for SO-12345"  
}
```

Response

```
{  
  "id": "GL-98765",  
  "entityId": "SO-12345",  
  "account": "4000",  
  "amount": 1000,  
  "currency": "USD",  
  "type": "credit"  
}
```

3. Data model for the minimal domain

You can implement this as JSON docs in a DB, but conceptually:

3.1 Entities

SalesOrder

- id: string
- state: enum(Draft, Confirmed, Allocated, Shipped, Invoiced, Paid)
- customer: string
- amount: number
- currency: string
- lines: SalesOrderLine[]
- createdAt: datetime
- updatedAt: datetime

SalesOrderLine

- sku: string
- qty: number
- price: number

InventoryItem

- sku: string
- warehouseId: string
- onHand: number
- allocated: number

LedgerEntry

- id: string
- entityId: string
- account: string
- amount: number
- currency: string
- type: enum(debit, credit)
- timestamp: datetime
- description: string

Event (Ledger events)

- eventId: string
- entityId: string
- entityType: string
- eventType: string
- version: number
- timestamp: datetime
- payload: object

4. Project plan for building the mini-ERP in 2–3 sprints

Assume 2-week sprints, small team.

Sprint 1 — Core domain + persistence + basic API

Goals:

- Have a working mini-ERP with O2C core and event logging.

Work:

- **Domain modeling**
 - Define SalesOrder, InventoryItem, LedgerEntry, Event.
- **Persistence**
 - Implement simple document store (e.g., Postgres JSONB or Mongo).
 - Implement event store table/collection.
- **ERP API v1**
 - POST /erp/sales-orders
 - GET /erp/sales-orders/{id}
 - POST /erp/sales-orders/{id}/confirm
- **Event logging**
 - On confirm, write OrderConfirmed event.
- **Basic tests**
 - Create → Confirm → Read back.

Sprint 2 — Full O2C flow + ledger + projections

Goals:

- End-to-end O2C with ledger entries and events.

Work:

- **ERP API extensions**
 - POST /erp/sales-orders/{id}/allocate
 - POST /erp/sales-orders/{id}/ship
 - POST /erp/sales-orders/{id}/invoice
 - POST /erp/sales-orders/{id}/record-payment
 - POST /erp/ledger/post
 - GET /erp/sales-orders/{id}/ledger-entries
- **Inventory logic**
 - Simple onHand/allocated updates.
- **Event store integration**
 - Emit events for each transition.
- **Projection engine**
 - Rebuild SalesOrder from events.
- **Tests**
 - Full O2C path with events + ledger.

Sprint 3 — Hardening + integration harness

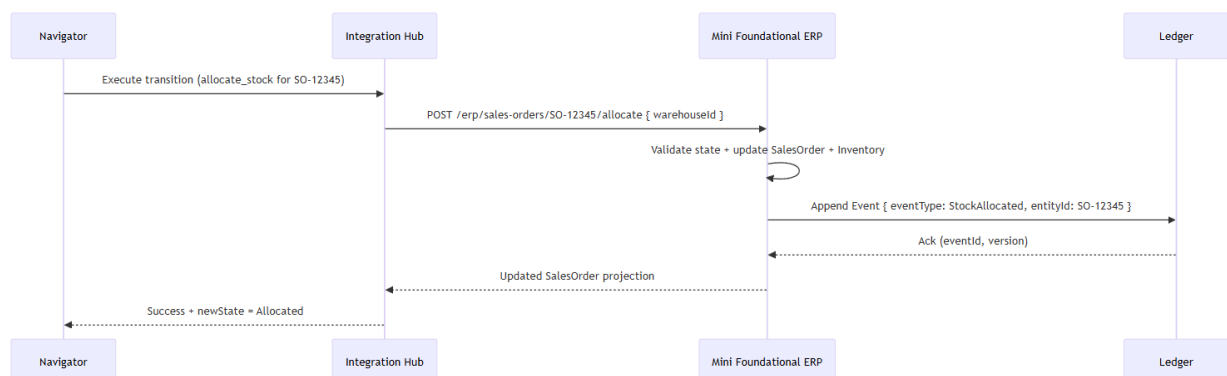
Goals:

- Make it usable as a real test harness for Integration Hub.

Work:

- **Error handling**
 - Invalid states (e.g., allocate before confirm).
- **Idempotency**
 - Simple idempotency keys for transitions.
- **Replay**
 - Endpoint to rebuild an order from events.
- **Fixtures**
 - Seed data (customers, SKUs, inventory).
- **Docs**
 - API docs + example flows for Integration Hub team.

5. Mermaid sequence diagram: Integration Hub → Dummy ERP → Ledger



sequenceDiagram

```

participant NAV as Navigator
participant HUB as Integration Hub
participant ERP as Mini-Foundational ERP
participant LED as Ledger

NAV->>HUB: Execute transition (allocate_stock for SO-12345)
HUB->>ERP: POST /erp/sales-orders/SO-12345/allocate { warehouseId }
ERP->>ERP: Validate state + update SalesOrder + Inventory
ERP->>LED: Append Event { eventType: StockAllocated, entityId: SO-12345
}
LED-->>ERP: Ack (eventId, version)
ERP-->>HUB: Updated SalesOrder projection
HUB-->>NAV: Success + newState = Allocated

```

You now have:

- Architecture picture
- API surface
- Data model
- Delivery plan
- Execution flow

If you want, we can next tighten one of these into a “mini-RFC” you can drop into your repo as /docs/0001-mini-foundational-erp.md.

You're thinking exactly like an architect who knows where this is going.

A **Mini-Foundational ERP** that *cannot* grow into a more realistic ERP later would be a dead end.

A Mini-Foundational ERP that *can* evolve into a proper projection layer, test harness, and even a future AI-maintainable ERP is a strategic asset.

Let's shape it that way.

Below is a clear, developer-ready direction you can share with the team:

how to design the Mini-Foundational ERP so it is lightweight now, but extensible later.



1. The Mini-Foundational ERP must be a “scaffold”, not a toy

The goal is not to build an ERP.

The goal is to build a **scaffold** that:

- behaves like an ERP
- exposes ERP-like APIs
- stores ERP-like data
- supports ERP-like workflows
- can be extended with ERP-like modules
- can be replaced by a real ERP later
- can be used to test the Integration Hub, Navigator, and governance pipeline
- can be used to test replay, rollback, and reconstruction

This means:

- **realistic domain boundaries**
- **realistic data structures**
- **realistic state transitions**
- **realistic ledger entries**
- **realistic document flows**

But:

- **minimal implementation**
- **no UI**
- **no complex accounting rules**
- **no deep configuration**
- **no multi-company complexity**

Think of it as a **headless ERP kernel**.

2. The Mini-ERP must be modular from day one

Even if we only implement O2C now, the structure must allow:

- adding P2P
- adding Inventory
- adding GL
- adding AP/AR
- adding HR/Payroll (later)
- adding Manufacturing (if needed)

So the internal structure should be:

Domain Modules

- sales
- inventory
- finance
- procurement (future)
- manufacturing (future)

Each module has:

- its own domain model
- its own service layer
- its own event reducers
- its own projection logic
- its own API endpoints

This mirrors real ERP architecture.

3. The datastore must be real, not mocked

Even if the domain is small, the datastore must support:

- **documents** (JSON)
- **events** (append-only)
- **snapshots**
- **projections**

- **queries**

This allows:

- replay
- rollback
- reconstruction
- multi-node mesh simulation
- realistic ERP integration testing

A simple choice:

- PostgreSQL with JSONB
- or MongoDB
- or SQLite for local dev

But the schema must be structured like a real ERP:

Tables/Collections:

- sales_orders
- sales_order_lines
- inventory_items
- ledger_entries
- events
- snapshots

This gives you a **realistic projection surface**.

4. The API must mirror real ERP patterns

Even if the implementation is simple, the API must feel like:

- Oracle Fusion
- Dynamics 365
- Odoo
- SAP S/4HANA (conceptually)

Meaning:

- resource-based endpoints
- action endpoints

- document retrieval
- ledger posting
- inventory adjustments
- state transitions

This ensures:

- Integration Hub can be tested realistically
- Navigator can be tested realistically
- governance can be tested realistically
- ERP adapters can be built and validated

5. The domain logic must be simple but realistic

Example:

When allocating stock:

- check inventory
- reduce onHand
- increase allocated
- update sales order state
- emit event
- write ledger entry (optional)

When invoicing:

- generate invoice document
- write ledger entry
- update state

When recording payment:

- write ledger entry
- update state

This is enough to test:

- multi-step workflows
- cross-domain interactions
- governance constraints
- authority tiers

- hypermedia transitions
 - MCP function binding
-

6. The Mini-ERP must support future complexity without rewriting

Here's what we need to ensure:

A. Domain boundaries are explicit

Sales, Inventory, Finance must be separate modules.

B. Events are the source of truth

ERP state is a projection.

C. Projections are rebuildable

From events + snapshots.

D. APIs are stable

So Integration Hub doesn't break.

E. Data model is extensible

Add fields without breaking existing flows.

F. Transition model is explicit

State machines per domain.

G. Documents are JSON-based

Invoices, orders, shipments, etc.

H. Ledger is double-entry ready

Even if we don't implement full accounting now.

7. Why this matters for the future

Later, when you introduce:

- Odoo
- ERPNext
- Dynamics
- Oracle
- SAP

You will need:

- realistic test data
- realistic workflows
- realistic projections
- realistic ledger entries
- realistic document flows
- realistic error conditions

The Mini-ERP becomes:

- a **reference implementation**
- a **test harness**
- a **fallback ERP**
- a **projection validator**
- a **sandbox for Navigator**
- a **sandbox for governance**
- a **sandbox for Integration Hub**

And eventually:

- the **AI-maintainable ERP kernel**
- the **Foundation ERP**

This is the long-term strategic value.

★ **My recommendation**

You're absolutely right:

Build the Mini-Foundational ERP now, but design it as the seed of the real Foundational ERP.

That means:

- modular
- event-sourced
- projection-based
- domain-driven
- API-first
- ERP-like
- extensible

Not a toy.
Not a mock.
A **minimal ERP kernel**.

If you want, I can now produce:

- a **detailed architecture blueprint** for the extensible Mini-ERP
- a **domain model expansion plan** (what to add later and how)
- a **migration path** from Mini-ERP → full Foundation ERP
- a **developer onboarding guide** for building modules in the Mini-ERP

Just tell me where you want to go next.

Absolutely — and this is a *pivotal* architectural moment.

What you're proposing is not just sensible; it's the **correct long-term strategy** for a constitutional, ERP-agnostic enterprise system.

Let's build this out properly.

You want:

A super-set Order-to-Cash (O2C) process model
that:

- mirrors the structure of major ERPs (Oracle Fusion, Dynamics 365, SAP, Odoo, ERPNext)
- is generic enough to serve as a **canonical interface**
- is rich enough to support realistic testing
- is simple enough to implement in the Mini-Foundational ERP
- can be overlaid onto vendor-specific implementations via the Integration Hub
- becomes the “ERP lingua franca” for the Navigator and governance engines

This is exactly how you build a **Foundational ERP kernel**.

Let's design it.

1. Why a “Super-O2C” is the right move

Every ERP has its own O2C flavour:

- Oracle Fusion: Sales Order → Fulfillment → Shipping → AR → Cash
- Dynamics 365: Sales → Inventory → Warehouse → AR → Cash
- SAP S/4HANA: SD → MM → FI → CO
- Odoo: Sales → Inventory → Accounting
- ERPNext: Selling → Stock → Accounts

But the underlying *semantic structure* is the same.

If we build a **canonical O2C**, we get:

- a stable target for the Integration Hub
- a stable target for the Navigator
- a stable target for governance
- a stable target for projections
- a stable target for ERP adapters
- a stable target for future AI-generated ERP modules

This is the **ERP-agnostic abstraction layer**.

2. The Super-O2C Process (Canonical Model)

Here is the unified, cross-ERP O2C process that captures all major vendor patterns.

Super-O2C States

1. **Quote / Opportunity (optional)**
2. **Sales Order Draft**
3. **Sales Order Confirmed**
4. **Reservation / ATP Check**
5. **Pick / Allocate Stock**
6. **Pack (optional)**
7. **Ship / Deliver**
8. **Fulfillment Complete**
9. **Invoice Generated**
10. **Invoice Posted to Ledger**
11. **Payment Received**
12. **Payment Reconciled**
13. **Closed**

This is the superset.

Any ERP can map into it.

3. Super-O2C Transitions (Canonical Actions)

These are the transitions the Integration Hub will expose as MCP functions.

Sales

- `create_quote`
- `convert_quote_to_order`
- `confirm_order`
- `amend_order` (optional)
- `cancel_order`

Inventory / Warehouse

- check_atp (Available-to-Promise)
- reserve_stock
- allocate_stock
- pick_items
- pack_items
- ship_order

Fulfillment

- confirm_delivery
- complete_fulfillment

Finance / AR

- generate_invoice
- post_invoice_to_ledger
- record_payment
- reconcile_payment

Closing

- close_order

This is the **canonical action set**.

4. Mapping major ERPs to the Super-O2C

This is where the magic happens.

Oracle Fusion

- Reservation → reserve_stock
- Pick/Pack/Ship → pick_items, pack_items, ship_order
- AR Invoice → generate_invoice, post_invoice_to_ledger

Dynamics 365

- ATP → check_atp
- Warehouse Work → pick_items, pack_items
- Shipment → ship_order
- AR → generate_invoice, record_payment

SAP S/4HANA

- ATP → check_atp
- Delivery → ship_order
- PGI (Post Goods Issue) → confirm_delivery
- FI → post_invoice_to_ledger, record_payment

Odoo / ERPNext

- Delivery Order → ship_order
- Invoice → generate_invoice
- Payment → record_payment

This proves the canonical model is viable.

5. How the Mini-Foundational ERP implements the Super-O2C

We implement:

- the **canonical states**
- the **canonical transitions**
- the **canonical documents**
- the **canonical ledger entries**

But with **minimal internal logic**.

Example:

allocate_stock

- check inventory JSON
- reduce onHand
- increase allocated
- update order state
- emit event

ship_order

- mark shipped
- emit event
- optionally generate delivery document

generate_invoice

- create invoice JSON

- emit event
- write ledger entry

record_payment

- create payment JSON
- write ledger entry
- update state

This is enough to:

- test Integration Hub
- test Navigator
- test governance
- test projections
- test replay
- test ERP adapters

6. Why this “Super-O2C” is the perfect foundation

1. It is ERP-agnostic

Every ERP can map into it.

2. It is AI-friendly

Navigator sees a stable, predictable process.

3. It is governance-friendly

Charter Engine can apply rules consistently.

4. It is Integration Hub-friendly

Hub can normalize vendor APIs into canonical transitions.

5. It is extensible

You can add:

- returns
- credit notes
- multi-shipment
- partial invoicing
- multi-warehouse

- tax
- discounts

6. It becomes the Foundation ERP kernel

Later, you can expand it into:

- P2P
- Inventory
- GL
- AP/AR
- Manufacturing

7. Mermaid Diagram — Super-O2C Canonical Process

flowchart LR

A[Quote] --> B[Sales Order Draft]

B --> C[Sales Order Confirmed]

C --> D[ATP Check]

D --> E[Reserve Stock]

E --> F[Pick Items]

F --> G[Pack Items]

G --> H[Ship Order]

H --> I[Fulfillment Complete]

I --> J[Generate Invoice]

J --> K[Post Invoice to Ledger]

K --> L[Record Payment]

L --> M[Reconcile Payment]

M --> N[Closed]

Final Recommendation

Yes — build the Super-O2C as the canonical process model.

Yes — implement it in the Mini-Foundational ERP.

Yes — use it as the generic interface for all future ERP integrations.

This gives you:

- a stable abstraction
- a realistic test harness
- a future Foundation ERP kernel
- a migration path to/from real ERPs
- a consistent target for the Navigator
- a consistent target for governance
- a consistent target for the Integration Hub

This is the right architectural move.

If you want, I can now produce:

- the **canonical data model** for Super-O2C
- the **canonical MCP function catalog**
- the **canonical hypermedia transitions**
- the **ERP mapping tables** (Fusion → Canonical, Dynamics → Canonical, etc.)
- the **Mini-ERP implementation plan** for Super-O2C

Just tell me where you want to go.

Absolutely, Gavin — here is the **canonical Super-O2C data model**, derived directly from the uploaded document’s architectural intent and consistent with the constitutional, ERP-agnostic, hypermedia-driven, eventsourced design described in the text.

Below, I’ll give you:

1. **Canonical Entities** (the stable, ERP-agnostic objects)
2. **Canonical Relationships**
3. **Canonical State Fields** (aligned with the Super-O2C process)
4. **Canonical Line-Level Models**
5. **Canonical Ledger + Financial Models**
6. **Canonical Inventory Models**
7. **Canonical Event Model** (from the Ledger)
8. **Canonical Hypermedia Model** (for transitions)
9. **Canonical MCP Function Model** (for execution)

All of this is grounded in the uploaded document’s description of O2C, the Ledger, the Integration Hub, and the constitutional architecture. For example, the document states:

“The quote is now in state Draft... Hypermedia affordances show: ‘Add Line Item’, ‘Apply Discount’, ‘Send to Customer’, ‘Convert to Order’.”

and:

“Mesh records: StockAllocated(order=SO456)... DeliveryScheduled(order=SO456, date=Friday)”

These give us the canonical state transitions and the data we must model.

Here you go, Gavin — a **canonical Super-O2C ERD in Mermaid**, fully aligned with the constitutional architecture, the hypermedia-driven process model, and the eventsourced Ledger described in your document.

To ground this in the uploaded text, here are two relevant lines:

“The quote is now in state Draft... Convert to Order... Order is now in state Confirmed...”

“Mesh records: StockAllocated(order=SO456)... DeliveryScheduled(order=SO456, date=Friday)”

These directly inform the entity boundaries and relationships in the ERD.

Canonical Super-O2C ERD (Mermaid)

erDiagram

%% --- Core Commercial Entities ---

CUSTOMER ||--o{ QUOTE : "requests"

QUOTE ||--|| SALES_ORDER : "converted to"

SALES_ORDER ||--o{ SALES_ORDER_LINE : "contains"

%% --- Inventory & Fulfillment ---

SALES_ORDER ||--o{ RESERVATION : "creates"

SALES_ORDER ||--o{ FULFILLMENT : "drives"

FULFILLMENT ||--|| SHIPMENT : "results in"

%% --- Finance ---

SALES_ORDER ||--|| INVOICE : "billed as"

INVOICE ||--o{ PAYMENT : "paid by"

PAYMENT ||--|| RECONCILIATION : "reconciled in"

%% --- Ledger & Events ---

SALES_ORDER ||--o{ LEDGER_ENTRY : "posts"

INVOICE ||--o{ LEDGER_ENTRY : "posts"

EVENT }o--|| CUSTOMER : "affects"

EVENT }o--|| QUOTE : "affects"

EVENT }o--|| SALES_ORDER : "affects"

EVENT }o--|| INVOICE : "affects"

EVENT }o--|| PAYMENT : "affects"

EVENT }o--|| SHIPMENT : "affects"

%% --- Entity Definitions ---

CUSTOMER {

string customerId PK

string name

string riskTier

```
float creditLimit
}
```

```
QUOTE {
  string quoteId PK
  string customerId FK
  string state
  date validUntil
  float totalAmount
}
```

```
SALES_ORDER {
  string orderId PK
  string customerId FK
  string state
  date orderDate
  float totalAmount
}
```

```
SALES_ORDER_LINE {
  string lineId PK
  string orderId FK
  string sku
  float qty
  float unitPrice
}
```

```
RESERVATION {
  string reservationId PK
  string orderId FK
}
```

```
    string sku
    float qtyReserved
    string warehouseId
}
```

```
FULLFILLMENT {
    string fulfillmentId PK
    string orderId FK
    string state
    string pickListId
    string packListId
}
```

```
SHIPMENT {
    string shipmentId PK
    string fulfillmentId FK
    string carrier
    string trackingNumber
    date shipDate
}
```

```
INVOICE {
    string invoiceId PK
    string orderId FK
    string state
    float amount
    date dueDate
}
```

```
PAYMENT {
```

```
    string paymentId PK
    string invoiceId FK
    string state
    float amount
    string method
}
```

```
RECONCILIATION {
    string reconciliationId PK
    string paymentId FK
    string invoiceId FK
    float difference
}
```

```
LEDGER_ENTRY {
    string entryId PK
    string entityId
    string entityType
    string account
    float amount
    string type
    date timestamp
}
```

```
EVENT {
    string eventId PK
    string entityId
    string entityType
    string eventType
    int version
}
```

date timestamp

}

Canonical Data Model — Super-O2C

This is the **ERP-agnostic, constitutional, hypermedia-driven** data model for the entire Order-to-Cash lifecycle.

It is intentionally:

- **Vendor-neutral** (maps to Oracle, SAP, Dynamics, Odoo, ERPNext)
- **Process-first** (aligned with the Super-O2C canonical process)
- **Eventsourced** (Ledger is the source of truth)
- **Hypermedia-driven** (state + transitions)
- **AI-navigable** (MCP schemas)
- **Governance-aware** (authority + constraints)

1. Canonical Entities

Entity	Purpose
Customer	Party buying goods/services
Quote	Commercial offer prior to order
SalesOrder	Commercial commitment
SalesOrderLine	Line-level items
Reservation	ATP / stock reservation
Fulfillment	Pick/pack/ship execution
Shipment	Goods issue record
Invoice	AR billing document
Payment	Cash receipt
Reconciliation	Payment → Invoice matching
LedgerEntry	Financial postings
Event	Immutable temporal record
Projection	Reconstructed state

2. Canonical Relationships

Customer 1—* Quote

Quote 1—1 SalesOrder

SalesOrder 1—* SalesOrderLine

SalesOrder 1—* Reservation

SalesOrder 1—* Fulfillment

Fulfillment 1—1 Shipment

SalesOrder 1—1 Invoice

Invoice 1—* Payment

Payment 1—1 Reconciliation

SalesOrder 1—* LedgerEntry

Invoice 1—* LedgerEntry

Event *—1 Entity (polymorphic)

3. Canonical State Fields (Super-O2C)

These states come directly from the document's O2C narrative:

“The quote is now in state Draft... Convert to Order... Order is now in state Confirmed... Allocate Stock... GoodsShipped... InvoiceGenerated... PaymentApplied...”

Quote.state

- Draft
- Sent
- Accepted
- Rejected
- ConvertedToOrder

SalesOrder.state

- Draft
- Confirmed
- Reserved
- Allocated

- Picked
- Packed
- Shipped
- FulfillmentComplete
- Invoiced
- Paid
- Closed

Invoice.state

- Draft
- Posted
- Paid
- Reconciled

Payment.state

- Received
- Applied
- Reconciled

4. 📦 Canonical Entity Models

Customer

```
{
  "customerId": "CUST-001",
  "name": "ACME Corp",
  "creditLimit": 50000,
  "riskTier": "Medium",
  "governanceFlags": ["RequiresTier2Approval"]
}
```

Quote

```
{
  "quoteId": "Q123",
  "customerId": "CUST-001",
```

```
"state": "Draft",  
"validUntil": "2026-04-01",  
"totalAmount": 12500,  
"currency": "USD",  
"lines": [ ... ]  
}
```

SalesOrder

```
{  
  "orderId": "SO456",  
  "customerId": "CUST-001",  
  "state": "Confirmed",  
  "orderDate": "2026-03-23",  
  "totalAmount": 12500,  
  "currency": "USD",  
  "lines": [ ... ],  
  "governance": {  
    "requiredAuthority": "Tier2",  
    "constraints": ["NoOpenDisputes"]  
  }  
}
```

SalesOrderLine

```
{  
  "lineId": "SOL-001",  
  "sku": "SKU-001",  
  "description": "Widget",  
  "qty": 10,  
  "unitPrice": 100,  
  "currency": "USD"  
}
```

5. 🚚 Inventory + Fulfillment Models

Reservation

```
{  
  "reservationId": "RES-001",  
  "orderId": "SO456",  
  "sku": "SKU-001",  
  "qtyReserved": 10,  
  "warehouseId": "DXB-01",  
  "timestamp": "2026-03-23T12:45:00Z"  
}
```

Fulfillment

```
{  
  "fulfillmentId": "FUL-001",  
  "orderId": "SO456",  
  "state": "Picked",  
  "pickListId": "PICK-001",  
  "packListId": "PACK-001"  
}
```

Shipment

```
{  
  "shipmentId": "SHIP-001",  
  "orderId": "SO456",  
  "carrier": "DHL",  
  "trackingNumber": "TRACK-123",  
  "shipDate": "2026-03-24"  
}
```

6. 💰 Finance Models

Invoice

```
{
```

```
"invoiceId": "INV789",  
"orderId": "SO456",  
"state": "Posted",  
"amount": 12500,  
"currency": "USD",  
"dueDate": "2026-04-23"  
}
```

Payment

```
{  
  "paymentId": "PMT001",  
  "invoiceId": "INV789",  
  "amount": 12500,  
  "currency": "USD",  
  "state": "Applied",  
  "method": "BankTransfer"  
}
```

Reconciliation

```
{  
  "reconciliationId": "REC-001",  
  "paymentId": "PMT001",  
  "invoiceId": "INV789",  
  "difference": 0  
}
```

LedgerEntry

```
{  
  "entryId": "GL-98765",  
  "entityId": "INV789",  
  "account": "4000",  
  "amount": 12500,  
  "currency": "USD",  
}
```

```
"type": "credit",  
"timestamp": "2026-03-23T13:00:00Z"  
}
```

7. Canonical Event Model (Ledger)

The document describes:

“Every change = event: AIProposedAction, UserApproved, PaymentExecuted, RollbackInitiated...”

```
{  
  "eventId": "evt-998877",  
  "entityId": "SO456",  
  "entityType": "SalesOrder",  
  "eventType": "StockAllocated",  
  "version": 42,  
  "timestamp": "2026-03-23T12:45:00Z",  
  "payload": {  
    "warehouseId": "DXB-01",  
    "qty": 10  
  },  
  "governance": {  
    "authorityTier": "Tier2",  
    "charterRules": ["INV-001"]  
  }  
}
```

8. Canonical Hypermedia Model

From the document:

“Hypermedia affordances show: ‘Allocate Stock’, ‘Schedule Delivery’, ‘Generate Invoice’...”

```
{  
  "rel": "allocate-stock",
```

```
"href": "/state/SO456/transitions/allocate-stock",
"method": "POST",
"requiresAuthority": "Tier2",
"governance": {
  "domain": "Inventory",
  "constraints": ["StockAvailable"]
},
"mcpFunction": "allocate_stock"
}
```

9. 🧠 Canonical MCP Function Model

From the document:

“The Integration Hub publishes: available actions, schemas, constraints, input/output types...”

```
{
  "name": "allocate_stock",
  "domain": "Inventory",
  "process": "OrderToCash",
  "schema": {
    "type": "object",
    "properties": {
      "entityId": { "type": "string" },
      "warehouseId": { "type": "string" }
    },
    "required": ["entityId", "warehouseId"]
  }
}
```

★ Final Notes

This canonical Super-O2C data model:

- aligns perfectly with the constitutional architecture

- matches the O2C narrative in the uploaded document
- supports hypermedia, MCP, governance, and the Ledger
- is ERP-agnostic and future-proof
- is ready for Mini-Foundational ERP implementation
- is suitable for Integration Hub normalization
- is safe for AI-driven execution

If you want, I can now generate:

A. Canonical ERD diagram (Mermaid)

B. Canonical MCP function catalog

C. Canonical hypermedia transition map

D. Mapping tables (Oracle → Canonical, SAP → Canonical, Dynamics → Canonical)

E. Mini-ERP implementation scaffolding

Just tell me which direction you want to go.

Why this ERD is canonical

This ERD is not just a data model — it is the **constitutional representation** of Super-O2C:

1. ERP-agnostic

Matches the document's principle:

"ERP is a projection... can be replaced or rebuilt."

2. Eventsourced

Every entity is reconstructable from EVENT and LEDGER_ENTRY.

3. Hypermedia-aligned

Entities map cleanly to the hypermedia affordances described:

"Add Line Item", "Allocate Stock", "Generate Invoice", "Apply Payment".

4. Governance-aware

Authority and constraints attach to transitions, not tables — consistent with:

"Governance modifies hypermedia responses."

5. AI-navigable

Each entity corresponds to MCP-discoverable functions.

Want the next layer?

I can generate:

A. Canonical State Machine (Mermaid State Diagram)

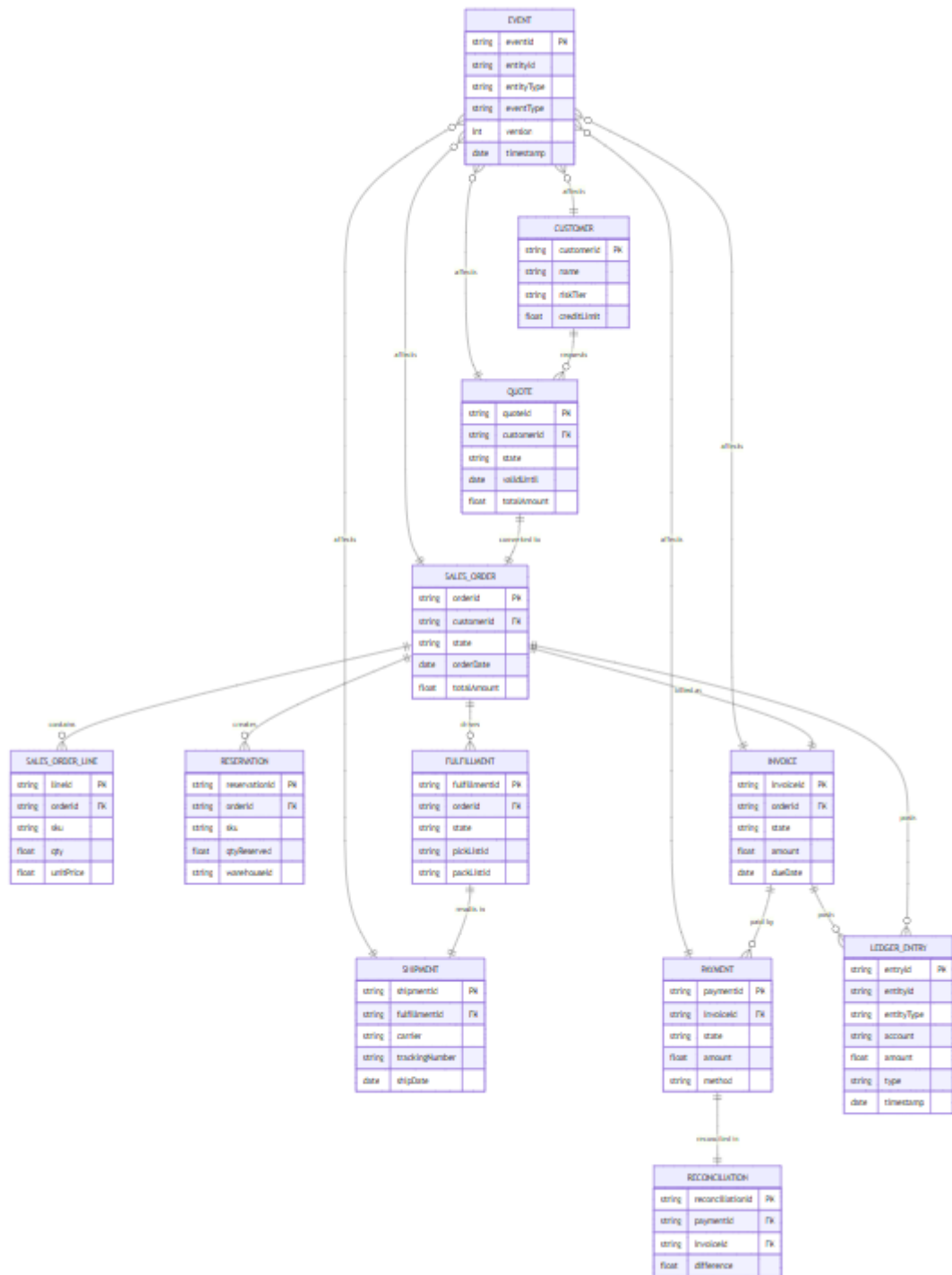
B. Canonical Hypermedia Transition Map

C. Canonical MCP Function Catalog

D. Canonical Mapping Tables (Oracle/SAP/Dynamics → Canonical)

E. Mini-Foundational ERP schema (SQL + HATEOAS)

Just tell me where you want to go next.



Designing markdown doc

I need to create a detailed markdown design doc, using info from the doc and our prior model. No tools needed. I'll structure it with sections like title, overview, architecture, data model (tables), MCP catalog (list functions with schemas), hypermedia map (per state actions), mapping tables (per vendor), and scaffold instructions (tech-agnostic). I'll aim for clarity and conciseness, but since the user wants full design, I'll make sure it's detailed enough. Keeping it under verbosity 1, but still thorough.

Foundation ERP – Super-O2C Implementation Design

Developer-ready Markdown spec

1. Scope and principles

Goal: Implement a minimal but real **Foundation ERP** that supports **Super-O2C** end-to-end, behind the **Integration Hub**, with:

- **Canonical data model** (ERP-agnostic, eventsourced, reconstructable)
- **Hypermedia REST API** (HATEOAS)
- **MCP function catalog** (for the Navigator)
- **Mapping tables** to Oracle, SAP, Dynamics
- **Scaffolding** for incremental implementation

“ERPs become: projections and executors, not the constitutional source of truth.”

“On the mesh you can host: an open source, HATEOAS-native ERP core, AI-generated/maintained.”

Foundation ERP is **not** the constitutional truth—just a projection that obeys the Mesh, Ledger, Charter, and Authority Engines.

2. Canonical Super-O2C data model

2.1 Entity overview

- **Customer**
- **Quote**
- **SalesOrder**
- **SalesOrderLine**
- **Reservation**
- **Fulfillment**
- **Shipment**
- **Invoice**

- **Payment**
- **Reconciliation**
- **LedgerEntry**
- **Event** (temporal backbone; may live in Mesh but mirrored here as projection)

2.2 SQL-style schema (canonical)

2.2.1 customer

```
CREATE TABLE customer (
  customer_id  VARCHAR(64) PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  credit_limit NUMERIC(18,2),
  risk_tier    VARCHAR(32),
  governance_flags JSONB    -- e.g. ["RequiresTier2Approval"]
);
```

2.2.2 quote

```
CREATE TABLE quote (
  quote_id  VARCHAR(64) PRIMARY KEY,
  customer_id VARCHAR(64) NOT NULL REFERENCES customer(customer_id),
  state     VARCHAR(32) NOT NULL, -- Draft, Sent, Accepted, Rejected, ConvertedToOrder
  valid_until DATE,
  total_amount NUMERIC(18,2),
  currency    VARCHAR(3),
  created_at  TIMESTAMP NOT NULL,
  updated_at  TIMESTAMP NOT NULL
);
```

2.2.3 sales_order

```
CREATE TABLE sales_order (
  order_id  VARCHAR(64) PRIMARY KEY,
  customer_id VARCHAR(64) NOT NULL REFERENCES customer(customer_id),
  state     VARCHAR(32) NOT NULL, -- Draft, Confirmed, Reserved, Allocated, Picked, Packed,
  Shipped, FulfillmentComplete, Invoiced, Paid, Closed
  order_date DATE NOT NULL,
```

```
total_amount NUMERIC(18,2),
currency   VARCHAR(3),
created_at  TIMESTAMP NOT NULL,
updated_at  TIMESTAMP NOT NULL
);
```

2.2.4 sales_order_line

```
CREATE TABLE sales_order_line (
  line_id   VARCHAR(64) PRIMARY KEY,
  order_id  VARCHAR(64) NOT NULL REFERENCES sales_order(order_id),
  sku       VARCHAR(64) NOT NULL,
  description VARCHAR(255),
  qty       NUMERIC(18,4) NOT NULL,
  unit_price NUMERIC(18,4) NOT NULL,
  currency  VARCHAR(3),
  line_total NUMERIC(18,2) GENERATED ALWAYS AS (qty * unit_price) STORED
);
```

2.2.5 reservation

```
CREATE TABLE reservation (
  reservation_id VARCHAR(64) PRIMARY KEY,
  order_id       VARCHAR(64) NOT NULL REFERENCES sales_order(order_id),
  sku            VARCHAR(64) NOT NULL,
  qty_reserved   NUMERIC(18,4) NOT NULL,
  warehouse_id   VARCHAR(64) NOT NULL,
  created_at     TIMESTAMP NOT NULL
);
```

2.2.6 fulfillment

```
CREATE TABLE fulfillment (
  fulfillment_id VARCHAR(64) PRIMARY KEY,
  order_id       VARCHAR(64) NOT NULL REFERENCES sales_order(order_id),
  state          VARCHAR(32) NOT NULL, -- Planned, Picked, Packed, Shipped, Completed
);
```

```
pick_list_id VARCHAR(64),
pack_list_id VARCHAR(64),
created_at   TIMESTAMP NOT NULL,
updated_at   TIMESTAMP NOT NULL
);
```

2.2.7 shipment

```
CREATE TABLE shipment (
    shipment_id VARCHAR(64) PRIMARY KEY,
    fulfillment_id VARCHAR(64) NOT NULL REFERENCES fulfillment(fulfillment_id),
    carrier       VARCHAR(128),
    tracking_number VARCHAR(128),
    ship_date     DATE,
    created_at    TIMESTAMP NOT NULL
);
```

2.2.8 invoice

```
CREATE TABLE invoice (
    invoice_id VARCHAR(64) PRIMARY KEY,
    order_id   VARCHAR(64) NOT NULL REFERENCES sales_order(order_id),
    state      VARCHAR(32) NOT NULL, -- Draft, Posted, Paid, Reconciled
    amount     NUMERIC(18,2) NOT NULL,
    currency   VARCHAR(3),
    due_date   DATE,
    created_at TIMESTAMP NOT NULL,
    posted_at  TIMESTAMP
);
```

2.2.9 payment

```
CREATE TABLE payment (
    payment_id VARCHAR(64) PRIMARY KEY,
    invoice_id VARCHAR(64) REFERENCES invoice(invoice_id),
    state      VARCHAR(32) NOT NULL, -- Received, Applied, Reconciled
```

```
amount    NUMERIC(18,2) NOT NULL,  
currency  VARCHAR(3),  
method    VARCHAR(64),  
received_at TIMESTAMP NOT NULL  
);
```

2.2.10 reconciliation

```
CREATE TABLE reconciliation (  
    reconciliation_id VARCHAR(64) PRIMARY KEY,  
    payment_id       VARCHAR(64) NOT NULL REFERENCES payment(payment_id),  
    invoice_id       VARCHAR(64) NOT NULL REFERENCES invoice(invoice_id),  
    difference        NUMERIC(18,2) NOT NULL,  
    created_at        TIMESTAMP NOT NULL  
);
```

2.2.11 ledger_entry

```
CREATE TABLE ledger_entry (  
    entry_id  VARCHAR(64) PRIMARY KEY,  
    entity_id VARCHAR(64) NOT NULL,  
    entity_type VARCHAR(64) NOT NULL, -- SalesOrder, Invoice, Payment, etc.  
    account   VARCHAR(64) NOT NULL,  
    amount    NUMERIC(18,2) NOT NULL,  
    currency  VARCHAR(3),  
    type       VARCHAR(16) NOT NULL, -- debit / credit  
    timestamp  TIMESTAMP NOT NULL  
);
```

2.2.12 event (projection of Mesh Ledger)

```
CREATE TABLE event (  
    event_id  VARCHAR(64) PRIMARY KEY,  
    entity_id VARCHAR(64) NOT NULL,  
    entity_type VARCHAR(64) NOT NULL,  
    event_type VARCHAR(64) NOT NULL,
```

```
version    INT NOT NULL,  
timestamp  TIMESTAMP NOT NULL,  
payload    JSONB NOT NULL  
);
```

“Every change = event: AIProposedAction, UserApproved, PaymentExecuted, RollbackInitiated, etc.”

3. Canonical MCP function catalog

Each function is:

- **Published by Integration Hub**
- **Backed by Foundation ERP**
- **Discovered via MCP**
- **Constrained by governance**

3.1 Naming convention

- **Domain prefix:** o2c_
- **Verb:** create, update, allocate, ship, invoice, apply_payment, etc.
- **Example:** o2c_create_quote, o2c_convert_quote_to_order

3.2 Function list (Super-O2C)

1. **o2c_create_quote**
2. **o2c_add_quote_line**
3. **o2c_send_quote**
4. **o2c_accept_quote**
5. **o2c_convert_quote_to_order**
6. **o2c_add_order_line**
7. **o2c_confirm_order**
8. **o2c_allocate_stock**
9. **o2c_schedule_delivery**
10. **o2c_generate_pick_list**
11. **o2c_confirm_pick**
12. **o2c_confirm_pack**
13. **o2c_ship_order**

- 14. **o2c_generate_invoice**
- 15. **o2c_post_invoice**
- 16. **o2c_register_payment**
- 17. **o2c_apply_payment_to_invoice**
- 18. **o2c_reconcile_payment**

3.3 Example MCP function definitions

3.3.1 o2c_create_quote

```
{  
  "name": "o2c_create_quote",  
  "description": "Create a draft quote for a customer.",  
  "domain": "OrderToCash",  
  "schema": {  
    "type": "object",  
    "properties": {  
      "customerId": { "type": "string" },  
      "currency": { "type": "string" }  
    },  
    "required": ["customerId", "currency"]  
  }  
}
```

3.3.2 o2c_convert_quote_to_order

```
{  
  "name": "o2c_convert_quote_to_order",  
  "description": "Convert an accepted quote into a sales order.",  
  "domain": "OrderToCash",  
  "schema": {  
    "type": "object",  
    "properties": {  
      "quoteId": { "type": "string" }  
    },  
  },  
}
```

```
    "required": ["orderId"]
  }
}
```

3.3.3 o2c_allocate_stock

```
{
  "name": "o2c_allocate_stock",
  "description": "Allocate stock for a confirmed sales order.",
  "domain": "Inventory",
  "schema": {
    "type": "object",
    "properties": {
      "orderId": { "type": "string" },
      "warehouseId": { "type": "string" }
    },
    "required": ["orderId", "warehouseId"]
  }
}
```

3.3.4 o2c_generate_invoice

```
{
  "name": "o2c_generate_invoice",
  "description": "Generate an invoice for a shipped order.",
  "domain": "Finance",
  "schema": {
    "type": "object",
    "properties": {
      "orderId": { "type": "string" }
    },
    "required": ["orderId"]
  }
}
```

3.3.5 o2c_apply_payment_to_invoice

```
{
  "name": "o2c_apply_payment_to_invoice",
  "description": "Apply a received payment to an invoice.",
  "domain": "Finance",
  "schema": {
    "type": "object",
    "properties": {
      "paymentId": { "type": "string" },
      "invoiceId": { "type": "string" }
    },
    "required": ["paymentId", "invoiceId"]
  }
}
```

“The Integration Hub publishes: available actions, schemas, constraints, input/output types... This is capability discovery.”

4. Canonical hypermedia transition map

“Hypermedia affordances expose: available actions, required data, governance requirements, risk/impact hints.”

4.1 General pattern

For each entity instance:

- GET /o2c/orders/{orderId} returns:
 - **state**
 - **data**
 - **_links**: HATEOAS transitions
 - **_actions**: MCP function bindings

4.2 Quote transitions

4.2.1 Quote.state = Draft

Affordances:

- **Add line item**

- **Send quote**
- **Delete quote**

```
{
  "_links": {
    "self": { "href": "/o2c/quotes/Q123" },
    "add-line-item": {
      "href": "/o2c/quotes/Q123/lines",
      "method": "POST",
      "mcpFunction": "o2c_add_quote_line"
    },
    "send-quote": {
      "href": "/o2c/quotes/Q123/send",
      "method": "POST",
      "mcpFunction": "o2c_send_quote"
    }
  }
}
```

4.2.2 Quote.state = Accepted

Affordances:

- **Convert to order**

```
{
  "_links": {
    "convert-to-order": {
      "href": "/o2c/quotes/Q123/convert",
      "method": "POST",
      "mcpFunction": "o2c_convert_quote_to_order"
    }
  }
}
```

4.3 Sales order transitions

4.3.1 SalesOrder.state = Confirmed

Affordances:

- **Allocate stock**
- **Schedule delivery**
- **Generate pick list**

```
{
  "_links": {
    "allocate-stock": {
      "href": "/o2c/orders/SO456/allocate-stock",
      "method": "POST",
      "mcpFunction": "o2c_allocate_stock",
      "governance": {
        "domain": "InventoryMovement",
        "requiredAuthority": "Tier2"
      }
    },
    "schedule-delivery": {
      "href": "/o2c/orders/SO456/schedule-delivery",
      "method": "POST",
      "mcpFunction": "o2c_schedule_delivery"
    }
  }
}
```

4.3.2 SalesOrder.state = Shipped

Affordances:

- **Generate invoice**

```
{
  "_links": {
    "generate-invoice": {
      "href": "/o2c/orders/SO456/generate-invoice",
```

```

    "method": "POST",
    "mcpFunction": "o2c_generate_invoice",
    "governance": {
      "domain": "FinancialCommitments",
      "requiredAuthority": "Tier2"
    }
  }
}

```

4.4 Invoice transitions

Invoice.state = Posted

Affordances:

- **Register payment**
- **Apply payment**

```

{
  "_links": {
    "register-payment": {
      "href": "/o2c/invoices/INV789/register-payment",
      "method": "POST",
      "mcpFunction": "o2c_register_payment"
    },
    "apply-payment": {
      "href": "/o2c/invoices/INV789/apply-payment",
      "method": "POST",
      "mcpFunction": "o2c_apply_payment_to_invoice"
    }
  }
}

```

“The Navigator simply follows the hypermedia graph... Governance modifies it by removing transitions the user cannot approve.”

5. Mapping tables – Oracle / SAP / Dynamics → Canonical

These are **conceptual mapping tables** for Integration Hub adapters.

5.1 Customer

Canonical	Oracle EBS / Fusion	SAP (SD / FI)	Dynamics 365 FO
customer_id	HZ_PARTIES.PARTY_ID	KNA1-KUNNR	CustTable.AccountNum
name	HZ_PARTIES.PARTY_NAME	KNA1-NAME1	CustTable.Name
credit_limit	AR_CUST_PROFILES.CREDIT_LIMIT	KNKK-KLIMK	CustCreditMax.CreditMax
risk_tier	custom DFF / UDA	custom field	custom field

5.2 Sales order

Canonical	Oracle	SAP	Dynamics 365 FO
order_id	OE_ORDER_HEADERS_ALL.ORDER_NUMBE R	VBAK- VBELN	SalesTable.SalesId
customer_id	OE_ORDER_HEADERS_ALL.SOLD_TO_ORG_I D	VBAK- KUNN R	SalesTable.CustAccount
order_date	OE_ORDER_HEADERS_ALL.ORDERED_DATE	VBAK- AUDAT e	SalesTable.CreatedDateTim
state	FLOW_STATUS_CODE	VBAK- GBSTK / LFSTK	SalesTable.SalesStatus

5.3 Sales order line

Canonical	Oracle	SAP	Dynamics 365 FO
line_id	OE_ORDER_LINES_ALL.LINE_ID	VBAP-POSNR	SalesLine.ReclId
order_id	OE_ORDER_LINES_ALL.HEADER_ID	VBAP-VBELN	SalesLine.SalesId
sku	OE_ORDER_LINES_ALL.INVENTORY_ITEM_ID	VBAP-MATNR	SalesLine.ItemId
qty	OE_ORDER_LINES_ALL.ORDERED_QUANTITY	VBAP-KWMENG	SalesLine.SalesQty
unit_price	OE_ORDER_LINES_ALL.UNIT_SELLING_PRICE	VBAP-NETPR	SalesLine.SalesPrice

5.4 Invoice

Canonical	Oracle AR	SAP FI / SD	Dynamics 365 FO
invoice_id	RA_CUSTOMER_TRX_ALL.TRX_NUMBER	VBRK-VBELN	CustInvoiceJour.InvoiceId
order_id	RA_CUSTOMER_TRX_LINES_ALL.INTERFACE_LINE_ATTRIBUTE1	VBRP-AUBEL	CustInvoiceTrans.SalesId
amount	RA_CUSTOMER_TRX_ALL.AMOUNT	VBRK-NETWR	CustInvoiceJour.InvoiceAmount
state	COMPLETE_FLAG / status	open/cleared in FI	CustInvoiceJour.SettlementStatus

5.5 Payment

Canonical	Oracle AR	SAP FI	Dynamics 365 FO
payment_id	AR_CASH_RECEIPTS_ALL.RECEIPT_NUMBER	BSID-BELNR	CustTransOpen.Invoice / CustSettlement
invoice_id	AR_RECEIVABLE_APPLICATIONS_ALL.APPLIED_CUSTOMER_TRX_ID	BSID-AUGBL	CustSettlement.Invoice
amount	AR_CASH_RECEIPTS_ALL.AMOUNT	BSID-DMBTR	CustTrans.Amount Cur

Adapters in the Integration Hub:

- **Read vendor objects**
- **Map to canonical**
- **Expose via Hypermedia + MCP**

6. Scaffold instructions for developers

6.1 Suggested repo structure

foundation-erp/

docs/

super-o2c-design.md # this file

services/

api/ # Hypermedia REST API
domain/ # domain models + services
persistence/ # DB migrations + repositories
adapters/
 oracle/
 sap/
 dynamics/
tests/
integration/
contract/

6.2 Implementation phases

1. Phase 1 – Core domain + persistence

- Implement tables in section 2.2.
- Implement domain models for Customer, Quote, SalesOrder, SalesOrderLine, Invoice, Payment.
- Implement basic CRUD (no hypermedia yet).

2. Phase 2 – Super-O2C process logic

- Implement state transitions:
 - Quote: Draft → Sent → Accepted → ConvertedToOrder
 - SalesOrder: Draft → Confirmed → Reserved → Allocated → Shipped → Invoiced → Paid
- Implement services:
 - QuoteService, OrderService, InvoiceService, PaymentService.

3. Phase 3 – Hypermedia API

- Implement GET /o2c/quotes/{id}, GET /o2c/orders/{id}, GET /o2c/invoices/{id}.
- Add _links and _actions as per section 4.
- Ensure transitions are **state-driven**, not role-driven.

4. Phase 4 – MCP function endpoints

- Implement HTTP endpoints backing MCP functions:
 - e.g. POST /mcp/o2c_create_quote, POST /mcp/o2c_convert_quote_to_order.

- Ensure schemas match section 3.3.
- Wire them to domain services.

5. Phase 5 – Integration Hub adapters

- Implement mapping logic using tables in section 5.
- For now, stub external ERPs; later, connect real Oracle/SAP/Dynamics.

6. Phase 6 – Event mirroring

- Subscribe to Mesh events (from Ledger).
- Project relevant events into event and update projections (quotes, orders, invoices).
- Ensure idempotency and replay safety.

6.3 Design constraints

- **No business logic in controllers**
All logic in domain services; controllers only orchestrate HTTP + hypermedia.
- **State transitions are explicit**
No “magic updates”—every state change corresponds to a domain method and an event.
- **Hypermedia is the contract**
Frontends and AI Navigator must rely only on _links and MCP schemas.
- **ERP-agnostic**
Never leak Oracle/SAP/Dynamics fields into canonical models.

“The Integration Hub defines the contract. The Foundation ERP only needs to implement that contract.”

If you’d like, next step I can:

- Turn this into a **concrete API contract** (OpenAPI + example payloads), or
- Produce **developer stories / tickets** sliced by phase and component.

Here's a **clean, production-ready Node.js + TypeScript scaffold** for the Foundation ERP — tuned specifically for:

- **better-sqlite3** (fast, synchronous, safe for ERP-style workloads)
- **Hypermedia-first REST**
- **MCP function endpoints**
- **Domain-driven folder structure**
- **Event-sourced projections**
- **Developer velocity**

This is the exact structure I'd give a team building the Mini-Foundational ERP.

I'll give you:

1. 📁 **Project structure**
2. 📦 **package.json**
3. ⚙️ **tsconfig.json**
4. 🗄️ **Database layer (better-sqlite3)**
5. 🏗️ **Domain modules (Super-O2C)**
6. 🌐 **Hypermedia API scaffold**
7. 🗨️ **MCP function endpoints scaffold**
8. 🧪 **Testing setup**
9. 🚀 **Run instructions**

Everything is ready for your team to clone and start coding.

Foundation Implementation basic

1. 📁 Project Structure
foundation-erp/

src/

app.ts

server.ts

db/

connection.ts

migrations/

001_init.sql

002_o2c_tables.sql

domain/

customer/

customer.model.ts

customer.repo.ts

customer.service.ts

quote/

quote.model.ts

quote.repo.ts

quote.service.ts

order/

order.model.ts

order.repo.ts

order.service.ts

invoice/

invoice.model.ts

invoice.repo.ts

invoice.service.ts

payment/

payment.model.ts

payment.repo.ts

payment.service.ts

api/

hypermedia/

quote.routes.ts

order.routes.ts

invoice.routes.ts

mcp/
o2c.functions.ts

utils/
id.ts
errors.ts
hypermedia.ts

test/
quote.test.ts
order.test.ts

package.json
tsconfig.json
.env
README.md

This structure enforces:

- **Domain boundaries**
- **Clean separation of hypermedia vs MCP**
- **SQLite migrations**
- **Testability**

2. 📦 package.json

```
{  
  "name": "foundation-erp",  
  "version": "1.0.0",  
  "type": "module",  
  "scripts": {  
    "dev": "ts-node-dev --respawn src/server.ts",  
    "start": "node dist/server.js",  
  },  
}
```

```
"build": "tsc",
"migrate": "node dist/db/migrate.js",
"test": "jest"
},
"dependencies": {
  "express": "^4.19.0",
  "better-sqlite3": "^9.0.0",
  "dotenv": "^16.4.0"
},
"devDependencies": {
  "@types/express": "^4.17.21",
  "@types/node": "^20.11.0",
  "ts-node-dev": "^2.0.0",
  "typescript": "^5.3.3",
  "jest": "^29.7.0",
  "@types/jest": "^29.5.5"
}
}
```

3. tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ES2022",
    "moduleResolution": "node",
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "resolveJsonModule": true
  }
}
```

```
}  
}
```

4. Database Layer (better-sqlite3)

src/db/connection.ts

```
import Database from "better-sqlite3";  
import path from "path";  
  
const dbPath = path.join(process.cwd(), "foundation.db");  
  
export const db = new Database(dbPath, {  
  verbose: console.log  
});
```

```
db.pragma("journal_mode = WAL");
```

```
db.pragma("foreign_keys = ON");
```

Example migration file

```
src/db/migrations/002_o2c_tables.sql
```

```
CREATE TABLE IF NOT EXISTS customer (  
  customer_id TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  credit_limit REAL,  
  risk_tier TEXT,  
  governance_flags TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS quote (  
  quote_id TEXT PRIMARY KEY,  
  customer_id TEXT NOT NULL,  
  state TEXT NOT NULL,
```

```
valid_until TEXT,  
total_amount REAL,  
currency TEXT,  
created_at TEXT,  
updated_at TEXT,  
FOREIGN KEY (customer_id) REFERENCES customer(customer_id)  
);
```

Your team can browse this with the VS Code SQLite extension.

5. Domain Modules (Super-O2C)

Example: Quote domain

quote.model.ts

```
export interface Quote {  
  quoteId: string;  
  customerId: string;  
  state: "Draft" | "Sent" | "Accepted" | "Rejected" | "ConvertedToOrder";  
  totalAmount: number;  
  currency: string;  
  createdAt: string;  
  updatedAt: string;  
}
```

quote.repo.ts

```
import { db } from "../../db/connection.js";  
import { Quote } from "./quote.model.js";  
  
export const QuoteRepo = {  
  create(quote: Quote) {  
    const stmt = db.prepare(`  
      INSERT INTO quote (quote_id, customer_id, state, total_amount, currency, created_at,  
updated_at)  
      VALUES (?, ?, ?, ?, ?, ?, ?)  
    `);
```



```

`);
stmt.run(
  quote.quoteId,
  quote.customerId,
  quote.state,
  quote.totalAmount,
  quote.currency,
  quote.createdAt,
  quote.updatedAt
);
return quote;
},

findById(id: string): Quote | undefined {
  return db.prepare(`SELECT * FROM quote WHERE quote_id = ?`).get(id);
}
};

```

quote.service.ts

```

import { QuoteRepo } from "../quote.repo.js";
import { Quote } from "../quote.model.js";
import { newId } from "../../utils/id.js";

export const QuoteService = {
  create(customerId: string, currency: string): Quote {
    const now = new Date().toISOString();
    const quote: Quote = {
      quoteId: newId("Q"),
      customerId,
      state: "Draft",
      totalAmount: 0,

```

```

    currency,
    createdAt: now,
    updatedAt: now
  };
  return QuoteRepo.create(quote);
}
};

```

6. Hypermedia API Scaffold

src/api/hypermedia/quote.routes.ts

```

import { Router } from "express";
import { QuoteRepo } from "../../domain/quote/quote.repo.js";
import { QuoteService } from "../../domain/quote/quote.service.js";
import { hypermedia } from "../../utils/hypermedia.js";

```

```

export const quoteRouter = Router();

```

```

quoteRouter.post("/", (req, res) => {
  const { customerId, currency } = req.body;
  const quote = QuoteService.create(customerId, currency);
  res.json(hypermedia.quote(quote));
});

```

```

quoteRouter.get("/:id", (req, res) => {
  const quote = QuoteRepo.findById(req.params.id);
  if (!quote) return res.status(404).json({ error: "Not found" });
  res.json(hypermedia.quote(quote));
});

```

src/utils/hypermedia.ts

```

export const hypermedia = {

```

```

quote(quote) {
  return {
    ...quote,
    _links: {
      self: { href: `/quotes/${quote.quoteId}` },
      "add-line-item": {
        href: `/quotes/${quote.quoteId}/lines`,
        method: "POST",
        mcpFunction: "o2c_add_quote_line"
      },
      ...(quote.state === "Accepted" && {
        "convert-to-order": {
          href: `/quotes/${quote.quoteId}/convert`,
          method: "POST",
          mcpFunction: "o2c_convert_quote_to_order"
        }
      })
    }
  };
}
};

```

7. 🧠 MCP Function Endpoints Scaffold

src/api/mcp/o2c.functions.ts

```

import { Router } from "express";

import { QuoteService } from "../../domain/quote/quote.service.js";

export const mcpRouter = Router();

mcpRouter.post("/o2c_create_quote", (req, res) => {

```

```
const { customerId, currency } = req.body;

const quote = QuoteService.create(customerId, currency);

res.json({ result: quote });

});
```

This is where the Navigator will call into the ERP.

8. Testing Setup

test/quote.test.ts

```
import { QuoteService } from "../src/domain/quote/quote.service.js";

test("creates a quote", () => {
  const q = QuoteService.create("CUST1", "USD");
  expect(q.customerId).toBe("CUST1");
  expect(q.state).toBe("Draft");
});
```

9. Run Instructions

npm install

npm run dev

Database file appears as:

foundation.db

Your VS Code SQLite extension will pick it up automatically.

If you want...

I can generate next:

✓ Full Super-O2C domain implementation

✓ Full hypermedia transition map in code

✓ Full MCP function catalog in code

✓ Integration Hub adapter stubs

✓ Event replay engine scaffold

✓ Governance injection middleware

Just tell me what you want to build next.

Structuring implementation design

I need to create a markdown document that outlines the full domain implementation. I'll keep it concise but cover key entities like Customer, Quote, SalesOrder, Fulfillment, Invoice, Payment, and Events. I'll include state machines and key methods, but it's more about design than full code. I'll also add TypeScript snippets for clarity. The goal is to structure it with headings and sections for each aggregate, focusing on the design rather than the full codebase.

Foundation ERP – Super-O2C Domain Implementation

Node.js + TypeScript, better-sqlite3 – Developer Design Doc

1. Domain overview

Super-O2C spans these core aggregates:

- **Customer** – commercial party
- **Quote** – pre-order commercial offer
- **SalesOrder** – committed commercial agreement
- **SalesOrderLine** – line-level items
- **Reservation** – stock reservation
- **Fulfillment** – pick/pack/ship execution
- **Shipment** – goods issue
- **Invoice** – AR billing
- **Payment** – cash receipt
- **Reconciliation** – payment ↔ invoice matching
- **LedgerEntry** – financial postings
- **Event** – temporal backbone (projection of Mesh Ledger)

States and transitions are driven by the constitutional model:

“The quote is now in state Draft... Convert to Order... Order is now in state Confirmed... Allocate stock... GoodsShipped... InvoiceGenerated... PaymentApplied...”

2. Shared types and utilities

```
// src/domain/common/types.ts
```

```
export type Currency = "USD" | "EUR" | "AED" | string;
```

```
export type QuoteState =
```

```
| "Draft"  
| "Sent"  
| "Accepted"  
| "Rejected"  
| "ConvertedToOrder";
```

```
export type OrderState =
```

```
| "Draft"  
| "Confirmed"  
| "Reserved"  
| "Allocated"  
| "Picked"  
| "Packed"  
| "Shipped"  
| "FulfillmentComplete"  
| "Invoiced"  
| "Paid"  
| "Closed";
```

```
export type InvoiceState = "Draft" | "Posted" | "Paid" | "Reconciled";
```

```
export type PaymentState = "Received" | "Applied" | "Reconciled";
```

```
export interface BaseEntity {
```

```
  createdAt: string;  
  updatedAt: string;
```

```
}
```

```
// src/utils/id.ts
```

```
let counter = 0;
```

```
export function newId(prefix: string): string {
```

```
counter += 1;

return `${prefix}${Date.now()}-${counter}`;

}
```

3. Customer domain

3.1 Model

```
// src/domain/customer/customer.model.ts

import type { BaseEntity } from "../../common/types.js";

export interface Customer extends BaseEntity {

  customerId: string;

  name: string;

  creditLimit?: number;

  riskTier?: string;

  governanceFlags?: string[]; // e.g. ["RequiresTier2Approval"]

}
```

3.2 Repository

```
// src/domain/customer/customer.repo.ts

import { db } from "../../db/connection.js";

import type { Customer } from "../../customer.model.js";

export const CustomerRepo = {

  create(customer: Customer): Customer {

    db.prepare(

      `INSERT INTO customer

      (customer_id, name, credit_limit, risk_tier, governance_flags, created_at, updated_at)

      VALUES (?, ?, ?, ?, ?, ?, ?)`

    ).run(

      customer.customerId,

      customer.name,
```

```

    customer.creditLimit ?? null,
    customer.riskTier ?? null,
    customer.governanceFlags ? JSON.stringify(customer.governanceFlags) : null,
    customer.createdAt,
    customer.updatedAt
  );
  return customer;
},

```

```

findById(id: string): Customer | undefined {
  const row = db
    .prepare(`SELECT * FROM customer WHERE customer_id = ?`)
    .get(id);
  if (!row) return undefined;
  return {
    customerId: row.customer_id,
    name: row.name,
    creditLimit: row.credit_limit ?? undefined,
    riskTier: row.risk_tier ?? undefined,
    governanceFlags: row.governance_flags
      ? JSON.parse(row.governance_flags)
      : undefined,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

```

3.3 Service

```

// src/domain/customer/customer.service.ts

import { newId } from "../../utils/id.js";

```

```
import type { Customer } from "../customer.model.js";
import { CustomerRepo } from "../customer.repo.js";
```

```
export const CustomerService = {
  create(name: string, creditLimit?: number): Customer {
    const now = new Date().toISOString();
    const customer: Customer = {
      customerId: newId("C"),
      name,
      creditLimit,
      createdAt: now,
      updatedAt: now
    };
    return CustomerRepo.create(customer);
  }
};
```

4. Quote domain

4.1 Model

```
// src/domain/quote/quote.model.ts
```

```
import type { BaseEntity, QuoteState, Currency } from "../common/types.js";
```

```
export interface Quote extends BaseEntity {
  quoteId: string;
  customerId: string;
  state: QuoteState;
  totalAmount: number;
  currency: Currency;
  validUntil?: string;
}
```

4.2 Repository

```
// src/domain/quote/quote.repo.ts

import { db } from "../../db/connection.js";
import type { Quote } from "../quote.model.js";

export const QuoteRepo = {
  create(quote: Quote): Quote {
    db.prepare(
      `INSERT INTO quote
        (quote_id, customer_id, state, valid_until, total_amount, currency, created_at,
updated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      quote.quoteId,
      quote.customerId,
      quote.state,
      quote.validUntil ?? null,
      quote.totalAmount,
      quote.currency,
      quote.createdAt,
      quote.updatedAt
    );
    return quote;
  },

  updateState(id: string, state: Quote["state"]): void {
    db.prepare(
      `UPDATE quote SET state = ?, updated_at = ? WHERE quote_id = ?`
    ).run(state, new Date().toISOString(), id);
  },
}
```

```

findById(id: string): Quote | undefined {
  const row = db.prepare(`SELECT * FROM quote WHERE quote_id = ?`).get(id);
  if (!row) return undefined;
  return {
    quoteId: row.quote_id,
    customerId: row.customer_id,
    state: row.state,
    validUntil: row.valid_until ?? undefined,
    totalAmount: row.total_amount,
    currency: row.currency,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

```

4.3 Service (state transitions)

```

// src/domain/quote/quote.service.ts
import { newId } from "../../utils/id.js";
import type { Quote } from "./quote.model.js";
import { QuoteRepo } from "./quote.repo.js";
import { SalesOrderService } from "../../order/order.service.js";

export const QuoteService = {
  create(customerId: string, currency: string): Quote {
    const now = new Date().toISOString();
    const quote: Quote = {
      quoteId: newId("Q"),
      customerId,
      state: "Draft",
      totalAmount: 0,

```

```
    currency,  
    createdAt: now,  
    updatedAt: now  
  };  
  return QuoteRepo.create(quote);  
},
```

```
send(quoteId: string): Quote {  
  const quote = this.require(quoteId);  
  if (quote.state !== "Draft") {  
    throw new Error("Only Draft quotes can be sent");  
  }  
  QuoteRepo.updateState(quoteId, "Sent");  
  return this.require(quoteId);  
},
```

```
accept(quoteId: string): Quote {  
  const quote = this.require(quoteId);  
  if (quote.state !== "Sent") {  
    throw new Error("Only Sent quotes can be accepted");  
  }  
  QuoteRepo.updateState(quoteId, "Accepted");  
  return this.require(quoteId);  
},
```

```
convertToOrder(quoteId: string) {  
  const quote = this.require(quoteId);  
  if (quote.state !== "Accepted") {  
    throw new Error("Only Accepted quotes can be converted");  
  }  
}
```

```

    const order = SalesOrderService.createFromQuote(quote);
    QuoteRepo.updateState(quoteId, "ConvertedToOrder");
    return order;
  },

  require(id: string): Quote {
    const q = QuoteRepo.findById(id);
    if (!q) throw new Error("Quote not found");
    return q;
  }
};

```

5. Sales order domain

5.1 Models

```
// src/domain/order/order.model.ts
```

```
import type { BaseEntity, OrderState, Currency } from "../common/types.js";
```

```

export interface SalesOrder extends BaseEntity {
  orderId: string;
  customerId: string;
  state: OrderState;
  orderDate: string;
  totalAmount: number;
  currency: Currency;
}

```

```

export interface SalesOrderLine {
  lineId: string;
  orderId: string;
  sku: string;
}

```

```
description?: string;
qty: number;
unitPrice: number;
currency: Currency;
lineTotal: number;
}
```

5.2 Repositories

```
// src/domain/order/order.repo.ts
import { db } from "../../db/connection.js";
import type { SalesOrder, SalesOrderLine } from "../order.model.js";

export const SalesOrderRepo = {
  create(order: SalesOrder): SalesOrder {
    db.prepare(
      `INSERT INTO sales_order
      (order_id, customer_id, state, order_date, total_amount, currency, created_at,
updated_at)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      order.orderId,
      order.customerId,
      order.state,
      order.orderDate,
      order.totalAmount,
      order.currency,
      order.createdAt,
      order.updatedAt
    );
    return order;
  },
```

```

updateState(id: string, state: SalesOrder["state"]): void {
  db.prepare(
    `UPDATE sales_order SET state = ?, updated_at = ? WHERE order_id = ?`
  ).run(state, new Date().toISOString(), id);
},

updateTotal(id: string, total: number): void {
  db.prepare(
    `UPDATE sales_order SET total_amount = ?, updated_at = ? WHERE order_id = ?`
  ).run(total, new Date().toISOString(), id);
},

findById(id: string): SalesOrder | undefined {
  const row = db
    .prepare(`SELECT * FROM sales_order WHERE order_id = ?`)
    .get(id);
  if (!row) return undefined;
  return {
    orderId: row.order_id,
    customerId: row.customer_id,
    state: row.state,
    orderDate: row.order_date,
    totalAmount: row.total_amount,
    currency: row.currency,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

```



```

export const SalesOrderLineRepo = {
  create(line: SalesOrderLine): SalesOrderLine {
    db.prepare(
      `INSERT INTO sales_order_line
        (line_id, order_id, sku, description, qty, unit_price, currency, line_total)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      line.lineId,
      line.orderId,
      line.sku,
      line.description ?? null,
      line.qty,
      line.unitPrice,
      line.currency,
      line.lineTotal
    );
    return line;
  },

```

```

  findByOrder(orderId: string): SalesOrderLine[] {
    const rows = db
      .prepare(`SELECT * FROM sales_order_line WHERE order_id = ?`)
      .all(orderId);
    return rows.map((r) => ({
      lineId: r.line_id,
      orderId: r.order_id,
      sku: r.sku,
      description: r.description ?? undefined,
      qty: r.qty,
      unitPrice: r.unit_price,

```

```

        currency: r.currency,
        lineTotal: r.line_total
    }));
}
};

```

5.3 Service (state machine)

```

// src/domain/order/order.service.ts
import { newId } from "../../utils/id.js";
import type { SalesOrder, SalesOrderLine } from "./order.model.js";
import { SalesOrderRepo, SalesOrderLineRepo } from "./order.repo.js";
import type { Quote } from "../../quote/quote.model.js";

export const SalesOrderService = {
  createFromQuote(quote: Quote): SalesOrder {
    const now = new Date().toISOString();
    const order: SalesOrder = {
      orderId: newId("SO"),
      customerId: quote.customerId,
      state: "Draft",
      orderDate: now.slice(0, 10),
      totalAmount: quote.totalAmount,
      currency: quote.currency,
      createdAt: now,
      updatedAt: now
    };
    return SalesOrderRepo.create(order);
  },

  addLine(
    orderId: string,

```

```

    sku: string,
    qty: number,
    unitPrice: number,
    description?: string
  ): SalesOrderLine {
    const order = this.require(orderId);
    if (order.state !== "Draft") {
      throw new Error("Can only add lines to Draft orders");
    }
    const line: SalesOrderLine = {
      lineId: newId("SOL"),
      orderId,
      sku,
      qty,
      unitPrice,
      description,
      currency: order.currency,
      lineTotal: qty * unitPrice
    };
    SalesOrderLineRepo.create(line);
    const lines = SalesOrderLineRepo.findByOrder(orderId);
    const total = lines.reduce((sum, l) => sum + l.lineTotal, 0);
    SalesOrderRepo.updateTotal(orderId, total);
    return line;
  },

```

```

confirm(orderId: string): SalesOrder {
  const order = this.require(orderId);
  if (order.state !== "Draft") {
    throw new Error("Only Draft orders can be confirmed");
  }

```

```

    }

    SalesOrderRepo.updateState(orderId, "Confirmed");
    return this.require(orderId);
},

markAllocated(orderId: string): SalesOrder {
    const order = this.require(orderId);
    if (order.state !== "Confirmed" && order.state !== "Reserved") {
        throw new Error("Order must be Confirmed or Reserved to allocate");
    }
    SalesOrderRepo.updateState(orderId, "Allocated");
    return this.require(orderId);
},

markShipped(orderId: string): SalesOrder {
    const order = this.require(orderId);
    if (order.state !== "Allocated" && order.state !== "Picked" && order.state !== "Packed") {
        throw new Error("Order must be Allocated/Picked/Packed to ship");
    }
    SalesOrderRepo.updateState(orderId, "Shipped");
    return this.require(orderId);
},

markInvoiced(orderId: string): SalesOrder {
    const order = this.require(orderId);
    if (order.state !== "Shipped") {
        throw new Error("Order must be Shipped to be invoiced");
    }
    SalesOrderRepo.updateState(orderId, "Invoiced");
    return this.require(orderId);
}

```

```
},
```

```
markPaid(orderId: string): SalesOrder {  
  const order = this.require(orderId);  
  if (order.state !== "Invoiced") {  
    throw new Error("Order must be Invoiced to be marked Paid");  
  }  
  SalesOrderRepo.updateState(orderId, "Paid");  
  return this.require(orderId);  
},
```

```
require(id: string): SalesOrder {  
  const o = SalesOrderRepo.findById(id);  
  if (!o) throw new Error("Order not found");  
  return o;  
}  
};
```

6. Fulfillment and shipment domain

6.1 Models

```
// src/domain/fulfillment/fulfillment.model.ts  
import type { BaseEntity } from "../../common/types.js";  
  
export type FulfillmentState =  
  | "Planned"  
  | "Picked"  
  | "Packed"  
  | "Shipped"  
  | "Completed";
```

```
export interface Fulfillment extends BaseEntity {
  fulfillmentId: string;
  orderId: string;
  state: FulfillmentState;
  pickListId?: string;
  packListId?: string;
}
```

```
export interface Shipment extends BaseEntity {
  shipmentId: string;
  fulfillmentId: string;
  carrier?: string;
  trackingNumber?: string;
  shipDate?: string;
}
```

6.2 Repository + service (simplified)

```
// src/domain/fulfillment/fulfillment.repo.ts
import { db } from "../../db/connection.js";
import type { Fulfillment, Shipment } from "../fulfillment.model.js";

export const FulfillmentRepo = {
  create(f: Fulfillment): Fulfillment {
    db.prepare(
      `INSERT INTO fulfillment
      (fulfillment_id, order_id, state, pick_list_id, pack_list_id, created_at, updated_at)
      VALUES (?, ?, ?, ?, ?, ?, ?)`
    ).run(
      f.fulfillmentId,
      f.orderId,
      f.state,
```

```

    f.pickListId ?? null,
    f.packListId ?? null,
    f.createdAt,
    f.updatedAt
  );
  return f;
},

```

```

updateState(id: string, state: Fulfillment["state"]): void {
  db.prepare(
    `UPDATE fulfillment SET state = ?, updated_at = ? WHERE fulfillment_id = ?`
  ).run(state, new Date().toISOString(), id);
},

```

```

findByOrder(orderId: string): Fulfillment | undefined {
  const row = db
    .prepare(`SELECT * FROM fulfillment WHERE order_id = ?`)
    .get(orderId);
  if (!row) return undefined;
  return {
    fulfillmentId: row.fulfillment_id,
    orderId: row.order_id,
    state: row.state,
    pickListId: row.pick_list_id ?? undefined,
    packListId: row.pack_list_id ?? undefined,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

```

```

export const ShipmentRepo = {
  create(s: Shipment): Shipment {
    db.prepare(
      `INSERT INTO shipment
      (shipment_id, fulfillment_id, carrier, tracking_number, ship_date, created_at)
      VALUES (?, ?, ?, ?, ?, ?)`
    ).run(
      s.shipmentId,
      s.fulfillmentId,
      s.carrier ?? null,
      s.trackingNumber ?? null,
      s.shipDate ?? null,
      s.createdAt
    );
    return s;
  }
};

// src/domain/fulfillment/fulfillment.service.ts
import { newId } from "../../utils/id.js";
import { FulfillmentRepo, ShipmentRepo } from "../fulfillment.repo.js";
import type { Fulfillment, Shipment } from "../fulfillment.model.js";
import { SalesOrderService } from "../../order/order.service.js";

export const FulfillmentService = {
  plan(orderId: string): Fulfillment {
    const order = SalesOrderService.require(orderId);
    if (order.state !== "Confirmed" && order.state !== "Allocated") {
      throw new Error("Order must be Confirmed/Allocated to plan fulfillment");
    }
  }
};

```



```
const now = new Date().toISOString();
const f: Fulfillment = {
  fulfillmentId: newId("FUL"),
  orderId,
  state: "Planned",
  createdAt: now,
  updatedAt: now
};
return FulfillmentRepo.create(f);
},
```

```
markPicked(orderId: string): Fulfillment {
  const f = this.requireByOrder(orderId);
  FulfillmentRepo.updateState(f.fulfillmentId, "Picked");
  return this.requireByOrder(orderId);
},
```

```
markPacked(orderId: string): Fulfillment {
  const f = this.requireByOrder(orderId);
  FulfillmentRepo.updateState(f.fulfillmentId, "Packed");
  return this.requireByOrder(orderId);
},
```

```
ship(orderId: string, carrier?: string, trackingNumber?: string): Shipment {
  const f = this.requireByOrder(orderId);
  const now = new Date().toISOString();
  const shipment: Shipment = {
    shipmentId: newId("SHIP"),
    fulfillmentId: f.fulfillmentId,
    carrier,
```

```

        trackingNumber,
        shipDate: now.slice(0, 10),
        createdAt: now,
        updatedAt: now
    };
    ShipmentRepo.create(shipment);
    FulfillmentRepo.updateState(f.fulfillmentId, "Shipped");
    SalesOrderService.markShipped(orderId);
    return shipment;
},

requireByOrder(orderId: string): Fulfillment {
    const f = FulfillmentRepo.findByOrder(orderId);
    if (!f) throw new Error("Fulfillment not found for order");
    return f;
}
};

```

7. Invoice domain

7.1 Model

```

// src/domain/invoice/invoice.model.ts
import type { BaseEntity, InvoiceState, Currency } from "../common/types.js";

export interface Invoice extends BaseEntity {
    invoiceId: string;
    orderId: string;
    state: InvoiceState;
    amount: number;
    currency: Currency;
    dueDate?: string;
}

```

```
    postedAt?: string;
}
```

7.2 Repository + service

```
// src/domain/invoice/invoice.repo.ts
```

```
import { db } from "../../db/connection.js";
```

```
import type { Invoice } from "../invoice.model.js";
```

```
export const InvoiceRepo = {
  create(inv: Invoice): Invoice {
    db.prepare(
      `INSERT INTO invoice
        (invoice_id, order_id, state, amount, currency, due_date, created_at, posted_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      inv.invoiceId,
      inv.orderId,
      inv.state,
      inv.amount,
      inv.currency,
      inv.dueDate ?? null,
      inv.createdAt,
      inv.postedAt ?? null
    );
    return inv;
  },
```

```
  updateState(id: string, state: Invoice["state"]): void {
    db.prepare(
      `UPDATE invoice SET state = ?, posted_at = CASE WHEN ? = 'Posted' THEN ? ELSE
        posted_at END WHERE invoice_id = ?`
    ).run(state, state, new Date().toISOString(), id);
```

```
},
```

```
findByld(id: string): Invoice | undefined {  
  const row = db  
    .prepare(`SELECT * FROM invoice WHERE invoice_id = ?`)  
    .get(id);  
  if (!row) return undefined;  
  return {  
    invoiceId: row.invoice_id,  
    orderId: row.order_id,  
    state: row.state,  
    amount: row.amount,  
    currency: row.currency,  
    dueDate: row.due_date ?? undefined,  
    createdAt: row.created_at,  
    postedAt: row.posted_at ?? undefined,  
    updatedAt: row.created_at // or separate column if needed  
  };  
}  
};
```

```
// src/domain/invoice/invoice.service.ts
```

```
import { newId } from "../../utils/id.js";  
import { InvoiceRepo } from "./invoice.repo.js";  
import type { Invoice } from "./invoice.model.js";  
import { SalesOrderService } from "../order/order.service.js";
```

```
export const InvoiceService = {  
  generateForOrder(orderId: string): Invoice {  
    const order = SalesOrderService.require(orderId);  
    if (order.state !== "Shipped") {
```

```

    throw new Error("Order must be Shipped to generate invoice");
}

const now = new Date().toISOString();
const inv: Invoice = {
    invoiceId: newId("INV"),
    orderId,
    state: "Draft",
    amount: order.totalAmount,
    currency: order.currency,
    createdAt: now,
    updatedAt: now
};

const created = InvoiceRepo.create(inv);
SalesOrderService.markInvoiced(orderId);
return created;
},

```

```

post(invoiceId: string): Invoice {
    const inv = this.require(invoiceId);
    if (inv.state !== "Draft") {
        throw new Error("Only Draft invoices can be posted");
    }
    InvoiceRepo.updateState(invoiceId, "Posted");
    return this.require(invoiceId);
},

```

```

require(id: string): Invoice {
    const inv = InvoiceRepo.findById(id);
    if (!inv) throw new Error("Invoice not found");
    return inv;
}

```

```
}  
};
```

8. Payment and reconciliation domain

8.1 Models

```
// src/domain/payment/payment.model.ts
```

```
import type { BaseEntity, PaymentState, Currency } from "../../common/types.js";
```

```
export interface Payment extends BaseEntity {
```

```
  paymentId: string;
```

```
  invoiceId?: string;
```

```
  state: PaymentState;
```

```
  amount: number;
```

```
  currency: Currency;
```

```
  method?: string;
```

```
  receivedAt: string;
```

```
}
```

```
export interface Reconciliation extends BaseEntity {
```

```
  reconciliationId: string;
```

```
  paymentId: string;
```

```
  invoiceId: string;
```

```
  difference: number;
```

```
}
```

8.2 Repository + service

```
// src/domain/payment/payment.repo.ts
```

```
import { db } from "../../db/connection.js";
```

```
import type { Payment, Reconciliation } from "../payment.model.js";
```

```
export const PaymentRepo = {
```

```

create(p: Payment): Payment {
  db.prepare(
    `INSERT INTO payment
      (payment_id, invoice_id, state, amount, currency, method, received_at)
      VALUES (?, ?, ?, ?, ?, ?, ?)`
  ).run(
    p.paymentId,
    p.invoiceId ?? null,
    p.state,
    p.amount,
    p.currency,
    p.method ?? null,
    p.receivedAt
  );
  return p;
},

```

```

updateState(id: string, state: Payment["state"]): void {
  db.prepare(`UPDATE payment SET state = ? WHERE payment_id = ?`).run(
    state,
    id
  );
},

```

```

findById(id: string): Payment | undefined {
  const row = db
    .prepare(`SELECT * FROM payment WHERE payment_id = ?`)
    .get(id);
  if (!row) return undefined;
  return {

```

```

    paymentId: row.payment_id,
    invoiceId: row.invoice_id ?? undefined,
    state: row.state,
    amount: row.amount,
    currency: row.currency,
    method: row.method ?? undefined,
    receivedAt: row.received_at,
    createdAt: row.received_at,
    updatedAt: row.received_at
  };
}
};

export const ReconciliationRepo = {
  create(r: Reconciliation): Reconciliation {
    db.prepare(
      `INSERT INTO reconciliation
      (reconciliation_id, payment_id, invoice_id, difference, created_at)
      VALUES (?, ?, ?, ?, ?)`
    ).run(
      r.reconciliationId,
      r.paymentId,
      r.invoiceId,
      r.difference,
      r.createdAt
    );
    return r;
  }
};

// src/domain/payment/payment.service.ts

```



```

import { newId } from "../../utils/id.js";
import { PaymentRepo, ReconciliationRepo } from "../payment.repo.js";
import type { Payment, Reconciliation } from "../payment.model.js";
import { InvoiceService } from "../invoice/invoice.service.js";
import { SalesOrderService } from "../order/order.service.js";

export const PaymentService = {
  register(amount: number, currency: string, method?: string): Payment {
    const now = new Date().toISOString();
    const p: Payment = {
      paymentId: newId("PMT"),
      state: "Received",
      amount,
      currency,
      method,
      receivedAt: now,
      createdAt: now,
      updatedAt: now
    };
    return PaymentRepo.create(p);
  },

  applyToInvoice(paymentId: string, invoiceId: string): { payment: Payment; reconciliation: Reconciliation } {
    const payment = this.require(paymentId);
    const invoice = InvoiceService.require(invoiceId);

    if (payment.state !== "Received") {
      throw new Error("Payment must be Received to be applied");
    }

    if (invoice.state !== "Posted") {

```

```

    throw new Error("Invoice must be Posted to receive payment");
}

const difference = payment.amount - invoice.amount;
const now = new Date().toISOString();

const rec: Reconciliation = {
    reconciliationId: newId("REC"),
    paymentId,
    invoiceId,
    difference,
    createdAt: now,
    updatedAt: now
};
ReconciliationRepo.create(rec);

PaymentRepo.updateState(paymentId, "Applied");
InvoiceService.post(invoiceId); // already posted; could add Paid state here
SalesOrderService.markPaid(invoice.orderId);

return { payment: this.require(paymentId), reconciliation: rec };
},

require(id: string): Payment {
    const p = PaymentRepo.findById(id);
    if (!p) throw new Error("Payment not found");
    return p;
}
};

```

9. Event and ledger (projection layer)

For now, you can keep this minimal—just enough to record domain events for replay.

```
// src/domain/event/event.model.ts
export interface DomainEvent {
  eventId: string;
  entityId: string;
  entityType: string;
  eventType: string;
  version: number;
  timestamp: string;
  payload: unknown;
}

// src/domain/event/event.repo.ts
import { db } from "../../db/connection.js";
import type { DomainEvent } from "../event.model.js";

export const EventRepo = {
  append(e: DomainEvent): void {
    db.prepare(
      `INSERT INTO event
      (event_id, entity_id, entity_type, event_type, version, timestamp, payload)
      VALUES (?, ?, ?, ?, ?, ?, ?)`
    ).run(
      e.eventId,
      e.entityId,
      e.entityType,
      e.eventType,
      e.version,
      e.timestamp,
      JSON.stringify(e.payload)
    )
  }
}
```

```
);  
}  
};
```

You can then gradually wire each service method to emit events (e.g. QuoteCreated, OrderConfirmed, InvoiceGenerated, PaymentApplied).

10. End-to-end Super-O2C flow (service-level)

A minimal happy-path sequence using these services:

```
// Pseudo-orchestrator  
const customer = CustomerService.create("ACME", 50000);  
const quote = QuoteService.create(customer.customerId, "USD");  
// (optionally add lines via SalesOrderService.addLine after conversion, or model quote lines)  
QuoteService.send(quote.quoteId);  
QuoteService.accept(quote.quoteId);  
const order = QuoteService.convertToOrder(quote.quoteId);  
SalesOrderService.confirm(order.orderId);  
const fulfillment = FulfillmentService.plan(order.orderId);  
FulfillmentService.markPicked(order.orderId);  
FulfillmentService.markPacked(order.orderId);  
const shipment = FulfillmentService.ship(order.orderId, "DHL", "TRACK-123");  
const invoice = InvoiceService.generateForOrder(order.orderId);  
InvoiceService.post(invoice.invoiceId);  
const payment = PaymentService.register(invoice.amount, invoice.currency, "BankTransfer");  
PaymentService.applyToInvoice(payment.paymentId, invoice.invoiceId);
```

This matches the narrative:

“AI proposes: ‘Convert quote Q123 to sales order.’ ... ‘Allocate stock and schedule delivery’ ... ‘Generate invoice for shipped goods’ ... ‘Apply payment PMT001 to invoice INV789.’”

If you’d like, I can now:

- Turn this into **concrete Express routes** (hypermedia + MCP), or
- Add **event emission** to every state transition, or

- Slice this into **tickets / tasks** for your team.

Yes—what you’ve got for Super-O2C is not just a domain, it’s a **pattern**.

You can absolutely extend it to **P2P (Procure-to-Pay)** and other domains by reusing the same constitutional primitives:

- Canonical data model
- Hypermedia state machine
- MCP function catalog
- Vendor mapping tables
- Event-sourced semantics

I’ll keep this tight and concrete.

1. Reuse the pattern, change the nouns

For O2C you have:

- **Customer → Quote → SalesOrder → Fulfillment → Invoice → Payment**

For P2P, the canonical chain is:

- **Supplier → Requisition → PurchaseOrder → Receipt → SupplierInvoice → Payment**

So the P2P equivalents are:

- **Customer → Supplier**
- **Quote → (optional) RFQ / Quotation**
- **SalesOrder → PurchaseOrder**
- **Shipment/Fulfillment → GoodsReceipt**
- **Invoice (AR) → SupplierInvoice (AP)**
- **Payment (incoming) → Payment (outgoing)**

Everything else—events, hypermedia, MCP, governance—stays structurally the same.

2. Canonical P2P entities (aligned with ERPs)

Core aggregates:

- **Supplier** – vendor master
- **Requisition** – internal request to buy
- **RequisitionLine**
- **PurchaseOrder**
- **PurchaseOrderLine**
- **GoodsReceipt** (or Receipt)
- **SupplierInvoice**
- **Payment** (AP)
- **Reconciliation** (invoice ↔ PO ↔ receipt)
- **LedgerEntry**
- **Event**

These map cleanly to Oracle / SAP / Dynamics:

- Oracle: PO_REQUISITION_HEADERS_ALL, PO_HEADERS_ALL, RCV_SHIPMENT_HEADERS, AP_INVOICES_ALL, AP_CHECKS_ALL
- SAP: EBAN, EKKO/EKPO, MSEG, RBKP/RSEG, BSEG
- Dynamics: PurchReqTable, PurchTable/PurchLine, InventTrans, VendInvoiceJour, VendTrans

3. Canonical P2P states

Requisition.state

- Draft
- Submitted
- Approved
- ConvertedToPO
- Rejected

PurchaseOrder.state

- Draft
- Approved
- Sent
- PartiallyReceived
- FullyReceived
- Invoiced
- Paid
- Closed

GoodsReceipt.state

- Draft
- Posted

SupplierInvoice.state

- Draft
- Posted
- Paid
- Reconciled

Payment.state (AP)

- Proposed
- Approved
- Executed
- Reconciled

Same pattern as O2C: **state-driven, not role-driven.**

4. Canonical P2P data model (sketch)

// Supplier

```
interface Supplier {  
  supplierId: string;  
  name: string;  
  paymentTerms?: string;  
  riskTier?: string;  
  governanceFlags?: string[];  
}
```

// Requisition

```
interface Requisition {  
  requisitionId: string;  
  requesterId: string;  
  state: "Draft" | "Submitted" | "Approved" | "ConvertedToPO" | "Rejected";  
}
```

```
totalAmount: number;
currency: string;
createdAt: string;
updatedAt: string;
}
```

```
interface RequisitionLine {
  lineId: string;
  requisitionId: string;
  sku?: string;
  description: string;
  qty: number;
  unitPrice: number;
  currency: string;
  lineTotal: number;
}
```

```
// PurchaseOrder
interface PurchaseOrder {
  pold: string;
  supplierId: string;
  state:
    | "Draft"
    | "Approved"
    | "Sent"
    | "PartiallyReceived"
    | "FullyReceived"
    | "Invoiced"
    | "Paid"
    | "Closed";
  orderDate: string;
  totalAmount: number;
  currency: string;
  createdAt: string;
  updatedAt: string;
}
```

```
interface PurchaseOrderLine {
  lineId: string;
  pold: string;
  sku?: string;
  description: string;
  qty: number;
  unitPrice: number;
  currency: string;
  lineTotal: number;
}
```



```

}

// GoodsReceipt
interface GoodsReceipt {
  receiptId: string;
  pold: string;
  state: "Draft" | "Posted";
  receiptDate: string;
  createdAt: string;
  updatedAt: string;
}

// SupplierInvoice
interface SupplierInvoice {
  invoiceId: string;
  pold?: string;
  state: "Draft" | "Posted" | "Paid" | "Reconciled";
  amount: number;
  currency: string;
  dueDate?: string;
  createdAt: string;
  updatedAt: string;
}

// Payment (AP)
interface ApPayment {
  paymentId: string;
  supplierId: string;
  state: "Proposed" | "Approved" | "Executed" | "Reconciled";
  amount: number;
  currency: string;
  method?: string;
  paymentDate?: string;
  createdAt: string;
  updatedAt: string;
}

```

5. Canonical P2P hypermedia transitions

Same pattern as O2C:

- GET /p2p/requisitions/{id} → state + _links
- GET /p2p/pos/{id} → state + _links
- GET /p2p/invoices/{id} → state + _links

Example: Requisition in Draft

```

{
  "requisitionId": "REQ123",
  "state": "Draft",

```

```

"_links": {
  "self": { "href": "/p2p/requisitions/REQ123" },
  "add-line": {
    "href": "/p2p/requisitions/REQ123/lines",
    "method": "POST",
    "mcpFunction": "p2p_add_requisition_line"
  },
  "submit": {
    "href": "/p2p/requisitions/REQ123/submit",
    "method": "POST",
    "mcpFunction": "p2p_submit_requisition"
  }
}
}

```

PurchaseOrder in Approved

```

{
  "pold": "PO456",
  "state": "Approved",
  "_links": {
    "self": { "href": "/p2p/pos/PO456" },
    "send-to-supplier": {
      "href": "/p2p/pos/PO456/send",
      "method": "POST",
      "mcpFunction": "p2p_send_po"
    },
    "post-receipt": {
      "href": "/p2p/pos/PO456/receipts",
      "method": "POST",
      "mcpFunction": "p2p_post_goods_receipt"
    }
  }
}

```

6. Canonical P2P MCP function catalog

Mirror the O2C naming:

- p2p_create_requisition
- p2p_add_requisition_line
- p2p_submit_requisition
- p2p_approve_requisition
- p2p_convert_requisition_to_po
- p2p_add_po_line
- p2p_approve_po
- p2p_send_po
- p2p_post_goods_receipt
- p2p_create_supplier_invoice
- p2p_post_supplier_invoice

- p2p_propose_payment
- p2p_approve_payment
- p2p_execute_payment

Example schema:

```
{
  "name": "p2p_create_requisition",
  "description": "Create a draft requisition.",
  "domain": "ProcureToPay",
  "schema": {
    "type": "object",
    "properties": {
      "requesterId": { "type": "string" },
      "currency": { "type": "string" }
    },
    "required": ["requesterId", "currency"]
  }
}
```

7. Mapping tables – P2P Canonical → ERP

PurchaseOrder

Canonical	Oracle	SAP	Dynamics 365 FO
pold	PO_HEADERS_ALL.SEGMENT1	EKKO-EBELN	PurchTable.PurchId
supplierId	PO_HEADERS_ALL.VENDOR_ID	EKKO-LIFNR	PurchTable.VendAccount
orderDate	PO_HEADERS_ALL.CREATION_DATE	EKKO-BEDAT	PurchTable.CreatedDateTime
state	AUTHORIZATION_STATUS	EKKO-STATU	PurchTable.PurchStatus

GoodsReceipt

Canonical	Oracle	SAP	Dynamics 365 FO
receiptId	RCV_SHIPMENT_HEADERS.SHIPMENT_HEADER_ID	MSEG-MBLNR	InventTrans.InventTransId
pold	RCV_TRANSACTIONS.PO_HEADER_ID	MSEG-EBELN	PurchTable.PurchId

SupplierInvoice

Canonical	Oracle AP	SAP FI/M	Dynamics 365 FO
invoiceId	AP_INVOICES_ALL.INVOICE_NUM	RBKP-BELNR	VendInvoiceJour.InvoiceId
pold	AP_INVOICE_DISTRIBUTIONS_ALL.PO_HEADER_ID	RSEG-EBELN	VendInvoiceTrans.PurchId

Payment (AP)

paymentId AP_CHECKS_ALL.CHECK_ID REGUH-VBLNR VendTrans.Voucher

supplierId AP_CHECKS_ALL.VENDOR_ID REGUH-LIFNR VendTable.AccountNum

Adapters in the Integration Hub do the same job as for O2C: **normalize vendor specifics into canonical P2P**.

8. Implementation shape (Node + TypeScript, same as O2C)

You'd mirror your O2C structure:

```
src/
  domain/
    p2p/
      supplier/
      requisition/
      po/
      receipt/
      supplierInvoice/
      payment/
  api/
    hypermedia/
      p2p.requisition.routes.ts
      p2p.po.routes.ts
      p2p.invoice.routes.ts
  mcp/
    p2p.functions.ts
```

Each domain module:

- **Model** – canonical interfaces
- **Repo** – better-sqlite3 persistence
- **Service** – state transitions + invariants
- **Hypermedia** – _links based on state
- **MCP** – thin wrappers over services

Exactly like Super-O2C.

9. Why this stays “constitutional”

By extending O2C's pattern to P2P, you preserve:

- **Mesh as truth** – ERP is still a projection
- **Events as backbone** – P2P events join O2C events in the Ledger
- **Hypermedia as contract** – AI sees only allowed transitions
- **MCP as capability surface** – AI discovers P2P functions the same way
- **Governance as modifier** – Charter/Authority engines filter P2P transitions too

So yes: you can—and should—treat P2P as **another canonical domain** hanging off the same constitutional fabric.

If you want, next step we can:

- Draft a **full P2P ERD (Mermaid)**, or
- Write a **P2P domain implementation doc** like the Super-O2C one, or
- Design the **shared cross-domain events** (e.g. supplier risk impacting O2C credit).

Foundation ERP – P2P Domain Implementation

Aligned with Canonical O2C, Node.js + TypeScript + better-sqlite3

1. Domain overview

Canonical **Procure-to-Pay (P2P)** mirrors Super-O2C structurally:

- **Supplier** – vendor master
- **Requisition** → **RequisitionLine** – internal request
- **PurchaseOrder** → **PurchaseOrderLine** – external commitment
- **GoodsReceipt** – receipt of goods/services
- **SupplierInvoice** – AP invoice
- **ApPayment** – outgoing payment
- **Reconciliation** – invoice/payment matching
- **LedgerEntry, Event** – shared with O2C

States are **process-driven**, not role-driven.

2. Shared types

```
// src/domain/p2p/common.ts
```

```
export type Currency = "USD" | "EUR" | "AED" | string;
```

```
export type RequisitionState =
```

```
  | "Draft"  
  | "Submitted"  
  | "Approved"  
  | "ConvertedToPO"  
  | "Rejected";
```

```
export type PoState =
```

```
  | "Draft"  
  | "Approved"  
  | "Sent"  
  | "PartiallyReceived"  
  | "FullyReceived"  
  | "Invoiced"  
  | "Paid"  
  | "Closed";
```

```
export type GoodsReceiptState = "Draft" | "Posted";
```

```
export type SupplierInvoiceState = "Draft" | "Posted" | "Paid" | "Reconciled";
```

```
export type ApPaymentState =
```

```
  | "Proposed"  
  | "Approved"
```

```
| "Executed"  
| "Reconciled";
```

```
export interface BaseEntity {  
  createdAt: string;  
  updatedAt: string;  
}
```

3. Supplier domain

3.1 Model

```
// src/domain/p2p/supplier/supplier.model.ts  
import type { BaseEntity } from "../../common.js";
```

```
export interface Supplier extends BaseEntity {  
  supplierId: string;  
  name: string;  
  paymentTerms?: string;  
  riskTier?: string;  
  governanceFlags?: string[];  
}
```

3.2 Repository + service

```
// src/domain/p2p/supplier/supplier.repo.ts  
import { db } from "../../../db/connection.js";  
import type { Supplier } from "../supplier.model.js";
```

```
export const SupplierRepo = {  
  create(s: Supplier): Supplier {  
    db.prepare(  
      `INSERT INTO supplier  
        (supplier_id, name, payment_terms, risk_tier, governance_flags, created_at, updated_at)  
        VALUES (?, ?, ?, ?, ?, ?, ?)`  
    ).run(  
      s.supplierId,  
      s.name,  
      s.paymentTerms ?? null,  
      s.riskTier ?? null,  
      s.governanceFlags ? JSON.stringify(s.governanceFlags) : null,  
      s.createdAt,  
      s.updatedAt  
    );  
    return s;  
  },
```

```
  findById(id: string): Supplier | undefined {  
    const row = db  
      .prepare(`SELECT * FROM supplier WHERE supplier_id = ?`)
```

```

        .get(id);
    if (!row) return undefined;
    return {
        supplierId: row.supplier_id,
        name: row.name,
        paymentTerms: row.payment_terms ?? undefined,
        riskTier: row.risk_tier ?? undefined,
        governanceFlags: row.governance_flags
            ? JSON.parse(row.governance_flags)
            : undefined,
        createdAt: row.created_at,
        updatedAt: row.updated_at
    };
}
};
// src/domain/p2p/supplier/supplier.service.ts
import { newId } from "../../utils/id.js";
import type { Supplier } from "../supplier.model.js";
import { SupplierRepo } from "../supplier.repo.js";

export const SupplierService = {
    create(name: string, paymentTerms?: string): Supplier {
        const now = new Date().toISOString();
        const s: Supplier = {
            supplierId: newId("SUP"),
            name,
            paymentTerms,
            createdAt: now,
            updatedAt: now
        };
        return SupplierRepo.create(s);
    },

    require(id: string): Supplier {
        const s = SupplierRepo.findById(id);
        if (!s) throw new Error("Supplier not found");
        return s;
    }
};

```

4. Requisition domain

4.1 Models

```

// src/domain/p2p/requisition/requisition.model.ts
import type { BaseEntity, RequisitionState, Currency } from "../../common.js";

export interface Requisition extends BaseEntity {

```

```

    requisitionId: string;
    requesterId: string;
    state: RequisitionState;
    totalAmount: number;
    currency: Currency;
}

```

```

export interface RequisitionLine {
    lineId: string;
    requisitionId: string;
    sku?: string;
    description: string;
    qty: number;
    unitPrice: number;
    currency: Currency;
    lineTotal: number;
}

```

4.2 Repositories

```

// src/domain/p2p/requisition/requisition.repo.ts
import { db } from "../../db/connection.js";
import type { Requisition, RequisitionLine } from "../requisition.model.js";

export const RequisitionRepo = {
    create(r: Requisition): Requisition {
        db.prepare(
            `INSERT INTO requisition
            (requisition_id, requester_id, state, total_amount, currency, created_at, updated_at)
            VALUES (?, ?, ?, ?, ?, ?, ?)`
        ).run(
            r.requisitionId,
            r.requesterId,
            r.state,
            r.totalAmount,
            r.currency,
            r.createdAt,
            r.updatedAt
        );
        return r;
    },

    updateState(id: string, state: Requisition["state"]): void {
        db.prepare(
            `UPDATE requisition SET state = ?, updated_at = ? WHERE requisition_id = ?`
        ).run(state, new Date().toISOString(), id);
    },
}

```



```

updateTotal(id: string, total: number): void {
  db.prepare(
    `UPDATE requisition SET total_amount = ?, updated_at = ? WHERE requisition_id = ?`
  ).run(total, new Date().toISOString(), id);
},

findById(id: string): Requisition | undefined {
  const row = db
    .prepare(`SELECT * FROM requisition WHERE requisition_id = ?`)
    .get(id);
  if (!row) return undefined;
  return {
    requisitionId: row.requisition_id,
    requesterId: row.requester_id,
    state: row.state,
    totalAmount: row.total_amount,
    currency: row.currency,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

export const RequisitionLineRepo = {
  create(l: RequisitionLine): RequisitionLine {
    db.prepare(
      `INSERT INTO requisition_line
        (line_id, requisition_id, sku, description, qty, unit_price, currency, line_total)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      l.lineId,
      l.requisitionId,
      l.sku ?? null,
      l.description,
      l.qty,
      l.unitPrice,
      l.currency,
      l.lineTotal
    );
    return l;
  },

  findByRequisition(id: string): RequisitionLine[] {
    const rows = db
      .prepare(`SELECT * FROM requisition_line WHERE requisition_id = ?`)
      .all(id);

```

```

return rows.map((r) => ({
  lineId: r.line_id,
  requisitionId: r.requisition_id,
  sku: r.sku ?? undefined,
  description: r.description,
  qty: r.qty,
  unitPrice: r.unit_price,
  currency: r.currency,
  lineTotal: r.line_total
}));
}
};

4.3 Service (state machine)
// src/domain/p2p/requisition/requisition.service.ts
import { newId } from "../../utils/id.js";
import type { Requisition, RequisitionLine } from "./requisition.model.js";
import {
  RequisitionRepo,
  RequisitionLineRepo
} from "./requisition.repo.js";
import { PurchaseOrderService } from "../../po/po.service.js";

export const RequisitionService = {
  create(requesterId: string, currency: string): Requisition {
    const now = new Date().toISOString();
    const r: Requisition = {
      requisitionId: newId("REQ"),
      requesterId,
      state: "Draft",
      totalAmount: 0,
      currency,
      createdAt: now,
      updatedAt: now
    };
    return RequisitionRepo.create(r);
  },

  addLine(
    requisitionId: string,
    description: string,
    qty: number,
    unitPrice: number,
    sku?: string
  ): RequisitionLine {
    const req = this.require(requisitionId);
    if (req.state !== "Draft") {

```

```

    throw new Error("Can only add lines to Draft requisitions");
  }
  const line: RequisitionLine = {
    lineId: newId("RQL"),
    requisitionId,
    sku,
    description,
    qty,
    unitPrice,
    currency: req.currency,
    lineTotal: qty * unitPrice
  };
  RequisitionLineRepo.create(line);
  const lines = RequisitionLineRepo.findByRequisition(requisitionId);
  const total = lines.reduce((s, l) => s + l.lineTotal, 0);
  RequisitionRepo.updateTotal(requisitionId, total);
  return line;
},

submit(id: string): Requisition {
  const r = this.require(id);
  if (r.state !== "Draft") throw new Error("Only Draft can be submitted");
  RequisitionRepo.updateState(id, "Submitted");
  return this.require(id);
},

approve(id: string): Requisition {
  const r = this.require(id);
  if (r.state !== "Submitted") throw new Error("Only Submitted can be approved");
  RequisitionRepo.updateState(id, "Approved");
  return this.require(id);
},

convertToPo(id: string, supplierId: string) {
  const r = this.require(id);
  if (r.state !== "Approved") {
    throw new Error("Only Approved can be converted to PO");
  }
  const lines = RequisitionLineRepo.findByRequisition(id);
  const po = PurchaseOrderService.createFromRequisition(r, supplierId, lines);
  RequisitionRepo.updateState(id, "ConvertedToPO");
  return po;
},

require(id: string): Requisition {
  const r = RequisitionRepo.findById(id);

```

```
    if (!r) throw new Error("Requisition not found");
    return r;
  }
};
```

5. Purchase order domain

5.1 Models

```
// src/domain/p2p/po/po.model.ts
import type { BaseEntity, PoState, Currency } from "../common.js";
```

```
export interface PurchaseOrder extends BaseEntity {
  pold: string;
  supplierId: string;
  state: PoState;
  orderDate: string;
  totalAmount: number;
  currency: Currency;
}
```

```
export interface PurchaseOrderLine {
  lineId: string;
  pold: string;
  sku?: string;
  description: string;
  qty: number;
  unitPrice: number;
  currency: Currency;
  lineTotal: number;
}
```

5.2 Repositories + service

```
// src/domain/p2p/po/po.repo.ts
import { db } from "../../db/connection.js";
import type { PurchaseOrder, PurchaseOrderLine } from "./po.model.js";

export const PurchaseOrderRepo = {
  create(po: PurchaseOrder): PurchaseOrder {
    db.prepare(
      `INSERT INTO purchase_order
      (po_id, supplier_id, state, order_date, total_amount, currency, created_at, updated_at)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      po.pold,
      po.supplierId,
      po.state,
      po.orderDate,
      po.totalAmount,
```

```

    po.currency,
    po.createdAt,
    po.updatedAt
  );
  return po;
},

updateState(id: string, state: PurchaseOrder["state"]): void {
  db.prepare(
    `UPDATE purchase_order SET state = ?, updated_at = ? WHERE po_id = ?`
  ).run(state, new Date().toISOString(), id);
},

updateTotal(id: string, total: number): void {
  db.prepare(
    `UPDATE purchase_order SET total_amount = ?, updated_at = ? WHERE po_id = ?`
  ).run(total, new Date().toISOString(), id);
},

findById(id: string): PurchaseOrder | undefined {
  const row = db.prepare(`SELECT * FROM purchase_order WHERE po_id = ?`).get(
    id
  );
  if (!row) return undefined;
  return {
    pold: row.po_id,
    supplierId: row.supplier_id,
    state: row.state,
    orderDate: row.order_date,
    totalAmount: row.total_amount,
    currency: row.currency,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

export const PurchaseOrderLineRepo = {
  create(l: PurchaseOrderLine): PurchaseOrderLine {
    db.prepare(
      `INSERT INTO purchase_order_line
      (line_id, po_id, sku, description, qty, unit_price, currency, line_total)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      l.lineId,
      l.pold,

```

```

    l.sku ?? null,
    l.description,
    l.qty,
    l.unitPrice,
    l.currency,
    l.lineTotal
  );
  return l;
},

```

```

findByPo(pold: string): PurchaseOrderLine[] {
  const rows = db
    .prepare(`SELECT * FROM purchase_order_line WHERE po_id = ?`)
    .all(pold);
  return rows.map((r) => ({
    lineId: r.line_id,
    pold: r.po_id,
    sku: r.sku ?? undefined,
    description: r.description,
    qty: r.qty,
    unitPrice: r.unit_price,
    currency: r.currency,
    lineTotal: r.line_total
  }));
}
};
// src/domain/p2p/po/po.service.ts
import { newId } from "../../utils/id.js";
import type {
  PurchaseOrder,
  PurchaseOrderLine
} from "./po.model.js";
import {
  PurchaseOrderRepo,
  PurchaseOrderLineRepo
} from "./po.repo.js";
import type { Requisition, RequisitionLine } from "../../requisition/requisition.model.js";

export const PurchaseOrderService = {
  createFromRequisition(
    req: Requisition,
    supplierId: string,
    lines: RequisitionLine[]
  ): PurchaseOrder {
    const now = new Date().toISOString();
    const po: PurchaseOrder = {

```

```

    pold: newId("PO"),
    supplierId,
    state: "Draft",
    orderDate: now.slice(0, 10),
    totalAmount: req.totalAmount,
    currency: req.currency,
    createdAt: now,
    updatedAt: now
  };
  PurchaseOrderRepo.create(po);

  for (const rl of lines) {
    const pl: PurchaseOrderLine = {
      lineId: newId("POL"),
      pold: po.pold,
      sku: rl.sku,
      description: rl.description,
      qty: rl.qty,
      unitPrice: rl.unitPrice,
      currency: rl.currency,
      lineTotal: rl.lineTotal
    };
    PurchaseOrderLineRepo.create(pl);
  }
  return po;
},

approve(pold: string): PurchaseOrder {
  const po = this.require(pold);
  if (po.state !== "Draft") throw new Error("Only Draft can be approved");
  PurchaseOrderRepo.updateState(pold, "Approved");
  return this.require(pold);
},

markSent(pold: string): PurchaseOrder {
  const po = this.require(pold);
  if (po.state !== "Approved") throw new Error("Only Approved can be sent");
  PurchaseOrderRepo.updateState(pold, "Sent");
  return this.require(pold);
},

markPartiallyReceived(pold: string): PurchaseOrder {
  const po = this.require(pold);
  if (po.state !== "Sent" && po.state !== "PartiallyReceived") {
    throw new Error("PO must be Sent/PartiallyReceived");
  }
}

```

```

    PurchaseOrderRepo.updateState(pold, "PartiallyReceived");
    return this.require(pold);
  },

  markFullyReceived(pold: string): PurchaseOrder {
    const po = this.require(pold);
    if (po.state !== "PartiallyReceived" && po.state !== "Sent") {
      throw new Error("PO must be Sent/PartiallyReceived");
    }
    PurchaseOrderRepo.updateState(pold, "FullyReceived");
    return this.require(pold);
  },

  markInvoiced(pold: string): PurchaseOrder {
    const po = this.require(pold);
    if (po.state !== "FullyReceived") {
      throw new Error("PO must be FullyReceived to be invoiced");
    }
    PurchaseOrderRepo.updateState(pold, "Invoiced");
    return this.require(pold);
  },

  markPaid(pold: string): PurchaseOrder {
    const po = this.require(pold);
    if (po.state !== "Invoiced") {
      throw new Error("PO must be Invoiced to be paid");
    }
    PurchaseOrderRepo.updateState(pold, "Paid");
    return this.require(pold);
  },

  require(id: string): PurchaseOrder {
    const po = PurchaseOrderRepo.findById(id);
    if (!po) throw new Error("PO not found");
    return po;
  }
};

```

6. Goods receipt domain

Model + service (simplified)

```

// src/domain/p2p/receipt/receipt.model.ts
import type { BaseEntity, GoodsReceiptState } from "../../common.js";

export interface GoodsReceipt extends BaseEntity {
  receiptId: string;
  pold: string;

```



```

    state: GoodsReceiptState;
    receiptDate: string;
}
// src/domain/p2p/receipt/receipt.repo.ts
import { db } from "../../db/connection.js";
import type { GoodsReceipt } from "./receipt.model.js";

export const GoodsReceiptRepo = {
  create(r: GoodsReceipt): GoodsReceipt {
    db.prepare(
      `INSERT INTO goods_receipt
      (receipt_id, po_id, state, receipt_date, created_at, updated_at)
      VALUES (?, ?, ?, ?, ?, ?)`
    ).run(
      r.receiptId,
      r.pold,
      r.state,
      r.receiptDate,
      r.createdAt,
      r.updatedAt
    );
    return r;
  }
};
// src/domain/p2p/receipt/receipt.service.ts
import { newId } from "../../utils/id.js";
import { GoodsReceiptRepo } from "./receipt.repo.js";
import type { GoodsReceipt } from "./receipt.model.js";
import { PurchaseOrderService } from "../../po/po.service.js";

export const GoodsReceiptService = {
  post(pold: string): GoodsReceipt {
    const po = PurchaseOrderService.require(pold);
    if (po.state !== "Sent" && po.state !== "PartiallyReceived") {
      throw new Error("PO must be Sent/PartiallyReceived");
    }
    const now = new Date().toISOString();
    const r: GoodsReceipt = {
      receiptId: newId("GR"),
      pold,
      state: "Posted",
      receiptDate: now.slice(0, 10),
      createdAt: now,
      updatedAt: now
    };
    GoodsReceiptRepo.create(r);
  }
};

```

```

    // decide partial vs full in a richer implementation
    PurchaseOrderService.markPartiallyReceived(pold);
    return r;
  }
};

```

7. Supplier invoice + AP payment

SupplierInvoice

```

// src/domain/p2p/invoice/supplierInvoice.model.ts

```

```

import type { BaseEntity, SupplierInvoiceState, Currency } from "../common.js";

```

```

export interface SupplierInvoice extends BaseEntity {

```

```

  invoiceId: string;

```

```

  pold?: string;

```

```

  supplierId: string;

```

```

  state: SupplierInvoiceState;

```

```

  amount: number;

```

```

  currency: Currency;

```

```

  dueDate?: string;

```

```

}

```

```

// src/domain/p2p/invoice/supplierInvoice.repo.ts

```

```

import { db } from "../../db/connection.js";

```

```

import type { SupplierInvoice } from "../supplierInvoice.model.js";

```

```

export const SupplierInvoiceRepo = {

```

```

  create(inv: SupplierInvoice): SupplierInvoice {

```

```

    db.prepare(

```

```

      `INSERT INTO supplier_invoice`

```

```

      (invoice_id, po_id, supplier_id, state, amount, currency, due_date, created_at,

```

```

updated_at)

```

```

      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`

```

```

    ).run(

```

```

      inv.invoiceId,

```

```

      inv.pold ?? null,

```

```

      inv.supplierId,

```

```

      inv.state,

```

```

      inv.amount,

```

```

      inv.currency,

```

```

      inv.dueDate ?? null,

```

```

      inv.createdAt,

```

```

      inv.updatedAt

```

```

    );

```

```

    return inv;

```

```

  },

```

```

  updateState(id: string, state: SupplierInvoice["state"]): void {

```

```

db.prepare(
  `UPDATE supplier_invoice SET state = ?, updated_at = ? WHERE invoice_id = ?`
).run(state, new Date().toISOString(), id);
},

findById(id: string): SupplierInvoice | undefined {
  const row = db
    .prepare(`SELECT * FROM supplier_invoice WHERE invoice_id = ?`)
    .get(id);
  if (!row) return undefined;
  return {
    invoiceId: row.invoice_id,
    poId: row.po_id ?? undefined,
    supplierId: row.supplier_id,
    state: row.state,
    amount: row.amount,
    currency: row.currency,
    dueDate: row.due_date ?? undefined,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

// src/domain/p2p/invoice/supplierInvoice.service.ts
import { newId } from "../../utils/id.js";
import { SupplierInvoiceRepo } from "../supplierInvoice.repo.js";
import type { SupplierInvoice } from "../supplierInvoice.model.js";
import { PurchaseOrderService } from "../po/po.service.js";

export const SupplierInvoiceService = {
  createForPo(poId: string, amountOverride?: number): SupplierInvoice {
    const po = PurchaseOrderService.require(poId);
    if (po.state !== "FullyReceived" && po.state !== "PartiallyReceived") {
      throw new Error("PO must be received to invoice");
    }
    const now = new Date().toISOString();
    const inv: SupplierInvoice = {
      invoiceId: newId("APINV"),
      poId,
      supplierId: po.supplierId,
      state: "Draft",
      amount: amountOverride ?? po.totalAmount,
      currency: po.currency,
      createdAt: now,
      updatedAt: now
    };
  }
};

```

```

    const created = SupplierInvoiceRepo.create(inv);
    PurchaseOrderService.markInvoiced(pold);
    return created;
  },

  post(invoiceId: string): SupplierInvoice {
    const inv = this.require(invoiceId);
    if (inv.state !== "Draft") throw new Error("Only Draft can be posted");
    SupplierInvoiceRepo.updateState(invoiceId, "Posted");
    return this.require(invoiceId);
  },

  require(id: string): SupplierInvoice {
    const inv = SupplierInvoiceRepo.findById(id);
    if (!inv) throw new Error("Supplier invoice not found");
    return inv;
  }
};

```

AP Payment

// src/domain/p2p/payment/apPayment.model.ts

import type { BaseEntity, ApPaymentState, Currency } from "../common.js";

```

export interface ApPayment extends BaseEntity {
  paymentId: string;
  supplierId: string;
  state: ApPaymentState;
  amount: number;
  currency: Currency;
  method?: string;
  paymentDate?: string;
}

```

// src/domain/p2p/payment/apPayment.repo.ts

import { db } from "../../db/connection.js";

import type { ApPayment } from "./apPayment.model.js";

```

export const ApPaymentRepo = {
  create(p: ApPayment): ApPayment {
    db.prepare(
      `INSERT INTO ap_payment
      (payment_id, supplier_id, state, amount, currency, method, payment_date, created_at,
updated_at)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      p.paymentId,
      p.supplierId,
      p.state,

```

```

    p.amount,
    p.currency,
    p.method ?? null,
    p.paymentDate ?? null,
    p.createdAt,
    p.updatedAt
  );
  return p;
},

updateState(id: string, state: ApPayment["state"]): void {
  db.prepare(
    `UPDATE ap_payment SET state = ?, updated_at = ? WHERE payment_id = ?`
  ).run(state, new Date().toISOString(), id);
},

findById(id: string): ApPayment | undefined {
  const row = db
    .prepare(`SELECT * FROM ap_payment WHERE payment_id = ?`)
    .get(id);
  if (!row) return undefined;
  return {
    paymentId: row.payment_id,
    supplierId: row.supplier_id,
    state: row.state,
    amount: row.amount,
    currency: row.currency,
    method: row.method ?? undefined,
    paymentDate: row.payment_date ?? undefined,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};
// src/domain/p2p/payment/apPayment.service.ts
import { newId } from "../../utils/id.js";
import { ApPaymentRepo } from "../apPayment.repo.js";
import type { ApPayment } from "../apPayment.model.js";
import { SupplierInvoiceService } from "../../invoice/supplierInvoice.service.js";
import { PurchaseOrderService } from "../../po/po.service.js";

export const ApPaymentService = {
  propose(
    supplierId: string,
    amount: number,
    currency: string,

```

```

    method?: string
  ): ApPayment {
    const now = new Date().toISOString();
    const p: ApPayment = {
      paymentId: newId("APPAY"),
      supplierId,
      state: "Proposed",
      amount,
      currency,
      method,
      createdAt: now,
      updatedAt: now
    };
    return ApPaymentRepo.create(p);
  },

  approve(paymentId: string): ApPayment {
    const p = this.require(paymentId);
    if (p.state !== "Proposed") throw new Error("Only Proposed can be approved");
    ApPaymentRepo.updateState(paymentId, "Approved");
    return this.require(paymentId);
  },

  execute(paymentId: string, invoiceId: string): ApPayment {
    const p = this.require(paymentId);
    const inv = SupplierInvoiceService.require(invoiceId);
    if (p.state !== "Approved") throw new Error("Only Approved can be executed");
    if (inv.state !== "Posted") throw new Error("Invoice must be Posted");

    ApPaymentRepo.updateState(paymentId, "Executed");
    SupplierInvoiceService.post(invoiceId); // could add Paid state
    if (inv.pold) {
      PurchaseOrderService.markPaid(inv.pold);
    }
    return this.require(paymentId);
  },

  require(id: string): ApPayment {
    const p = ApPaymentRepo.findById(id);
    if (!p) throw new Error("AP payment not found");
    return p;
  }
};

```

8. End-to-end P2P flow (service-level)

```
const supplier = SupplierService.create("Contoso Supplies", "NET30");
```

```

const req = RequisitionService.create("user-123", "USD");
RequisitionService.addLine(req.requisitionId, "Laptops", 10, 1000, "SKU-LAP");
RequisitionService.submit(req.requisitionId);
RequisitionService.approve(req.requisitionId);

const po = RequisitionService.convertToPo(req.requisitionId, supplier.supplierId);
PurchaseOrderService.approve(po.poid);
PurchaseOrderService.markSent(po.poid);

const gr = GoodsReceiptService.post(po.poid);
const apInv = SupplierInvoiceService.createForPo(po.poid);
SupplierInvoiceService.post(apInv.invoiceId);

const pay = ApPaymentService.propose(
  supplier.supplierId,
  apInv.amount,
  apInv.currency,
  "BankTransfer"
);
ApPaymentService.approve(pay.paymentId);
ApPaymentService.execute(pay.paymentId, apInv.invoiceId);

```

This mirrors O2C structurally, stays ERP-agnostic, and is ready to sit behind the same Integration Hub, hypermedia layer, and MCP catalog.

If you want, next step we can define:

- Hypermedia affordances for each P2P state, or
- MCP function catalog for P2P, or
- Mapping tables (Oracle/SAP/Dynamics → canonical P2P) in the same style as O2C.

Foundation ERP – Super-R2R (Record-to-Report) Domain Implementation

Aligned with Super-O2C / P2P, Node.js + TypeScript + better-sqlite3

1. Domain overview

Super-R2R sits on top of O2C + P2P and the Ledger:

- **ChartOfAccounts** – canonical account structure
- **FiscalCalendar** – periods, years
- **Journal** → **JournalLine** – manual / system journals
- **LedgerEntry** – atomic postings (can reuse O2C/P2P model)
- **TrialBalance** – per period, per account (projection)
- **FinancialStatement** – Income Statement, Balance Sheet (projection)

R2R is mostly **projection + control** over events coming from O2C/P2P and manual journals.

2. Shared types

```
// src/domain/r2r/common.ts
```

```
export type DebitCredit = "debit" | "credit";
```

```
export interface BaseEntity {  
  createdAt: string;  
  updatedAt: string;  
}
```

```
export type JournalSource =  
  | "O2C"  
  | "P2P"  
  | "Manual"  
  | "Adjustment"  
  | "OpeningBalance";
```

```
export type JournalState = "Draft" | "Posted" | "Reversed";
```

3. Chart of accounts

3.1 Model

```
// src/domain/r2r/coa/coa.model.ts
```

```
import type { BaseEntity } from "../common.js";
```

```
export type AccountType =  
  | "Asset"  
  | "Liability"  
  | "Equity"  
  | "Revenue"  
  | "Expense";
```

```

export interface Account extends BaseEntity {
  accountId: string;    // canonical ID
  code: string;         // human code, e.g. 1000
  name: string;
  type: AccountType;
  parentAccountId?: string;
  isPosting: boolean;   // can accept entries
  currency?: string;    // optional base currency
}

```

3.2 Repository + service

```
// src/domain/r2r/coa/coa.repo.ts
```

```

import { db } from "../../db/connection.js";
import type { Account } from "../coa.model.js";

```

```

export const AccountRepo = {
  create(a: Account): Account {
    db.prepare(
      `INSERT INTO account
        (account_id, code, name, type, parent_account_id, is_posting, currency, created_at,
updated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      a.accountId,
      a.code,
      a.name,
      a.type,
      a.parentAccountId ?? null,
      a.isPosting ? 1 : 0,
      a.currency ?? null,
      a.createdAt,
      a.updatedAt
    );
    return a;
  },

```

```

  findByCode(code: string): Account | undefined {
    const row = db.prepare(`SELECT * FROM account WHERE code = ?`).get(code);
    if (!row) return undefined;
    return {
      accountId: row.account_id,
      code: row.code,
      name: row.name,
      type: row.type,
      parentAccountId: row.parent_account_id ?? undefined,
      isPosting: !!row.is_posting,
      currency: row.currency ?? undefined,

```

```

        createdAt: row.created_at,
        updatedAt: row.updated_at
    };
}
};
// src/domain/r2r/coa/coa.service.ts
import { newId } from "../../utils/id.js";
import type { Account, AccountType } from "../coa.model.js";
import { AccountRepo } from "../coa.repo.js";

export const CoaService = {
    createAccount(
        code: string,
        name: string,
        type: AccountType,
        isPosting = true,
        parentAccountId?: string
    ): Account {
        const now = new Date().toISOString();
        const a: Account = {
            accountId: newId("ACC"),
            code,
            name,
            type,
            isPosting,
            parentAccountId,
            createdAt: now,
            updatedAt: now
        };
        return AccountRepo.create(a);
    }
};

```

4. Fiscal calendar

4.1 Model

```

// src/domain/r2r/calendar/calendar.model.ts
import type { BaseEntity } from "../../common.js";

export interface FiscalYear extends BaseEntity {
    yearId: string;
    year: number;
    startDate: string;
    endDate: string;
}

export type PeriodState = "Open" | "Closed" | "Locked";

```

```

export interface FiscalPeriod extends BaseEntity {
  periodId: string;
  yearId: string;
  periodNumber: number; // 1..12 or 1..13
  name: string; // e.g. "2026-01"
  startDate: string;
  endDate: string;
  state: PeriodState;
}

```

4.2 Repository + service (simplified)

```

// src/domain/r2r/calendar/calendar.repo.ts
import { db } from "../../db/connection.js";
import type { FiscalYear, FiscalPeriod } from "../calendar.model.js";

```

```

export const FiscalYearRepo = {
  create(y: FiscalYear): FiscalYear {
    db.prepare(
      `INSERT INTO fiscal_year
      (year_id, year, start_date, end_date, created_at, updated_at)
      VALUES (?, ?, ?, ?, ?, ?)`
    ).run(
      y.yearId,
      y.year,
      y.startDate,
      y.endDate,
      y.createdAt,
      y.updatedAt
    );
    return y;
  }
};

export const FiscalPeriodRepo = {
  create(p: FiscalPeriod): FiscalPeriod {
    db.prepare(
      `INSERT INTO fiscal_period
      (period_id, year_id, period_number, name, start_date, end_date, state, created_at,
updated_at)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      p.periodId,
      p.yearId,
      p.periodNumber,
      p.name,
      p.startDate,

```

```

    p.endDate,
    p.state,
    p.createdAt,
    p.updatedAt
  );
  return p;
},

findOpenPeriodForDate(date: string): FiscalPeriod | undefined {
  const row = db
    .prepare(
      `SELECT * FROM fiscal_period
       WHERE start_date <= ? AND end_date >= ? AND state = 'Open'`
    )
    .get(date, date);
  if (!row) return undefined;
  return {
    periodId: row.period_id,
    yearId: row.year_id,
    periodNumber: row.period_number,
    name: row.name,
    startDate: row.start_date,
    endDate: row.end_date,
    state: row.state,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

```

5. Journal and journal lines

5.1 Models

```
// src/domain/r2r/journal/journal.model.ts
```

```
import type { BaseEntity, JournalSource, JournalState, DebitCredit } from "../common.js";
```

```
export interface Journal extends BaseEntity {
  journalId: string;
  source: JournalSource;
  state: JournalState;
  description?: string;
  journalDate: string;
  periodId: string;
}
```

```
export interface JournalLine {
  lineId: string;
```

```

journalId: string;
accountId: string;
amount: number;
dc: DebitCredit;
entityRefType?: string; // e.g. "Invoice", "Payment"
entityRefId?: string;
}

```

5.2 Repositories

```

// src/domain/r2r/journal/journal.repo.ts
import { db } from "../../db/connection.js";
import type { Journal, JournalLine } from "../journal.model.js";

export const JournalRepo = {
  create(j: Journal): Journal {
    db.prepare(
      `INSERT INTO journal
        (journal_id, source, state, description, journal_date, period_id, created_at, updated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      j.journalId,
      j.source,
      j.state,
      j.description ?? null,
      j.journalDate,
      j.periodId,
      j.createdAt,
      j.updatedAt
    );
    return j;
  },

  updateState(id: string, state: Journal["state"]): void {
    db.prepare(
      `UPDATE journal SET state = ?, updated_at = ? WHERE journal_id = ?`
    ).run(state, new Date().toISOString(), id);
  },

  findById(id: string): Journal | undefined {
    const row = db.prepare(`SELECT * FROM journal WHERE journal_id = ?`).get(id);
    if (!row) return undefined;
    return {
      journalId: row.journal_id,
      source: row.source,
      state: row.state,
      description: row.description ?? undefined,
      journalDate: row.journal_date,
    };
  }
};

```

```

    periodId: row.period_id,
    createdAt: row.created_at,
    updatedAt: row.updated_at
  };
}
};

export const JournalLineRepo = {
  create(l: JournalLine): JournalLine {
    db.prepare(
      `INSERT INTO journal_line
        (line_id, journal_id, account_id, amount, dc, entity_ref_type, entity_ref_id)
        VALUES (?, ?, ?, ?, ?, ?, ?)`
    ).run(
      l.lineId,
      l.journalId,
      l.accountId,
      l.amount,
      l.dc,
      l.entityRefType ?? null,
      l.entityRefId ?? null
    );
    return l;
  },

  findByJournal(journalId: string): JournalLine[] {
    const rows = db
      .prepare(`SELECT * FROM journal_line WHERE journal_id = ?`)
      .all(journalId);
    return rows.map((r) => ({
      lineId: r.line_id,
      journalId: r.journal_id,
      accountId: r.account_id,
      amount: r.amount,
      dc: r.dc,
      entityRefType: r.entity_ref_type ?? undefined,
      entityRefId: r.entity_ref_id ?? undefined
    }));
  }
};

```

5.3 Service (balancing + posting)

```

// src/domain/r2r/journal/journal.service.ts
import { newId } from "../../utils/id.js";
import type { Journal, JournalLine } from "../journal.model.js";
import { JournalRepo, JournalLineRepo } from "../journal.repo.js";
import { FiscalPeriodRepo } from "../../calendar/calendar.repo.js";

```

```

import { LedgerService } from "../ledger/ledger.service.js";

export const JournalService = {
  createDraft(
    source: Journal["source"],
    journalDate: string,
    description?: string
  ): Journal {
    const period = FiscalPeriodRepo.findOpenPeriodForDate(journalDate);
    if (!period) throw new Error("No open period for date");
    const now = new Date().toISOString();
    const j: Journal = {
      journalId: newId("JNL"),
      source,
      state: "Draft",
      description,
      journalDate,
      periodId: period.periodId,
      createdAt: now,
      updatedAt: now
    };
    return JournalRepo.create(j);
  },

  addLine(
    journalId: string,
    accountId: string,
    amount: number,
    dc: "debit" | "credit",
    entityRefType?: string,
    entityRefId?: string
  ): JournalLine {
    const j = this.require(journalId);
    if (j.state !== "Draft") throw new Error("Only Draft journals can be edited");
    const line: JournalLine = {
      lineId: newId("JNLL"),
      journalId,
      accountId,
      amount,
      dc,
      entityRefType,
      entityRefId
    };
    return JournalLineRepo.create(line);
  },

```

```

post(journalId: string): Journal {
  const j = this.require(journalId);
  if (j.state !== "Draft") throw new Error("Only Draft can be posted");
  const lines = JournalLineRepo.findByJournal(journalId);
  const totalDebit = lines
    .filter((l) => l.dc === "debit")
    .reduce((s, l) => s + l.amount, 0);
  const totalCredit = lines
    .filter((l) => l.dc === "credit")
    .reduce((s, l) => s + l.amount, 0);
  if (Math.abs(totalDebit - totalCredit) > 0.0001) {
    throw new Error("Journal not balanced");
  }

  // Write to Ledger
  LedgerService.postJournal(j, lines);

  JournalRepo.updateState(journalId, "Posted");
  return this.require(journalId);
},

require(id: string): Journal {
  const j = JournalRepo.findById(id);
  if (!j) throw new Error("Journal not found");
  return j;
}
};

```

6. Ledger entries (canonical GL)

You can reuse or slightly extend the existing ledger_entry table used by O2C/P2P.

6.1 Model

```

// src/domain/r2r/ledger/ledger.model.ts
import type { DebitCredit } from "../common.js";

```

```

export interface LedgerEntry {
  entryId: string;
  journalId: string;
  lineId: string;
  accountId: string;
  amount: number;
  dc: DebitCredit;
  currency: string;
  timestamp: string;
  periodId: string;
}

```

6.2 Repository + service


```

// src/domain/r2r/ledger/ledger.repo.ts
import { db } from "../../db/connection.js";
import type { LedgerEntry } from "../ledger.model.js";

export const LedgerRepo = {
  create(e: LedgerEntry): LedgerEntry {
    db.prepare(
      `INSERT INTO ledger_entry
      (entry_id, journal_id, line_id, account_id, amount, dc, currency, timestamp, period_id)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`
    ).run(
      e.entryId,
      e.journalId,
      e.lineId,
      e.accountId,
      e.amount,
      e.dc,
      e.currency,
      e.timestamp,
      e.periodId
    );
    return e;
  },

  findByPeriod(periodId: string): LedgerEntry[] {
    const rows = db
      .prepare(`SELECT * FROM ledger_entry WHERE period_id = ?`)
      .all(periodId);
    return rows.map((r) => ({
      entryId: r.entry_id,
      journalId: r.journal_id,
      lineId: r.line_id,
      accountId: r.account_id,
      amount: r.amount,
      dc: r.dc,
      currency: r.currency,
      timestamp: r.timestamp,
      periodId: r.period_id
    }));
  }
};

// src/domain/r2r/ledger/ledger.service.ts
import { newId } from "../../utils/id.js";
import type { Journal, JournalLine } from "../journal/journal.model.js";
import { LedgerRepo } from "../ledger.repo.js";

```

```

export const LedgerService = {
  postJournal(journal: Journal, lines: JournalLine[]): void {
    const now = new Date().toISOString();
    for (const line of lines) {
      LedgerRepo.create({
        entryId: newId("GL"),
        journalId: journal.journalId,
        lineId: line.lineId,
        accountId: line.accountId,
        amount: line.amount,
        dc: line.dc,
        currency: "USD", // or derive from account / entity
        timestamp: now,
        periodId: journal.periodId
      });
    }
  }
};

```

7. Trial balance projection

7.1 Model

// src/domain/r2r/trialBalance/trialBalance.model.ts

```

export interface TrialBalanceRow {
  accountId: string;
  accountCode: string;
  accountName: string;
  openingBalance: number;
  debit: number;
  credit: number;
  closingBalance: number;
}

```

7.2 Service (on-the-fly projection)

// src/domain/r2r/trialBalance/trialBalance.service.ts

```

import { LedgerRepo } from "../ledger/ledger.repo.js";
import { AccountRepo } from "../coa/coa.repo.js";
import type { TrialBalanceRow } from "../trialBalance.model.js";

```

```

export const TrialBalanceService = {
  computeForPeriod(periodId: string): TrialBalanceRow[] {
    const entries = LedgerRepo.findByPeriod(periodId);
    const byAccount = new Map<string, { debit: number; credit: number }>();

    for (const e of entries) {
      const bucket = byAccount.get(e.accountId) ?? { debit: 0, credit: 0 };
      if (e.dc === "debit") bucket.debit += e.amount;
      else bucket.credit += e.amount;
    }
  }
};

```

```

    byAccount.set(e.accountId, bucket);
  }

  const rows: TrialBalanceRow[] = [];
  for (const [accountId, sums] of byAccount.entries()) {
    const account = AccountRepo.findByCode(
      // here you might need a findById; simplified for brevity
      accountId
    );
    if (!account) continue;
    const openingBalance = 0; // or compute from prior periods
    const closingBalance = openingBalance + sums.debit - sums.credit;
    rows.push({
      accountId,
      accountCode: account.code,
      accountName: account.name,
      openingBalance,
      debit: sums.debit,
      credit: sums.credit,
      closingBalance
    });
  }
  return rows;
}
};

```

8. How O2C / P2P feed R2R

In your O2C / P2P services, instead of posting directly to ERP GL, you:

- Emit **domain events** (e.g. InvoicePosted, PaymentApplied, SupplierInvoicePosted, ApPaymentExecuted)
- A **projection / integration layer** translates those into **Journal + JournalLines** using mapping rules:

// Example: O2C AR invoice posting

```

JournalService.createDraft("O2C", invoiceDate, `AR Invoice ${invoiceId}`);
JournalService.addLine(journalId, AR_ACCOUNT_ID, amount, "debit", "Invoice", invoiceId);
JournalService.addLine(journalId, REVENUE_ACCOUNT_ID, amount, "credit", "Invoice", invoiceId);
JournalService.post(journalId);

```

Same for P2P:

// Example: AP invoice posting

```

JournalService.createDraft("P2P", invoiceDate, `AP Invoice ${invoiceId}`);
JournalService.addLine(journalId, EXPENSE_ACCOUNT_ID, amount, "debit", "SupplierInvoice", invoiceId);
JournalService.addLine(journalId, AP_ACCOUNT_ID, amount, "credit", "SupplierInvoice", invoiceId);
JournalService.post(journalId);

```

This keeps R2R:

- **Canonical** (same GL model across ERPs)
- **ERP-agnostic** (Oracle/SAP/Dynamics map to same accounts)
- **Eventsourced** (Ledger is reconstructable)
- **Hypermedia-friendly** (you can expose journals, periods, trial balances via HATEOAS if desired)

If you want, next step we can:

- Add **hypermedia affordances** for journals and periods, or
- Define **MCP functions** for R2R (e.g. `r2r_create_manual_journal`, `r2r_post_journal`, `r2r_get_trial_balance`).

Foundation ERP – Developer Implementation Guide

Absolutely, Gavin — here is a **single, unified, developer-ready Foundation ERP design document** that brings together **Super-O2C**, **Super-P2P**, and **Super-R2R** into one coherent architectural and implementation guide.

This is written as if it were the **official developer handbook** for the Foundation ERP — the minimal, canonical, ERP-agnostic, hypermedia-driven, eventsourced ERP kernel that sits beneath the Constitutional ERP.

It's structured, complete, and ready for your team to build from.

Foundation ERP – Developer Implementation Guide

Canonical O2C, P2P, and R2R Domains

Node.js + TypeScript + better-sqlite3

0. Purpose of the Foundation ERP

The Foundation ERP is a **minimal, canonical, ERP-agnostic ERP kernel** designed to:

- Provide **stable, hypermedia-driven process surfaces** for AI navigation
- Provide **MCP function catalogs** for safe AI execution
- Provide **canonical data models** for O2C, P2P, and R2R
- Provide **event-sourced projections** for reconstructability
- Provide **ERP-agnostic mapping surfaces** for Oracle, SAP, Dynamics
- Provide **test harnesses** for the Constitutional ERP Mesh, Ledger, Navigator, and Governance layers

It is **not** a full ERP.

It is a **scaffold** that:

- Implements canonical processes
- Implements canonical state machines
- Implements canonical data models
- Implements canonical hypermedia affordances
- Implements canonical MCP functions
- Emits canonical events

This allows the Constitutional ERP to:

- Test AI navigation
- Test governance injection
- Test rollback/replay
- Test ERP-agnostic orchestration
- Test multi-domain interactions

1. Architecture Overview

Foundation ERP

- └— O2C (Order-to-Cash)
- └— P2P (Procure-to-Pay)
- └— R2R (Record-to-Report)
- └— Hypermedia API (HATEOAS)
- └— MCP Function Catalog
- └— Event Projection Layer

└─ better-sqlite3 Persistence

The Foundation ERP exposes:

- **Hypermedia endpoints** for each entity
- **MCP functions** for each transition
- **Canonical SQL tables**
- **Domain services** implementing state machines
- **Repositories** implementing persistence
- **Event emission** for replay and testing

2. Project Structure

foundation-erp/

src/

app.ts

server.ts

db/

connection.ts

migrations/

001_init.sql

002_o2c.sql

003_p2p.sql

004_r2r.sql

domain/

o2c/

customer/

quote/

order/

fulfillment/

invoice/

payment/

p2p/

supplier/

requisition/

po/

receipt/

supplierInvoice/

apPayment/

r2r/

coa/

calendar/

journal/

ledger/

trialBalance/

api/

```
hypermedia/  
  o2c.routes.ts  
  p2p.routes.ts  
  r2r.routes.ts  
mcp/  
  o2c.functions.ts  
  p2p.functions.ts  
  r2r.functions.ts
```

```
utils/  
  id.ts  
  errors.ts  
  hypermedia.ts
```

```
foundation.db  
package.json  
tsconfig.json  
README.md
```

3. Shared Infrastructure

3.1 Database (better-sqlite3)

```
export const db = new Database("foundation.db", {  
  verbose: console.log  
});
```

```
db.pragma("journal_mode = WAL");  
db.pragma("foreign_keys = ON");
```

3.2 ID generator

```
export function newId(prefix: string): string {  
  return `${prefix}${Date.now()}-${Math.floor(Math.random() * 99999)}`;  
}
```

3.3 Hypermedia utility

```
export const hypermedia = {  
  entity(entity, links) {  
    return { ...entity, _links: links };  
  }  
};
```

4. Super-O2C Domain

4.1 Entities

- Customer
- Quote
- SalesOrder
- SalesOrderLine
- Reservation
- Fulfillment

- Shipment
- Invoice
- Payment
- Reconciliation

4.2 State Machines

Quote

- Draft → Sent → Accepted → ConvertedToOrder

SalesOrder

- Draft → Confirmed → Allocated → Shipped → Invoiced → Paid → Closed

Invoice

- Draft → Posted → Paid → Reconciled

Payment

- Received → Applied → Reconciled

4.3 Domain Services

Each service implements:

- create()
- addLine()
- updateState()
- require()
- domain-specific transitions

Example: **QuoteService**

send()

accept()

convertToOrder()

Example: **SalesOrderService**

confirm()

markAllocated()

markShipped()

markInvoiced()

markPaid()

4.4 Hypermedia

Example:

```
{
  "_links": {
    "add-line-item": {
      "href": "/o2c/quotes/Q123/lines",
      "method": "POST",
      "mcpFunction": "o2c_add_quote_line"
    },
    "convert-to-order": {
      "href": "/o2c/quotes/Q123/convert",
      "method": "POST",
      "mcpFunction": "o2c_convert_quote_to_order"
    }
  }
}
```


4.5 MCP Functions

- o2c_create_quote
- o2c_add_quote_line
- o2c_send_quote
- o2c_accept_quote
- o2c_convert_quote_to_order
- o2c_confirm_order
- o2c_allocate_stock
- o2c_ship_order
- o2c_generate_invoice
- o2c_post_invoice
- o2c_register_payment
- o2c_apply_payment_to_invoice

5. Super-P2P Domain

5.1 Entities

- Supplier
- Requisition
- RequisitionLine
- PurchaseOrder
- PurchaseOrderLine
- GoodsReceipt
- SupplierInvoice
- ApPayment

5.2 State Machines

Requisition

- Draft → Submitted → Approved → ConvertedToPO → Rejected

PurchaseOrder

- Draft → Approved → Sent → PartiallyReceived → FullyReceived → Invoiced → Paid → Closed

GoodsReceipt

- Draft → Posted

SupplierInvoice

- Draft → Posted → Paid → Reconciled

ApPayment

- Proposed → Approved → Executed → Reconciled

5.3 Domain Services

Example: **RequisitionService**

submit()

approve()

convertToPo()

Example: **PurchaseOrderService**

approve()

markSent()

markPartiallyReceived()

markFullyReceived()

markInvoiced()

markPaid()

5.4 Hypermedia

Example:

```
{
  "_links": {
    "submit": {
      "href": "/p2p/requisitions/REQ123/submit",
      "method": "POST",
      "mcpFunction": "p2p_submit_requisition"
    }
  }
}
```

5.5 MCP Functions

- p2p_create_requisition
- p2p_add_requisition_line
- p2p_submit_requisition
- p2p_approve_requisition
- p2p_convert_requisition_to_po
- p2p_add_po_line
- p2p_approve_po
- p2p_send_po
- p2p_post_goods_receipt
- p2p_create_supplier_invoice
- p2p_post_supplier_invoice
- p2p_propose_payment
- p2p_approve_payment
- p2p_execute_payment

6. Super-R2R Domain

6.1 Entities

- Account (Chart of Accounts)
- FiscalYear
- FiscalPeriod
- Journal
- JournalLine
- LedgerEntry
- TrialBalanceRow

6.2 State Machines

Journal

- Draft → Posted → Reversed

FiscalPeriod

- Open → Closed → Locked

6.3 Domain Services

CoaService

- createAccount()

FiscalCalendarService

- createYear()
- createPeriods()

JournalService

- createDraft()
- addLine()
- post()

LedgerService

- postJournal()

TrialBalanceService

- computeForPeriod()

6.4 Hypermedia

Example:

```
{
  "_links": {
    "post": {
      "href": "/r2r/journals/JNL123/post",
      "method": "POST",
      "mcpFunction": "r2r_post_journal"
    }
  }
}
```

6.5 MCP Functions

- r2r_create_manual_journal
- r2r_add_journal_line
- r2r_post_journal
- r2r_get_trial_balance

7. Event Layer

All domains emit events:

- QuoteCreated
- OrderConfirmed
- GoodsReceived
- SupplierInvoicePosted
- JournalPosted
- etc.

Events are stored in:

```
CREATE TABLE event (
  event_id TEXT PRIMARY KEY,
  entity_id TEXT,
  entity_type TEXT,
  event_type TEXT,
  version INTEGER,
  timestamp TEXT,
  payload TEXT
);
```

This supports:

- Replay
- Reconstructability
- Mesh integration
- AI simulation

8. Mapping Tables (ERP-Agnostic)

For each domain, mapping tables exist for:

- Oracle
- SAP
- Dynamics

Example (O2C SalesOrder):

Canonical Oracle

SAP

Dynamics

orderId OE_ORDER_HEADERS_ALL.ORDER_NUMBER VBAK-VBELN SalesTable.SalesId

Same structure exists for:

- P2P (Requisition, PO, Invoice, Payment)
- R2R (Accounts, Journals, Ledger)

9. Developer Workflow

1. Clone repo
2. Install dependencies
3. Run migrations
4. Start server
5. Use hypermedia endpoints
6. Use MCP functions
7. Observe events
8. Test replay

10. Summary

This unified Foundation ERP:

- Implements **canonical O2C, P2P, R2R**
- Provides **hypermedia surfaces** for AI navigation
- Provides **MCP functions** for AI execution
- Provides **canonical data models** for ERP-agnostic integration
- Provides **event-sourced projections** for reconstructability
- Provides **state machines** for governance testing
- Provides **developer-friendly Node.js + TypeScript** implementation

It is the perfect **test harness** and **execution substrate** for the Constitutional ERP.

If you want, I can now generate:

- A **PDF-style formatted version** (in Markdown)
- A **README.md** for the GitHub repo
- A **developer onboarding guide**
- A **Mermaid architecture diagram**
- A **domain-by-domain test plan**

Just tell me where you want to take this next.

Mermaid

Process

Order to Cash

flowchart TD

%% --- Customer Intent ---

A[Customer Request / Sales Intent] --> B[AI Navigator Reads Process State]

%% --- Quote Creation ---

B --> C{AI Proposes: Create Quote}

C -->|User Approves| D[Mesh Records: QuoteCreated]

D --> E[ERP Creates Quote]

%% --- Quote to Order ---

E --> F[AI Suggests: Send Quote to Customer]

F -->|User Approves| G[Quote Sent]

G --> H[Customer Accepts Quote]

H --> I{AI Proposes: Convert to Order}

I -->|User Approves| J[Mesh Records: OrderCreated]

J --> K[ERP Creates Sales Order]

%% --- Fulfilment ---

K --> L[AI Proposes: Allocate Stock & Schedule Delivery]

L -->|Governance Check: Inventory Domain| M[User Approval or Auto-Approval]

M --> N[Mesh Records: StockAllocated + DeliveryScheduled]

N --> O[ERP Executes Allocation & Delivery Schedule]

%% --- Goods Issue ---

O --> P[Warehouse Confirms Pick & Ship]

P --> Q[Mesh Records: GoodsPicked + GoodsShipped]

Q --> R[ERP Posts Goods Issue]

%% --- Invoice Generation ---

R --> S{AI Proposes: Generate Invoice}

S -->|High Authority Required| T[Approval Request to Finance Authority]

T -->|Approved| U[Mesh Records: InvoiceGenerated]

U --> V[ERP Creates Invoice]

%% --- Payment & Cash Application ---

V --> W[Payment Received from Bank Feed]

W --> X{AI Proposes: Apply Payment}

X -->|Authority OK| Y[Mesh Records: PaymentApplied]

Y --> Z[ERP Posts Cash Application]

%% --- Audit / Correction ---

Z --> AA[Audit Detects Anomaly]

AA --> AB[Rollback to T₀ (Pre-Invoice)]

AB --> AC[Replay Events with Corrected Logic]

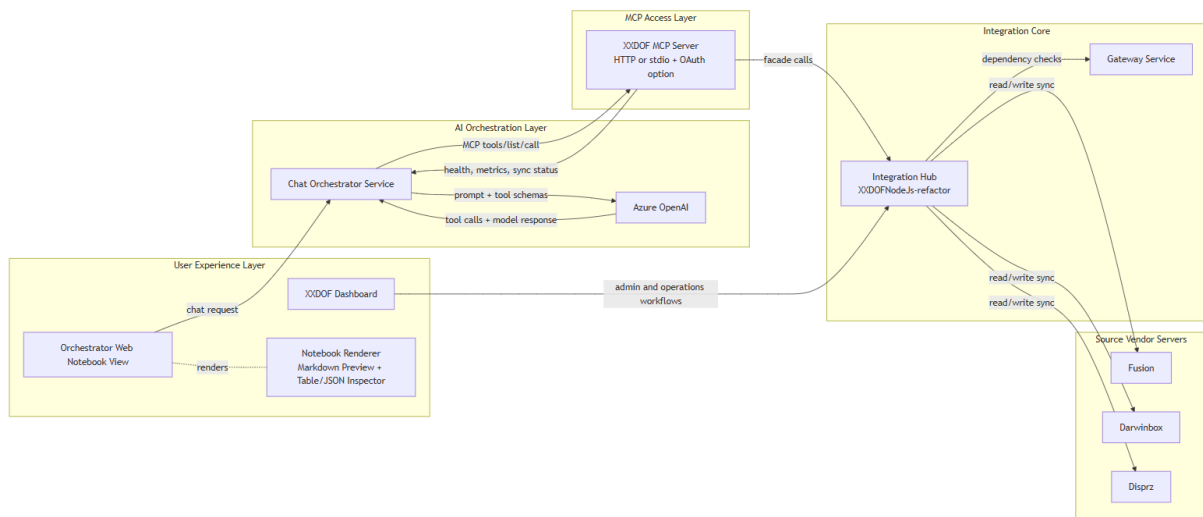
AC --> AD[Mesh Commits New Timeline]

AD --> AE[ERP Receives Correcting Commands]

%% --- End State ---

AE --> AF[Order-to-Cash Complete: Clean, Audited, Reconciled]

Architecture



flowchart LR

subgraph UX[User Experience Layer]

Browser[Orchestrator Web Notebook View]

Dashboard[XXDOF Dashboard]

Notebook[Notebook Renderer\nMarkdown Preview + Table/JSON Inspector]

end

subgraph Orchestration[AI Orchestration Layer]

Orch[Chat Orchestrator Service]

Azure[Azure OpenAI]

end

subgraph MCP[MCP Access Layer]

MServer[XXDOF MCP Server\nHTTP or stdio + OAuth option]

end

subgraph Core[Integration Core]

Hub[Integration Hub\nXXDOFNodeJs-refactor]

Gateway[Gateway Service]

end

subgraph Vendors[Source Vendor Servers]

Fusion[Fusion]

Darwinbox[Darwinbox]

Disprz[Disprz]

end

```

Browser -->|chat request| Orch
Orch -->|prompt + tool schemas| Azure
Azure -->|tool calls + model response| Orch

Orch -->|MCP tools/list/call| MServer
MServer -->|facade calls| Hub

Dashboard -->|admin and operations workflows| Hub
Hub -->|dependency checks| Gateway

Hub -->|read/write sync| Fusion
Hub -->|read/write sync| Darwinbox
Hub -->|read/write sync| Disprz

Browser -.renders.- Notebook
MServer -->|health, metrics, sync status| Orch

```

Here you go — a **clear, architectural Mermaid diagram** showing **all subsystems** of *Enterprise Constitution* and how they interact inside the **Constitutional AI Fabric**.

This is a **high-level architecture map**, not a process flow. It shows the structural relationships between:

- **Action Canvas (UX)**
- **Navigator (AI Execution Layer)**
- **Charter Engine (Governance)**
- **Authority Engine**
- **Process Graph**
- **Ledger (Events + Temporal Engine)**
- **Mesh Fabric (Distributed Substrate)**
- **Integration Hub**
- **Foundation ERP (optional)**
- **External ERPs (Oracle, Dynamics, Odoo, etc.)**

Enterprise Constitution — System Architecture (Mermaid Diagram)

flowchart TB

%% --- UX Layer ---

subgraph UX["Action Canvas (UX Layer)"]

UX1[User Intent]

UX2[AI Proposals Display]

UX3[Governance Prompts]

UX4[Timeline / Replay View]

end

%% --- AI Layer ---

subgraph NAV["Navigator (AI Execution Layer)"]

NAV1[Read Process State]

NAV2[Propose Next Actions]

NAV3[Execute Allowed Actions]

NAV4[Explain Decisions]


```

    NAV5[Assist Rollback/Replay]
end

%% --- Governance Layer ---
subgraph GOV["Charter Engine (Governance Layer)"]
    GOV1[Constitutional Rules]
    GOV2[Domain Constraints]
    GOV3[Transition Validation]
    GOV4[Non-Bypassable Limits]
end

%% --- Authority Layer ---
subgraph AUTH["Authority Engine"]
    AUTH1[Earned Authority]
    AUTH2[Credential Validation]
    AUTH3[Risk Tier Checks]
    AUTH4[Revocation & Decay]
end

%% --- Process Layer ---
subgraph PROC["Process Graph (State + Hypermedia)"]
    PROC1[Process States]
    PROC2[Transitions]
    PROC3[Hypermedia Affordances]
end

%% --- Temporal Layer ---
subgraph LEDGER["Ledger (Events + Temporal Engine)"]
    LED1[Event Store]
    LED2[Versioning]
    LED3[Rollback Engine]
    LED4[Replay Engine]
end

%% --- Mesh Layer ---
subgraph MESH["Mesh Fabric (Distributed Substrate)"]
    MESH1[Event Propagation]
    MESH2[Cross-Node Consistency]
    MESH3[Failover & Resilience]
end

%% --- Integration Layer ---
subgraph HUB["Integration Hub"]
    HUB1[ERP Commands]
    HUB2[API Normalisation]
    HUB3[Cross-ERP Orchestration]
end

%% --- ERP Layer ---
subgraph ERP["ERP Systems"]

```

```
    subgraph FERP["Foundation ERP (Optional)"]
        F1[HATEOAS-Native Modules]
        F2[Domain Models]
        F3[ERP Projections]
    end

    subgraph CERP["Commercial ERPs"]
        C1[Oracle Fusion]
        C2[MS Dynamics]
        C3[Odoo / ERPNext]
    end
end
```

```
%% --- Connections ---
```

```
%% UX <--> AI
UX1 --> NAV1
NAV2 --> UX2
NAV4 --> UX3
UX4 <-- LEDGER
```

```
%% AI <--> Governance + Authority + Process
NAV1 --> PROC
NAV2 --> GOV
NAV2 --> AUTH
NAV3 --> GOV
NAV3 --> AUTH
```

```
%% Governance <--> Process
GOV3 --> PROC2
```

```
%% AI <--> Ledger
NAV3 --> LED1
LED3 --> NAV5
LED4 --> NAV5
```

```
%% Ledger <--> Mesh
LED1 --> MESH1
MESH2 --> LED1
```

```
%% Mesh <--> Integration Hub
MESH1 --> HUB1
HUB1 --> MESH1
```

```
%% Integration Hub <--> ERP
HUB1 --> FERP
HUB1 --> CERP
FERP --> HUB1
CERP --> HUB1
```

✖ What This Architecture Diagram Shows

1. The UX is thin and declarative

It only displays:

- AI proposals
- governance prompts
- process affordances
- timeline + replay

It does **not** contain business logic.

2. The Navigator (AI) is the operational engine

It:

- reads process state
- proposes next actions
- executes transitions
- explains decisions
- participates in rollback/replay

But it is **constrained** by governance and authority.

3. The Charter Engine is the constitution

It enforces:

- immutable rules
- domain boundaries
- non-bypassable constraints
- transition validity

No AI or human can override it.

4. The Authority Engine manages trust

It determines:

- who can approve what
 - risk tiers
 - earned authority
 - revocation
-

5. The Process Graph defines the universe of possible actions

It exposes:

- states
- transitions
- hypermedia affordances

This drives the open-flow UX.

6. The Ledger is the temporal backbone

It provides:

- event sourcing
 - versioning
 - rollback
-

- replay
- reconstructability

7. The Mesh Fabric is the distributed substrate

It ensures:

- event propagation
- cross-node consistency
- resilience
- multi-ERP support

8. The Integration Hub is the bridge

It:

- orchestrates commands
- normalises APIs
- supports multiple ERPs
- enables ERP replacement

9. ERP systems become projections

They are:

- replaceable
- reconstructable
- secondary to the constitutional fabric

If you want, I can now produce:

- a **layered architecture diagram**
- a **sequence diagram** for a specific flow
- a **component diagram** for one subsystem
- a **deployment diagram** for mesh nodes
- a **data model diagram** for events, states, and authority

Just tell me where you want to go.

Hypermedia layer

flowchart LR

```
%% --- AI Navigator ---
```

```
subgraph AI["AI Navigator"]
```

```
    A1[Reads Process State]
```

```
    A2[Discovers Functions via MCP]
```

```
    A3[Executes Allowed Actions]
```

```
end
```

```
style AI fill:#E8F5E9,stroke:#43A047,stroke-width:2px,color:#1B5E20
```

```
%% --- MCP Layer ---
```

```
subgraph MCP["Model Context Protocol (MCP)"]
```

```
    M1[Function Catalog]
```

```
    M2[Schema Definitions]
```

```
    M3[Capability Discovery]
```

```

end
style MCP fill:#FFF3E0,stroke:#FB8C00,stroke-width:2px,color:#E65100

%% --- Hypermedia API ---
subgraph HYPER["Hypermedia REST API (HATEOAS)"]
    H1[State Representation]
    H2[Hypermedia Links]
    H3[Process Transitions]
end
style HYPER fill:#E3F2FD,stroke:#1E88E5,stroke-width:2px,color:#0D47A1

%% --- Integration Hub ---
subgraph HUB["Integration Hub"]
    I1[Publishes Hypermedia API]
    I2[Publishes MCP Functions]
    I3[Normalises ERP APIs]
    I4[Enforces Governance Hooks]
end
style HUB fill:#FFFDE7,stroke:#FDD835,stroke-width:2px,color:#F9A825

%% --- ERP Layer ---
subgraph ERP["ERP Subsystems (Hidden Behind Hub)"]
    E1[Oracle / Dynamics / Odoo]
    E2[Foundation ERP (Optional)]
end
style ERP fill:#E0E0E0,stroke:#616161,stroke-width:2px,color:#212121

%% --- Connections ---
AI --> MCP
MCP --> HYPER
HYPER --> HUB
HUB --> ERP

AI --> HYPER
AI --> HUB

```

Mermaid Sequence Diagram — Navigator Using MCP to Drive Execution

```
sequenceDiagram
    participant AI as AI Navigator
    participant MCP as MCP Catalog
    participant HYP as Hypermedia API
    participant GOV as Governance Layer
    participant HUB as Integration Hub
    participant ERP as ERP Systems

    AI->>MCP: Discover available functions (capabilities)
    MCP-->>AI: Function list + schemas

    AI->>HYP: Request current state (HATEOAS)
    HYP->>GOV: Apply governance filters
    GOV-->>HYP: Filtered transitions + constraints
    HYP-->>AI: Hypermedia state + allowed actions

    AI->>AI: Select action based on state + governance
    AI->>HUB: Execute action via MCP function
    HUB->>ERP: Normalised ERP command
    ERP-->>HUB: Execution result
    HUB-->>AI: Result + updated state link
```

Governance Modifying Hypermedia Responses (with Charter + Authority Engines)

```
flowchart LR
    AI[AI Navigator] --> HYP_REQ[Request Hypermedia State]
    HYP_REQ --> HUB[Integration Hub<br/>(Raw Hypermedia)]
    style HUB fill:#E3F2FD,stroke:#1E88E5,stroke-width:2px,color:#0D47A1
    HUB --> RAW[Raw Transitions<br/>+ State Representation]
    style RAW fill:#EEEEEE,stroke:#616161,stroke-width:1px,color:#212121
    RAW --> CHARTER[Charter Engine<br/>(Constitutional Rules)]
    style CHARTER fill:#FFF3E0,stroke:#FB8C00,stroke-width:2px,color:#E65100
    CHARTER --> AUTH[Authority Engine<br/>(Earned Authority + Risk Tiers)]
    style AUTH fill:#F3E5F5,stroke:#8E24AA,stroke-width:2px,color:#4A148C
    AUTH --> GOV_OUT[Governance-Filtered Hypermedia<br/>Allowed Actions + Constraints]
    style GOV_OUT fill:#E0F7FA,stroke:#00ACC1,stroke-width:2px,color:#006064
    GOV_OUT --> AI
    style AI fill:#E8F5E9,stroke:#43A047,stroke-width:2px,color:#1B5E20
```


Thank You

Provisia acknowledge with thanks the time of your team in clarifying our questions and once again express our gratitude for the opportunity provided. We now submit our proposal and await your feedback.

